

The SharpCorePDF Manual

VERSION 1.5.0 · .NET 10 AND .NET 11

Create, edit and read PDFs in .NET without a third-party engine: merge plans, page editing, authoring, text and layout extraction, signatures and PDF/A — behind a domain you can reason about.

- → The whole public API, from **MergePlan** to **MergeResult**
- → Page selections, output policies, encryption and document inspection
- → Editing, authoring a page, extraction in four formats and the object graph
- → Digital signatures, archival output, and a cookbook of recipes for services that ship documents

SharpCorePDF is published as the **SharpCorePDF** meta-package, with separate **SharpCorePDF.Domain**, **.Application**, **.Infrastructure** and **.DependencyInjection** packages on nuget.org.
This manual describes version 1.5.0 exactly.

Contents

Preface 7
- Who this manual is for 7
- How the manual is organised 7
- Conventions 8
- What you need 8

1. What SharpCorePDF Is 9
- One engine behind all of them 9
- What a merge actually does 10
- The guarantees you can build on 10
- The packages 10
- What "no third-party engine" means in practice 11

2. What SharpCorePDF Is Not 12
- It is not a page layout or reporting library 12
- It does not rewrite page content yet 12
- It is not a renderer, converter or OCR engine 12
- It is not a PDF/A or PDF/UA validator 13
- It is not a general-purpose PDF parser, but it does show you the file 13
- What is deliberately bounded inside a merge 13
- What is deliberately bounded inside inspection 14
- Not a service, not a CLI 14
- The short version 14

3. Installation and First Merge 16
- Install the package 16
- Create the project 16
- Register the services 16
- Merge 16
- Read the result 17
- Inspect before you merge 18
- A complete Program.cs 18
- Where to go next 19

4. The Merge Plan 20
- Building a plan 20
- Sources and identity 20
- Order is the merge 21
- The primary source 21
- Validation happens twice 22
- Executing a plan 22
- Reuse and thread safety 23

5. Page Selection 24
- The notation 24
- Building a selection without text 24
- Resolving against a document 25
- Preflight instead of exceptions 26
- Common shapes 26
- A note on bookmarks and links 27

6. Output Options 28
- Target version 28
- Choosing the primary source 29
- Document information 29

Bookmarks, labels and annotations 30
Reproducible output 30

7. Encryption and Permissions 32
Algorithms 32
Passwords 32
Permissions 32
Encrypted sources 34
Reproducibility and encryption 34

8. Inspection 35
The document model 35
What inspection is not 36
Encrypted and damaged documents 36
Reading what the pages contain 37
The result shape 38
Payloads and the inline budget 38
Reading the text 40
Warnings 41
Using inspection as a preflight 42

9. Errors, Exceptions and Warnings 43
The exception family 43
What throws when 43
Handling exceptions 44
Writing files safely 45
Warnings versus errors 45
Reproducing a failure 46

10. Dependency Injection 47
One call 47
Without the container 48
Three ways to configure the engine 48
Engine options 49
Configuration file 50
Logging and diagnostics 50
The producer string 50
Testing with the container 51

11. Architecture 52
The layers 52
The life of a merge 53
Why the plan is immutable 53
What the engine writes 53
The memory model 53
Boundaries and extension points 54
Concurrency 55

12. Authoring a Page 56
A page from nothing 56
Coordinates 56
Text and fonts 57
Images 57
Metadata, identity and reproducible output 58
An XMP packet, bookmarks and imported pages 58
What it deliberately does not do 59
The surface at a glance 59

13. Cookbook 61
Merge every PDF in a folder, in a defined order 61

Ask for a password only when one is needed 61
Extract a page range 62
Encrypt the result 63
Merge uploads in an ASP.NET Core endpoint 63
Report what was written 64
Override metadata and fix the modification date 64
Split a large job into batches 65
Accept a document only after checking it 66
Extract the text a document carries, and keep its coordinates 67
Write a cover page and put it in front of a document 67

14. Editing an Existing Document 70
One plan, applied in order 70
The operations 70
Overlays and underlays 71
Document-level edits 72
Output options 72
What survives an edit 73
Validation and failure 73
What an edit cannot change 74
Reading the result 74

15. Drawing 75
Coordinates, and the one name that misleads 75
Lines 75
Rectangles 75
Ellipses 76
Paths 76
Fill and stroke 76
Painting in a colour space of your own 77
Clipping 78
Transforms 78
The saved graphics state 79
What it deliberately does not do 79
The surface at a glance 79
Where to go next 80

16. Text Layout 81
Measuring a line 81
Metrics, and the line height they imply 81
Laying a paragraph out in a box 82
Drawing the layout 83
The layout result 84
Determinism 84
What it deliberately does not do 85
The surface at a glance 85

17. Image Formats 86
One call, four formats 86
PNG 86
JPEG 87
BMP 88
GIF 88
Natural size, placement and stretching 88
What a draw costs 88
Refusals 88
What this chapter does not do 89

18. Annotations 90

Adding one 90
Appearance streams 90
Updating and removing 91
What the editor reports 91
Where it is tested 91

19. Forms 93
The model 93
Writing a form 93
Appearances 94
Reset 94
Flatten 94
Refusals 95
What the engine reports 95
XFA 95
Reading the result back 95
Where it is tested 96
What this chapter does not cover 96

20. Extraction 97
The shape of a read 97
Letters 97
Words 98
Blocks, and the exports 99
One open, several questions 100
What an extraction is not 101
What an export says about itself 101

21. Layout Analysis 102
Why the two decisions are separate 102
Segmentation 102
Reading order 103
Keeping ink that is not prose 103
Comparing two readings 103
Where the analysis stops 104

22. The Object Graph 105
A node per object 105
References are reported, never followed 105
Streams without their bytes 106
The operators of a page 106
What the graph is for 107

23. Digital Signatures 108
Signing 108
The parts of the revision 109
Signing an archived document 109
Timestamps 110
Validating 110
What validation does not do 111
Reading the result 111
Evidence 111

24. Archival PDF (PDF/A) 113
Writing an archival document 113
What the writer writes 114
What is removed, and what is refused 114
Archival edits 115
Evidence 115
What this is not 116

25. How This Manual Is Built 117
The four stages 117
Markdown to HTML 117
The browser lays out, and nothing else 117
Measuring the contents 118
The footer, the outline and the metadata 118
Checking what was written 119
What the build does not do 119
Running it 120

Page numbers are the physical pages of this PDF; the cover is page 1. Every chapter starts on a new page. The running footer is added after printing.

Preface

SharpCorePDF is a PDF **creation, editing and reading** library for .NET. It merges PDF documents, reads the images, text, annotations, forms and bookmarks they already contain, writes fixed-position pages from nothing, and changes the assembly of an existing document — which pages it has, in what order and with what rotation. It does all of that without calling a native library, a command line tool or a third-party PDF engine.

This manual describes version **1.5.0** of the library. It is written for the developer who has to ship a feature, not only for the developer who wants to read source code: every chapter explains the behaviour you can rely on, the behaviour you should not rely on, and the code that exercises it.

Who this manual is for

You will get the most out of it if you are comfortable with C#, asynchronous methods and dependency injection. No knowledge of the PDF specification is assumed. Where a term from the specification matters — outline, page label, document information, security handler — the chapter explains it in the terms a caller needs, and stops there.

How the manual is organised

The first four chapters are the shortest path to a working merge:

- **Chapter 1** describes what the library does and the guarantees it makes.
- **Chapter 2** states, just as plainly, what it does not do.
- **Chapter 3** installs the package and merges three documents.
- **Chapter 4** explains the merge plan, the object at the centre of the API.

Chapters 5 to 10 cover one part of the API each: page selection, output options, encryption, inspection, errors and warnings, and dependency injection. Chapters 11 to 14 are for readers who want to understand the design, write a page that did not exist, need a recipe they can adapt, or change the assembly of an existing document: the architecture, authoring, a cookbook, then editing. Chapters 15 to 19 continue authoring: Chapter 15 is the shape surface — lines, rectangles, ellipses, paths, fill and stroke in a device or a named colour space, clipping and transforms — Chapter 16 is text layout, which measures a line, wraps a paragraph into a column and draws the result, Chapter 17 is the image formats, what is read out of each file and what each one is refused for, Chapter 18 is annotations, which are drawn over a page rather than into it, and Chapter 19 is forms: the field model, filling by name, reset and flatten.

Chapters 20 to 22 cover the reading surface that turns a page into data a program can use: extraction and its four export formats, the layout analysis that decides what a block is and which block is read first, and the object graph and content operators that describe the file itself. Chapters 23 and 24 cover the trust surface: digital signatures and archival (PDF/A) output. Chapter 25 turns the lens on this book: the converter that reads these chapters, the browser that lays the pages out, and the finishing pass this library writes.

This edition describes `1.5.0`, and every chapter describes what the package ships. Since `1.4.0` the changes a caller can see in the API are additive: a page can paint in a colour space the caller names (Chapter 15), the layout reader can read a page as horizontal row bands for a document known to be a table (Chapter 21), and a session keeps one parse open across several questions about the same document (Chapter 20). Chapters 16 to 19 were written while `1.1.0` was the package on the feed, and each of their heads used to say which part of its surface `1.1.0` already carried; they now describe the surface as it stands. The numbers are kept rather than reassigned so that a cross-reference in one edition keeps pointing at the same chapter in the next.

You do not have to read the chapters in order. Each one starts from a complete example and refers to earlier chapters only for background.

Conventions

Page numbers are one-based. Page 1 is the first page of a document, and every page number that crosses the API — in a page selection, in `PdfPageInfo.Number`, in an error message — is one-based. The PDF format itself numbers objects from zero internally; that detail never reaches you.

Code is complete enough to compile. Snippets omit `using` directives and surrounding class declarations when they are obvious, but every method call, property and argument name is the one in the documented API. The cookbook contains full methods for the recipes that need more than a fragment.

Warnings are not errors. A document that was repaired while being read still merges. A chapter that had to drop a bookmark still produces a result. Whenever the library makes such a decision it records a `PdfWarning`, and Chapter 9 explains which warnings mean what. Treat the `Kind` of a warning as the contract, never the wording of its message.

The manual is built from Markdown. The chapters in `docs/manual/src` are the source of both the GitHub documentation and the PDF. If you find a discrepancy between this text and the code, the code is the source of truth; please open an issue so the text can be corrected. Chapter 25 describes how those files become the PDF you are reading.

> **Version note.** This manual is written against 1.5.0. The public API is covered by an approved snapshot in each project, so a change to a public member is a deliberate, reviewed event rather than an accident of a refactoring. Code that compiles against 1.4.0 keeps compiling across this minor release — every addition is additive — and the changelog records every addition and every change of behaviour.

What you need

- The .NET SDK that can target **.NET 10** or **.NET 11**. The packages ship assemblies for both.
- A PDF to merge. Any reader can produce one; the tests use documents written by a different producer precisely so that the library is not tested only against its own output.
- For building this manual: the .NET SDK and Chrome or Edge, and nothing else. Chapter 25 describes the build. That is a maintainer concern, not a requirement of the library.

1. What SharpCorePDF Is

SharpCorePDF is a PDF **creation, editing and reading** library for .NET. It answers six questions:

1. *What is in this document?* — page count, page sizes, rotation, metadata, outlines, annotations, form fields, embedded files, encryption and the warnings the reader had to record.
2. *How do I produce one document from several?* — in the order I choose, with the pages I choose, under the output policies I choose.
3. *How do I write the one page that does not exist yet?* — a cover with a logo and text, fixed in place, written in memory and merged in front of the rest (Chapter 12).
4. *How do I change an existing document without editing its content?* — rotate, delete, move or duplicate pages and override document information, written to a new file (Chapter 14).
5. *What does this page say, and where does it say it?* — the letters, the words and the blocks, in the reading order the caller selects, with the links a page carries, exported as ALTO, PAGE XML, hOCR or SVG text (Chapter 20); the segmentation and order rules are Chapter 21, and the object graph and content operators behind them are Chapter 22.
6. *Can this document be trusted, and can it be archived?* — a signature appended as an incremental revision and validated with integrity and trust reported apart (Chapter 23), and an archival copy written to a PDF/A profile (Chapter 24).

Everything else in the API exists to make those operations predictable: an immutable plan, value objects for the inputs, read models for the outputs, typed exceptions for the failures, and warnings for the decisions that are not failures.

One engine behind all of them

Merging, inspection, authoring, editing and reading share the same reader and writer. A merge inspects every source anyway — it has to know the page count before it can resolve `"2-5"` — so inspection is not a separate code path with its own interpretation of a damaged file. If `InspectAsync` reports four pages, the merge that follows sees the same four pages and the same warnings. An authored page is written with the merge engine's writer and returned as a `PdfByteArraySource`, so it enters the merge through the same `IPdfContentSource` port as a file or an upload. An edit reads the source, applies its operations to the page tree and writes a complete new document, with the same copier the merge uses for every object a surviving page needs.

```C#
using SharpCorePDF.Application.Extensions;
using SharpCorePDF.Domain.Aggregates;
using SharpCorePDF.Infrastructure.Extensions;

var plan = MergePlan.Create()
    .AddFile("quarterly-cover.pdf")
    .AddFile("quarterly-body.pdf", pages: "1-12")
    .AddFile("quarterly-appendix.pdf", displayName: "Appendix");

var result = await merger.MergeToFileAsync(plan, "quarterly-report.pdf");
```

The call above produces a new file and returns a `MergeResult`:

```TEXT
18 pages from 3 sources, written as PDF 1.7, 421,338 bytes, in 41 ms
Primary source: quarterly-cover.pdf
Warnings: none
```

The result is a *summary*, not the document. It tells you how many pages were written, how long it took, which source led the merge, what each source contributed and which warnings the operation produced. That is enough for a log line, an audit record or a UI that says "18 pages, 3 documents".

What a merge actually does

A merge is not a concatenation of files. The engine:

- reads each source, following the page tree to its leaves and resolving indirect objects;
- copies every object the selected pages need into a new document, rewriting references so that the copies refer to each other and not to the source file;
- builds a new page tree in source order, preserving each page's size, rotation, content, resources and annotations;
- merges bookmarks into one outline whose top-level entries follow source order;
- applies the output policies: document information, page labels, form fields, embedded files, encryption and the PDF version written in the header;
- writes a complete file with a cross-reference table and a trailer.

The source files are only ever read. A merge or an edit does not modify them, does not need write access to them, and does not leave temporary files beside them.

The guarantees you can build on

The plan is immutable. Every method that changes a plan returns a new plan; nothing mutates an existing one. A plan can be built at start-up, validated once, shared between requests and executed concurrently without a lock. Two plans with the same sources and options are equal.

Plans are validated before work starts. `Validate()` reports every problem it can see without opening a document — no sources, a duplicate identity, an unsupported target version, an encryption algorithm that the target version cannot carry. `EnsureValid()` turns those problems into a single `MergePlanValidationException`. A bad plan fails before any output file is created.

Failures are typed. A malformed source, a missing password, an unsupported feature and an invalid plan each have their own exception type, all derived from `PdfException`. You can catch the base type for a coarse handler or the derived types for a precise one.

Recoverable problems are warnings. The reader rebuilds a missing cross-reference table, skips an unreadable bookmark and assumes a page size when the page tree does not state one. Each decision is recorded as a `PdfWarning` with a stable `Kind`, so a caller can decide whether the document is good enough without parsing error text.

Cancellation is honoured. Every asynchronous operation takes a `CancellationToken`. Cancellation throws `OperationCanceledException`, the .NET convention, and does not masquerade as a PDF error.

No native dependency. The package is managed code. There is no `pdfium`, no `qpdf`, no executable to deploy, no platform-specific runtime to select and nothing to install on the host beyond .NET.

The packages

The installable package is `SharpCorePDF`. It is a meta-package: it carries no assemblies of its own and depends on the four layers, so one `dotnet add package SharpCorePDF` is the whole install.

Package	Contains	Reference it when
`SharpCorePDF.Domain`	Plan, sources, value objects, read models, ports, exceptions, warnings	You build plans in a domain project that must not depend on the engine
`SharpCorePDF.Application`	The use cases behind the ports — merge, edit, inspect, read, sign, validate, archive — and the convenience extensions	You need the services without the concrete engine
`SharpCorePDF.Infrastructure`	Reader, engine, file and memory sources, engine options, the authoring builder, the signer and the archival writer	You construct the engine yourself instead of using DI

Package	Contains	Reference it when
`SharpCorePDF.DependencyInjection`	`AddSharpCorePDF()` and configuration binding	You register the library in a `IServiceCollection`
`SharpCorePDF`	Dependencies on all four	You just want the library

The layers exist so that a solution can keep its domain free of infrastructure, not so that every consumer has to think about them. If you are unsure, install `SharpCorePDF`.

What "no third-party engine" means in practice

The PDF parser, the object copier, the outline merger, the encryption writer and the cross-reference writer are implemented in this repository. The runtime dependencies of the library projects are the `Microsoft.Extensions.*` abstractions used by the dependency injection package; there is no PDF library underneath.

That has consequences worth knowing before you adopt it:

- The feature set is deliberately bounded. Chapter 2 lists the boundary.
- The writer emits classic cross-reference tables and requires PDF 1.4 or newer. It does not write incremental updates or linearised files.
- The reader is written to accept the shapes real producers emit — object streams, cross-reference streams, damaged tables, the six revisions of the standard security handler — and to say so in a warning when it has to repair something.

If your requirement is "merge these documents and tell me what you had to fix", that trade is a good one. If your requirement is a general PDF toolkit, Chapter 2 will tell you quickly whether this is the wrong library.

2. What SharpCorePDF Is Not

A precise boundary is part of the API. This chapter states what the library does not do, why, and what to use instead. If one of these lines is a requirement, SharpCorePDF is the wrong tool and no amount of configuration will change that.

It is not a page layout or reporting library

SharpCorePDF can start a document from a blank page: `PdfDocumentBuilder` adds pages of a chosen size, draws lines of text in a standard or an embedded TrueType font, places images at fixed positions and draws fixed-position vector graphics — lines, rectangles, ellipses, arbitrary paths, dashed strokes, transparency and clipping (Chapter 12, and Chapter 15 for the shape surface in full). It can also wrap a paragraph into a column, align or justify each line and draw it in one call (Chapter 16). That is enough for a cover page, an address block, a form-shaped frame or a fixed marker, and it is where authorship stops. There is no **page flow**: no table or list layout, no page breaks, no columns, and no hyphenation — a block that does not fit its column is refused, broken by character, clipped or continued, and a block that does not fit its page runs off it. Text is not shaped beyond what the font program itself does; the image formats are the four Chapter 12 covers, with SVG and the rest re-saved by the caller; and no font is ever read from the machine the code runs on. The merge side still receives documents as a file, a byte array, a stream or a custom `IPdfContentSource`, copies the pages you select and combines them.

Use instead: a reporting or layout library that produces PDF when a report has to flow across pages, lay out a table or hyphenate its text. SharpCorePDF can merge those reports afterwards and can add its own fixed-position cover.

It does not rewrite page content yet

SharpCorePDF can change the **assembly** of an existing document: rotate a page, delete it, move it, duplicate it, insert an authored page, crop the area a page shows, and override the document information (Chapter 14). It can also add, edit and remove annotations (Chapter 18), and create, fill, reset and flatten form fields (Chapter 19) — structures stored beside a page rather than inside it. What it cannot do *in this release* is change what a page already draws. There is no API to replace text or an image, add a redaction, or change a page's media box or its size. Page content and page geometry that survive an edit are copied verbatim.

Signing and archival output are separate surfaces that change the file rather than its assembly: a signature appends one revision to the bytes it is given (Chapter 23), and the archival writer states the structures a PDF/A profile requires (Chapter 24). Neither rewrites what a page paints.

The 1.1 parity plan extended editing to page content, annotations, forms and page geometry. Page geometry and the page-level operations shipped in `1.1.0`; annotation and form authoring, text layout and the remaining image formats shipped in `1.2.0`, and an edit can now record a page of another document over or under a page of the source. The releases since stayed inside that boundary: `1.3` to `1.5` widened **reading** — the reusable session, the row-band order, MacRoman-encoded fonts — and **drawing** (a colour space the caller names) rather than the editing model. What an edit still does not do is rewrite the operators a page already carries: a layer is added beside them. The capabilities that stay outside the product at every release — rendering, conversion, OCR, conformance validation, services and CLIs — are the sections below.

Use instead: a viewer/editor SDK for interactive editing, and a redaction product when content has to be removed. If the change is which pages the document contains, in what order, with what rotation and within what visible area, or which annotations and field values it carries, that is what an edit plan does.

It is not a renderer, converter or OCR engine

The library does not rasterise pages, does not render them to an image, does not convert PDF to Word, HTML or a reflowed document, and does not recognise scanned characters. It does read the text a document draws (Chapter 8): the glyphs, the words they make and the blocks those words make, each with its coordinates — what the file already says, not what a picture of it might say. That is enough for search over a born-digital document and not enough for a scan.

Use instead: a rendering engine for thumbnails and previews, an OCR product for scans, and a layout-aware conversion product when the output has to be a reflowed document rather than text with coordinates.

It is not a PDF/A or PDF/UA validator

The output is a well-formed PDF in the version you ask for. It is not validated against PDF/A, PDF/X, PDF/UA or any accessibility profile, and the library does not claim conformance. Writing *for* a profile is a different thing from validating one: `IPdfAWriter` states the structures PDF/A-1b, -2b or -3b requires — a stated version, an XMP identification, an output intent, embedded font programs, no encryption — and removes or refuses what the profile forbids, reporting each decision as a warning or a refusal (Chapter 24). What it does not do is certify the result, and a tagged structure tree for PDF/UA is not copied or rebuilt.

Use instead: a dedicated validator when a compliance profile is a contractual requirement — veraPDF for PDF/A, for example. Use the archival writer to make the document as close as the API allows, then validate.

It is not a general-purpose PDF parser, but it does show you the file

Inspection reports what a merge needs and what an operator wants to know before merging: the page count, page geometry, document information, outline size, embedded files, form presence, encryption state and warnings. Above that, the reading surface publishes the file's own structures as read-only projections:

- `IPdfContentReader` hands back the images, text, annotations, forms, bookmarks, attachments and XMP a document carries (Chapter 8), and `IPdfLayoutReader` the letters, words and blocks it draws, exportable as ALTO, PAGE XML, hOCR or SVG (Chapters 20 and 21).
- `IPdfObjectInspector` walks the object graph — one node per listed object, references reported rather than followed — and lists the operators a page's content stream states (Chapter 22).

What it is not is a mutable object model: there is no way to edit an object and write the document back, nothing here is a round-trip parser, and fonts are not enumerated as a first-class API — image metadata is reported per image, not as a document-level index.

Use instead: a general-purpose PDF object model for mutation and round-tripping. SharpCorePDF reads what it can describe and writes documents it assembled itself.

What is deliberately bounded inside a merge

These are not missing features in the sense above; they are decisions about what a merged document can sensibly contain. The library makes them explicit, and where a decision loses information it records a warning.

Area	Behaviour
Document-level features	Metadata follows `MetadataPolicy` (the primary source by default); viewer preferences, page layout and mode, form fields, named destinations and embedded files come from the **primary source**, because they cannot sensibly exist twice in one document
Page-level features	Content, resources, annotations, page size and rotation are merged from **every** source
Editing	An edit plan rotates (additively), deletes, moves and duplicates pages, and overrides document information. Page content and geometry are copied, not changed. Page labels are dropped when the page order or count changes, and XMP is dropped when document information is overridden; both are reported as warnings
Outlines	Merged into one tree, top-level entries in source order; bookmarks whose destination cannot be resolved are skipped with a warning. An edit redirects a destination to the first surviving page it named
Page labels	Dropped by default, because labels are tied to absolute page indices and misalign when sources are reordered or partially selected
AcroForm fields	The interactive form of the primary source is carried over; fields of other sources remain visible as page annotations but are not part of the merged form
Embedded files	Carried over from the primary source only

Area	Behaviour
Named destinations	Carried over from the primary source; destinations in other sources are dropped with a warning
Encryption	Read: standard security handler revisions 2–6. Written: RC4-40, RC4-128, AES-128, AES-256. Public-key encryption is not supported
Output file structure	Classic cross-reference table, complete rewrite, PDF 1.4–2.0, no linearisation and no object streams in the output. A signature is the one exception: it appends a single incremental revision (Chapter 23)
Digital signatures	Signature fields and their widget annotations may be copied with the primary form or the pages, but a copied signature is not a valid signature of the merged document. The library signs a document it is given as a separate, explicit step (Chapter 23); a merge does not

> **The signature point deserves its own sentence.** If you merge a signed document, the result is a new document that no signer signed. Do not present the output as signed. Validate signatures before merging if their presence matters to your workflow.

What is deliberately bounded inside inspection

- Encrypted documents that cannot be opened are **described**, not thrown: `IsReadable` is `false`, `RequiresPassword` says whether a password would help, and inspection still returns a value. A merge of the same source throws `PdfEncryptionException` because it cannot proceed.
- Damaged documents are repaired where repair is safe and reported through warnings. When repair is not possible, inspection throws `PdfSourceFormatException` instead of returning a half-truth.
- Inspection reads the whole source into memory (bounded by `MaxSourceBytes`). It is a document-level operation, not a streaming scan.

Not a service, not a CLI

The repository contains a library and examples, not a hosted API, a command line tool or a Windows shell extension. A service that merges documents is a few dozen lines around `IPdfMerger`; the cookbook shows one.

The short version

If you need to…	SharpCorePDF
Combine existing PDFs, all pages or a selection	Yes
Reorder or repeat pages from a source	Yes
Rotate, delete, move, duplicate, insert or crop pages in an existing document	Yes
Override the title, author or other document information	Yes
Preserve bookmarks, annotations and metadata	Yes
Encrypt the merged result	Yes
Learn page counts, sizes, outline size or encryption state before merging	Yes
Read the images, text, comments, form fields and bookmarks a document carries	Yes
Create a fixed-position page from scratch: text, PNG, JPEG, BMP and GIF images, vector graphics, standard or embedded fonts	Yes
Draw lines, paths, ellipses, clipping and transforms on a page (Chapter 15)	Yes
Sign a document, or validate a signature's integrity and trust	Yes
Write for PDF/A-1b, -2b or -3b	Yes
Read letters, words, blocks and reading order; export ALTO, PAGE XML, hOCR or SVG	Yes

If you need to…	SharpCorePDF
Walk the object graph and the operators of a page, read-only	Yes
Lay out a report: page flow, tables, columns, hyphenation, JPEG images	No
Wrap and align a paragraph inside a fixed box (Chapter 16)	Yes
Add, edit or remove annotations (Chapter 18)	Yes
Create, fill, reset and flatten form fields (Chapter 19)	Yes
Change a page's size or media box, or replace the text or images a page draws	No
Redact, or rebuild tagged structure	No
Render, convert, OCR or reflow text	No
Certify PDF/A or accessibility conformance	No

3. Installation and First Merge

This chapter takes an empty console project to a merged file and an inspection report. It should take about five minutes.

Install the package

```shell
dotnet add package SharpCorePDF --version 1.5.0
```

One package is enough. `SharpCorePDF` is a meta-package that brings the domain model, the application services, the engine and the dependency injection registration together. The four layer packages remain installable on their own — see Chapter 1 for when that is useful — but an application that merges documents does not need to choose.

The container is the one thing that package leaves to you. `SharpCorePDF.DependencyInjection` references `Microsoft.Extensions.DependencyInjection.Abstractions` — the assembly `AddSharpCorePDF()` extends and the one `IServiceCollection` lives in — while `ServiceCollection` and `BuildServiceProvider()` are in `Microsoft.Extensions.DependencyInjection`. An application that builds a provider itself adds that package:

```shell
dotnet add package Microsoft.Extensions.DependencyInjection
```

An application that runs inside an ASP.NET Core or Worker host already has it, which is why no line below mentions it again. Nothing else in this manual needs a second package, and Chapter 10 shows the manual wiring for a caller who wants no container at all.

The library targets **.NET 10** and **.NET 11**. A project on either target framework can reference it; the package selects the matching assembly.

Create the project

```shell
dotnet new console -n MergeDemo
cd MergeDemo
dotnet add package SharpCorePDF --version 1.5.0
```

Put two or more PDFs beside the project, or change the paths below. The example uses `cover.pdf`, `body.pdf` and `appendix.pdf`.

Register the services

The shortest correct wiring is a service collection. It costs four lines and gives you the merger, the inspector and the engine behind their ports, all as singletons.

```C#
using Microsoft.Extensions.DependencyInjection;
using SharpCorePDF.DependencyInjection;
using SharpCorePDF.Domain.Ports;

var services = new ServiceCollection()
    .AddSharpCorePDF()
    .BuildServiceProvider();

var merger = services.GetRequiredService<IPdfMerger>();
var inspector = services.GetRequiredService<IPdfInspector>();
```

`AddSharpCorePDF()` has three overloads: no arguments (default engine options), an explicit `PdfEngineOptions`, and an `IConfiguration` plus an optional section name. Chapter 10 covers all three and the options themselves.

You do not have to use dependency injection. The infrastructure types have public constructors, and Chapter 10 shows the manual wiring. The rest of this manual uses the container because it is the supported path for an application.

Merge

Add the following to `Program.cs`. The merge writes the selected pages of three documents, in the order they are added, to `merged.pdf`.

```C#
using SharpCorePDF.Application.Extensions;
using SharpCorePDF.Domain.Aggregates;
using SharpCorePDF.Infrastructure.Extensions;

var plan = MergePlan.Create()
    .AddFile("cover.pdf")
    .AddFile("body.pdf", pages: "1-3,7")
    .AddFile("appendix.pdf", displayName: "Appendix", pages: "2-");

var result = await merger.MergeToFileAsync(plan, "merged.pdf");

Console.WriteLine($"{result.PageCount} pages, {result.OutputBytes} bytes, {result.Duration.TotalMilliseconds:F0}
ms");
Console.WriteLine($"Primary source: {plan.FindSource(result.PrimarySourceId)?.DisplayName}");
```

Run it:

```SHELL
dotnet run
```

Expected output, with different timings and sizes:

```TEXT
6 pages, 184213 bytes, 37 ms
Primary source: cover.pdf
```

Three things are worth noticing in that snippet.

The page selection is a string. `"1-3,7"` means pages 1, 2, 3 and 7, in that order. `"2-"` means page 2 to the end. The notation is explained in Chapter 5; `null` or an omitted argument means every page.

The display name is for humans. It defaults to the file name and appears in `MergeResult`, in the source summaries and in messages. It does not have to be unique and it does not affect the output.

The extension methods come from two namespaces. `AddFile`, `AddBytes` and `AddStream` live in `SharpCorePDF.Infrastructure.Extensions`; `MergeToFileAsync` and `MergeToBytesAsync` live in `SharpCorePDF.Application.Extensions`. A missing `using` for either produces a compiler error that mentions the receiver type, not the namespace.

Read the result

`MergeResult` is the record of what happened. The properties used most often are:

Property	Meaning
`PageCount`	Pages written to the output
`SourceCount`	Number of sources that contributed at least one page

Property	Meaning
`OutputBytes`	Size of the merged document
`OutputVersion`	The PDF version written in the header
`Duration`	Wall-clock time spent in the merge
`PrimarySourceId`	The source that supplied document-level features
`Sources`	One `MergedSourceSummary` per input, in plan order, with contributed page count
`Warnings`	Every warning recorded while reading, merging or writing

The destination is not repeated in the result: the caller already has the path or the stream it passed in. Use `plan.FindSource(result.PrimarySourceId)` when you need the display name of the primary source, as in the example above.

If `Warnings` is empty, nothing had to be repaired or dropped. If it is not empty, the merge still succeeded; Chapter 9 tells you how to triage the warning kinds.

Inspect before you merge

Inspection uses the same reader. Use it to check a document before adding it to a plan, or to display a summary to a user.

C#

```
using SharpCorePDF.Infrastructure.Sources;

var info = await inspector.InspectAsync(new PdfFileSource("body.pdf"));

Console.WriteLine($"{info.PageCount} pages, PDF {info.Version}, encrypted: {info.IsEncrypted}");
Console.WriteLine($"Readable: {info.IsReadable}, warnings: {info.Warnings.Count}");

foreach (var page in info.Pages.Take(3))
{
    Console.WriteLine(
        $"  page {page.Number}: {page.Size.WidthPoints:F0} x {page.Size.HeightPoints:F0} pt, " +
        $"rotation {page.Rotation.Degrees}");
}
```

`PdfFileSource` is a small `IPdfContentSource` that opens the file when it is read. `InspectAsync` has an optional password parameter — `InspectAsync(source, password, cancellationToken)` — which Chapter 8 covers together with the rest of the read models.

A complete `Program.cs`

The pieces above, assembled and made a little more careful: it validates the plan before doing any work, prints warnings, and returns a non-zero exit code if the merge fails.

C#

```
using Microsoft.Extensions.DependencyInjection;
using SharpCorePDF.Application.Extensions;
using SharpCorePDF.Domain.Aggregates;
using SharpCorePDF.Domain.Exceptions;
using SharpCorePDF.Domain.Ports;
using SharpCorePDF.DependencyInjection;
using SharpCorePDF.Infrastructure.Extensions;

var services = new ServiceCollection().AddSharpCorePDF().BuildServiceProvider();
var merger = services.GetRequiredService<IPdfMerger>();

var plan = MergePlan.Create()
    .AddFile("cover.pdf")
    .AddFile("body.pdf", pages: "1-3,7")
    .AddFile("appendix.pdf", displayName: "Appendix");
```

```
try
{
    plan.EnsureValid();

    var result = await merger.MergeToFileAsync(plan, "merged.pdf");
    Console.WriteLine($"Wrote {result.PageCount} pages, {result.OutputVersion}.");

    foreach (var warning in result.Warnings)
    {
        Console.WriteLine($"warning {warning.Kind}: {warning.Message}");
    }

    return 0;
}
catch (PdfException exception)
{
    Console.Error.WriteLine($"{exception.GetType().Name}: {exception.Message}");
    return 1;
}
```

Two details in that listing are deliberate. `EnsureValid()` runs before the destination file is created, so an invalid plan cannot leave a truncated file behind. And the `catch` is for `PdfException`, the base type of every library failure, rather than `Exception`; an `OperationCanceledException` or an `OutOfMemoryException` is not a PDF problem and should not be reported as one.

Where to go next

- To control which pages come from which document, read Chapter 5.
- To set metadata, version, bookmarks or encryption, read Chapters 6 and 7.
- To decide what to do when a source is damaged or password-protected, read Chapters 8 and 9.
- To wire the library into an existing application, read Chapter 10.
- To write a cover page from nothing and merge it in front, read Chapter 12.

4. The Merge Plan

Everything the library does with several documents is described by a `MergePlan`: an ordered list of sources and one set of output options. A plan is a value. It is not a builder that gets mutated, not a job that gets executed, and not a handle that has to be disposed.

```C#
var plan = MergePlan.Create()
    .AddFile("cover.pdf")
    .AddFile("chapter-1.pdf", pages: "1-")
    .AddFile("chapter-2.pdf", pages: "1-")
    .WithOutput(options => options with { TargetVersion = PdfVersion.V1_7 });
```

That single expression states the order of the merge, what each source contributes and how the output must be written. The next chapters unpack the three parts; this one explains how they fit together.

Building a plan

`MergePlan.Create()` starts an empty plan. Every method that changes it returns a new plan, so the usual style is to chain calls and keep the result.

Member	Purpose
`Create(MergeOutputOptions? output = null)`	Starts a plan, optionally with output options
`For(params PdfSource[] sources)`	Starts a plan from sources you built yourself
`AddSource(PdfSource source)`	Appends a source entity
`AddSources(IEnumerable<PdfSource> sources)`	Appends several sources, in enumeration order
`RemoveSource(PdfSourceId id)`	Returns a plan without the source with that identity
`WithOutput(MergeOutputOptions output)`	Replaces the output options
`WithOutput(Func<MergeOutputOptions, MergeOutputOptions> mutate)`	Adjusts them with a `with` expression
`WithPrimarySource(PdfSourceId? id)`	Sets, or clears, the primary source
`FindSource(PdfSourceId id)`	Returns the source, or `null`
`Validate()`	Returns every problem that can be found without reading a document
`EnsureValid()`	Throws `MergePlanValidationException` if there is one
`Sources`	The sources, in merge order
`PrimarySource`	The source that supplies document-level features
`SourceCount`	The number of sources

The convenience methods `AddFile`, `AddBytes`, `AddStream` and `AddContent` are extension methods from `SharpCorePDF.Infrastructure.Extensions`. They create the matching content source and call `AddSource`, and they accept the page selection in its textual notation. The core `AddSource` is enough when you already have a `PdfSource`.

Sources and identity

A `PdfSource` has four pieces of state:

- **`Content`** — an `IPdfContentSource`, the object that can open the document for reading.

- `DisplayName` — the name used in messages and in `MergeResult`. It defaults to the content source's own name and can be changed with `Rename`.
- `Pages` — a `PageSelection`; `PageSelection.All` when nothing was specified.
- `Password` — the user or owner password for an encrypted document, or `null`.

It also has an `Id`, a `PdfSourceId` generated when the source is created. The identity is what lets a plan refer to a source later — for example as the primary source — without relying on its position, and it is what validation uses to detect an accidental duplicate.

Two `AddFile` calls for the same path produce two sources with two identities. That is legal and is how you repeat a document:

```C#
var plan = MergePlan.Create()
    .AddFile("terms.pdf", displayName: "Terms (before)")
    .AddFile("body.pdf")
    .AddFile("terms.pdf", displayName: "Terms (after)");
```

The duplicate that SharpCorePDF rejects is the same *identity* appearing twice, which can only happen if you pass one `PdfSource` instance to `AddSource` twice or supply an explicit identity through `PdfSource.Create(id, content, …)`. `AddSource` throws `MergePlanValidationException` immediately; it is a programming error rather than a use case, and it fails before any document is read.

Sources also support non-mutating edits of their own:

```C#
var source = PdfSource.Create(new PdfFileSource("report.pdf"))
    .WithPages(PageSelection.FromTo(2, 5))
    .Rename("Report, pages 2 to 5");
```

`WithPages`, `WithPassword` and `Rename` keep the identity, so a plan that already refers to the source as its primary source keeps working after the edit.

Order is the merge

The order of `Sources` is the order of the output. There is no sorting, no grouping by path and no implicit title page. Pages contributed by the first source come first, then those of the second, and so on. Within one source, the page selection decides the order, which is why `"3,1"` is a legal selection and produces page 3 before page 1.

This makes a plan readable as a table of contents. If you need a different order, build a different plan; reordering is intentional, not an optimisation the library performs behind your back.

The primary source

Some things can only exist once in a PDF: the document information dictionary, viewer preferences, page layout and mode, the interactive form, named destinations, embedded files. When sources disagree, the merge must choose one of them. It chooses the **primary source**.

By default the primary source is the first source in the plan. `WithPrimarySource` changes it:

```C#
var cover = plan.Sources[0];
var body = plan.Sources[1];

var withBodyMetadata = plan.WithPrimarySource(body.Id);
```

The primary source is usually the one whose identity matters most — the cover whose title should survive, or the document whose form fields the user expects to keep working. It does not have to be the first source, and setting it does not change the order of the pages. Chapter 6 lists exactly which output options follow the primary source and which can override it.

`PrimarySource` throws `MergePlanValidationException` for an empty plan, and for an id that is not part of the plan. Both are also caught by `Validate`.

Validation happens twice

`Validate` checks what can be known without opening a document:

- the plan has at least one source;
- no two sources share an identity;
- every source has a non-empty display name;
- the output options are internally consistent — a supported target version, a known encryption algorithm, a primary source that belongs to the plan.

`MergeAsync` calls `EnsureValid` itself, so a plan that was never validated still cannot reach the engine. Validation then continues against the real documents: each source is opened, its page count is known, and a selection such as `"9-12"` against a four-page document fails with `MergePlanValidationException` before the output is written.

Calling `EnsureValid` yourself is still worthwhile when you want the failure to happen before you create a destination file, or when you want to report all problems at once:

```C#
var problems = plan.Validate();

if (problems.Count > 0)
{
    throw new MergePlanValidationException(problems);
}
```

The exception's `Errors` property preserves the individual messages. They are written for humans and are not part of the compatibility contract; do not parse them.

Executing a plan

There are four ways to run a plan, all of them thin wrappers over the same engine:

Call	Destination	Returns
`merger.MergeAsync(plan, stream, ct)`	A writable stream you own	`MergeResult`
`merger.MergeToFileAsync(plan, path, ct)`	A file, created or overwritten	`MergeResult`
`merger.MergeToBytesAsync(plan, ct)`	An in-memory buffer	`byte[]`
`merger.MergeAsync(sources, stream, ct)`	A stream, from a list of content sources	`MergeResult`

`MergeToFileAsync` creates the destination directory when it does not exist, opens the file with `FileMode.Create` and exclusive sharing, and lets the merge write into it. The stream overload flushes the destination but does not close it: the caller owns the stream. `MergeToBytesAsync` is convenient for an HTTP response, but it holds the whole result in memory; use a file or a stream for large documents.

The list overload is a shorthand when every page of every source is wanted:

```C#
var result = await merger.MergeAsync(
    new IPdfContentSource[]
    {
        new PdfFileSource("a.pdf"),
        new PdfFileSource("b.pdf"),
    },
    destination);
```

It builds a plan with default options. As soon as you need a page selection, a primary source or non-default output options, build the plan explicitly; the shorthand does not grow parameters for those.

> **Cancellation leaves a partial destination.** A cancelled `MergeToFileAsync` throws `OperationCanceledException`, and the file it was writing may be incomplete. If the destination must never be observed half-written, merge to a temporary file in the same directory and move it into place after the call succeeds. Chapter 13 shows the pattern.

Reuse and thread safety

A plan never changes after it is built, so it can be stored, compared, cached and shared. The built-in content sources open a fresh read for every execution: `PdfFileSource` opens the file again, `PdfByteArraySource` returns a new read-only `MemoryStream` over the same bytes, and `PdfStreamSource` captures the stream once, under a lock, and serves every later read from the captured bytes. A plan whose sources are one of these can be executed concurrently.

A custom `IPdfContentSource` must honour the same contract — every call to `OpenReadAsync` returns an independent readable stream — if you intend to reuse or share its plan. If it cannot, create a new source, and therefore a new plan, per execution.

The merger itself is thread safe. The container registers it as a singleton, and the engine keeps no per-merge state on the instance.

5. Page Selection

A page selection says which pages of one source document enter the merged document, and in what order. It is the only place where the library lets you rearrange pages, and it is deliberately small: a comma-separated list of one-based ranges.

```C#
var plan = MergePlan.Create()
    .AddFile("report.pdf", pages: "1,3-5,8-")
    .AddFile("appendix.pdf", pages: "2,1");
```

The notation

Text	Meaning	Pages contributed from a 10-page source
`"1"`	One page	1
`"2-4"`	A closed range, inclusive	2, 3, 4
`"3-"`	From a page to the last page	3, 4, 5, 6, 7, 8, 9, 10
`"-2"`	From the first page to a page	1, 2
`"1,3-5,8-"`	Several ranges, in this order	1, 3, 4, 5, 8, 9, 10
`"3,1,3"`	Out of order, with a duplicate	3, 1
`" 2 - 4 "`	Whitespace around numbers is ignored	2, 3, 4

The rules behind the table:

- **Pages are one-based.** Page 1 is the first page a reader shows. PDF's internal zero-based index never appears in this API.
- **Ranges are inclusive at both ends.** `2-4` contains 2, 3 and 4.
- **A range must not run backwards.** `5-2` is a `FormatException`, not an empty selection.
- **A range must start at page 1 or later.** `0-3` is a `FormatException`.
- **Order is preserved.** Ranges are read left to right, and pages inside a range ascend. `"3,1"` puts page 3 before page 1.
- **Duplicates collapse on first occurrence.** `"3,1,3"` selects page 3, then page 1; the second 3 is dropped because it was already selected. Repeating a page is not how you duplicate it — add the source twice instead (Chapter 4).
- **A closed range may pass the last page; an open range may not start past it.** `"2-99"` on a four-page document contributes pages 2, 3 and 4 to the merge, and `"9-"` fails because it starts after the last page. `Resolve` itself still returns 2 to 99, so a preview must drop numbers greater than `PageCount`; the merge does that when it copies pages. A closed range that starts inside the document never throws for its tail.
- **The selection counts physical pages, not page labels.** If a document displays "iv" on its fourth physical page, select `"4"`, not `"-iv"`. Page labels are a presentation feature and are dropped by default anyway (Chapter 6).

Whitespace around the whole text and around each number is accepted. An empty string is not a valid selection: `PageSelection.Parse("")` throws `FormatException`. The `AddFile` extension treats `null`, `""` and whitespace as "every page" because it checks before calling `Parse`; if you call `PageSelection.Parse` yourself, handle the empty case explicitly.

Building a selection without text

Text is a convenience. The value objects are public, and a selection can be built directly when it comes from a UI, a database or a calculation.

```C#
using SharpCorePDF.Domain.Entities;
using SharpCorePDF.Domain.ValueObjects;
using SharpCorePDF.Infrastructure.Sources;

PageSelection firstThree = PageSelection.FromTo(1, 3);
PageSelection appendix = PageSelection.FromPage(12);
PageSelection coverAndBack = PageSelection.Of(
    PageRange.Single(1),
    PageRange.ToPage(2),
    PageRange.FromTo(8, 9));
PageSelection everything = PageSelection.All;

var source = PdfSource.Create(new PdfFileSource("report.pdf"), pages: coverAndBack);
```

Factory	Produces
`PageSelection.All`	`1-`
`PageSelection.Single(page)`	`page`
`PageSelection.FromTo(from, to)`	`from-to`
`PageSelection.FromPage(from)`	`from-`
`PageSelection.Of(params PageRange[])`	The ranges in the given order
`PageRange.Single(page)`	One range of one page
`PageRange.FromTo(from, to)`	A closed range
`PageRange.FromPage(from)`	An open-ended range
`PageRange.ToPage(to)`	A range from page 1
`PageRange.All`	`1-`
`PageSelection.Parse(text)` / `TryParse`	A selection from the notation
`PageRange.Parse(text)` / `TryParse`	A single range from the notation

`PageSelection` is immutable. `Add` appends a range and `Concat` appends another selection, both returning a new instance. `Ranges` exposes the ranges, `RangeCount` their number and `IsAll` tells you whether the selection is the `1-` shortcut.

Resolving against a document

A selection is document-independent until it is resolved. `Resolve(pageCount, sourceName)` returns the concrete page numbers, in merge order and without duplicates, and throws `MergePlanValidationException` when the document has no pages or a range starts past its end.

```C#
var info = await inspector.InspectAsync(new PdfFileSource("report.pdf"));
var selection = PageSelection.Parse("1,3-5,8-");

IReadOnlyList<int> pages = selection.Resolve(info.PageCount, info.SourceName);

Console.WriteLine(string.Join(", ", pages));
```

You rarely call `Resolve` in application code. The merger resolves every selection while it prepares the merge, before the engine writes pages, and the same resolution rules apply. Calling it yourself is useful for a preview — "this plan will produce 14 pages" — or for rejecting a request before a destination file exists.

Two properties are worth knowing when you build that preview:

- `ResolveCount(pageCount, sourceName)` returns `Resolve(...).Count`. It counts the *selected numbers*, including a closed range's tail beyond the document, so it can overstate what is written. For an exact count, count only the resolved numbers at most `pageCount`, or use the `PageCount` of a completed merge.
- Selection is not an I/O optimisation. The reader still reads the whole source document; the selection decides which pages are copied into the result, not how much of the file is read.

Preflight instead of exceptions

When page ranges come from a user, checking them before the merge produces a better message than catching the exception afterwards. The inspection model makes that a few lines:

```C#
var info = await inspector.InspectAsync(new PdfFileSource(path));

if (!info.IsReadable)
{
    return $"'{path}' cannot be read: {info.Warnings.FirstOrDefault()?.Message}";
}

if (PageSelection.TryParse(requestedPages, out var selection))
{
    try
    {
        var pages = selection.Resolve(info.PageCount, info.SourceName);
        Console.WriteLine($"{pages.Count} of {info.PageCount} pages selected.");
    }
    catch (MergePlanValidationException exception)
    {
        Console.WriteLine(exception.Message);
    }
}
```

The preflight and the merge share the same `Resolve` implementation, so an accepted selection is accepted by the merge. That is the advantage of the value object being public: you do not have to reimplement its rules in a validator.

Common shapes

```C#
// Everything, explicitly.
.AddFile("a.pdf", pages: "1-")

// Everything except the cover, then the cover at the end.
.AddFile("report.pdf", pages: "2-")
.AddFile("report.pdf", pages: "1", displayName: "Cover")

// A chapter per source, each starting at its own page 1.
.AddFile("chapter-01.pdf", pages: "1-")
.AddFile("chapter-02.pdf", pages: "1-")
.AddFile("chapter-03.pdf", pages: "1-")

// Two specific pages in reverse order.
.AddFile("spread.pdf", pages: "2,1")
```

The third shape is the one to reach for when documents are already split into chapters: the merge plan mirrors the file list, and no page arithmetic is needed. The second shape shows that a document may appear twice; each appearance is a separate source with its own selection and display name.

A note on bookmarks and links

When a document is partially selected, bookmarks, links and form fields that point at pages outside the selection still need a destination. The library points them at the first page the source contributes instead of dropping the whole bookmark or link. The result opens, the navigation stays usable, and the destination is approximate by design.

If a bookmark's destination cannot be resolved at all — for example because it refers to an object the source itself does not contain — it is skipped and a warning is recorded. Chapter 9 lists the warning kinds that can appear for bookmarks, named destinations and page labels.

6. Output Options

`MergeOutputOptions` describes the document you want, as opposed to the pages you want. It is an immutable record with a dozen settings, all optional, and it is attached to a plan with `WithOutput`:

```C#
var now = new DateTimeOffset(2025, 12, 31, 12, 0, 0, TimeSpan.Zero);

var plan = MergePlan.Create()
    .AddFile("cover.pdf")
    .AddFile("body.pdf")
    .WithOutput(options => options with
    {
        MetadataPolicy = MetadataPolicy.Merge,
        MetadataOverride = new PdfMetadata { Title = "Annual report 2025" },
        ModificationDate = now,
    });
```

Setting	Type	Default	Effect
`TargetVersion`	`PdfVersion`	`V1_7`	The version written to the file header
`PrimarySourceId`	`PdfSourceId?`	`null` (first source)	Which source leads document-level features
`MetadataPolicy`	`MetadataPolicy`	`FromPrimarySource`	Where document information comes from
`MetadataOverride`	`PdfMetadata?`	`null`	Fields that replace whatever the policy produced
`ModificationDate`	`DateTimeOffset?`	`null` (now)	The modification date, and the basis of the file identifier
`OutlinePolicy`	`OutlinePolicy`	`Preserve`	Whether bookmarks are merged or dropped
`PageLabelPolicy`	`PageLabelPolicy`	`Drop`	Whether page labels are copied or dropped
`PreserveAnnotations`	`bool`	`true`	Whether page annotations are copied
`PreserveAcroForm`	`bool`	`true`	Whether the primary source's interactive form is copied
`PreserveEmbeddedFiles`	`bool`	`true`	Whether the primary source's attachments are copied
`Encryption`	`PdfEncryptionOptions?`	`null`	Whether the result is encrypted
`Producer`	`string`	`"SharpCorePDF <version>"`	The value of the `/Producer` entry

The defaults follow one principle: **lose as little as possible, without inventing anything.** Pages, annotations and bookmarks are merged from every source. Features that cannot meaningfully exist twice in one document — document information, the interactive form, attachments, page labels — are taken from one source, the primary one. Nothing is encrypted unless you ask for it.

`MergeOutputOptions.Validate()` returns a list of problems rather than throwing, and `MergePlan.Validate()` includes those problems in its own report, so a bad combination surfaces before a file is opened. The checks cover an unsupported target version, undefined policy values, a blank producer and encryption settings that the target version cannot express (Chapter 7).

Target version

The engine writes a classic cross-reference table, which requires PDF 1.4 or newer; `V1_4` through `V2_0` are accepted and anything outside that range is rejected. `V1_7` is the default because it is the most widely supported version that still uses the traditional file structure.

The target version is a claim, not a conversion. The library does not rewrite content to match an older version and does not verify that copied objects only use features of that version. If your sources use transparency, layers or other features introduced after 1.4 and you write a 1.4 file, the file may open in lenient viewers and fail in strict ones. When in doubt, keep the default, or set the target to the highest version among your sources.

Choosing the primary source

`PrimarySourceId` names the source that leads document-level features. By default it is the first source in the plan, but any source may be promoted:

```C#
var body = PdfSource.Create(new PdfFileSource("body.pdf"), displayName: "Body");
var cover = PdfSource.Create(new PdfFileSource("cover.pdf"), displayName: "Cover");

var plan = MergePlan.Create()
    .AddSource(cover)
    .AddSource(body)
    .WithPrimarySource(body.Id);   // metadata, form and attachments come from body.pdf

plan.EnsureValid();
```

`WithPrimarySource` is the shortcut on the plan; `PrimarySourceId` in the output options is the same setting, useful when the options are built elsewhere. A primary source must exist in the plan and must be identified by `PdfSourceId`, not by file path. The call made last wins: `WithPrimarySource` changes the current output options, and a later `WithOutput` can replace the value again.

The primary source is only consulted for features that cannot be merged. Its pages are not treated specially: page order still comes from the plan, and its bookmarks are still merged with the rest. Every source in the plan must contribute at least one page, so a primary source whose selection resolves to no pages is rejected with `MergePlanValidationException` before anything is written; the primary is always a source that actually contributes pages.

Document information

`MetadataPolicy` has three values:

Value	Result
`FromPrimarySource`	The primary source's document information, copied including custom entries
`Merge`	The primary source's values, with empty fields filled from the other sources in plan order
`None`	No document information, apart from the producer and modification date

`FromPrimarySource` is the default because a merge is usually judged by its pages, and the metadata of the document that leads the merge is the most defensible choice. `Merge` is useful when the first file is a bare cover with no title but a later file has one. `None` produces the smallest, least revealing dictionary.

`MetadataOverride` is applied after the policy and replaces only the fields you set:

```C#
MetadataOverride = new PdfMetadata
{
    Title = "Annual report 2025",
    Author = "Finance",
    Keywords = "annual, report, 2025",
    CustomProperties = new Dictionary<string, string>
    {
        ["department"] = "finance",
    },
},
```

An empty string is a value, not an absence: it replaces the field with an empty string. `null` leaves the field as the policy produced it. Custom properties are merged by key, with the override winning. `Producer` and `ModificationDate` are the two fields `MetadataOverride` cannot set: the producer is always `MergeOutputOptions.Producer`, and the modification date is always `MergeOutputOptions.ModificationDate` (or the current UTC time when it is `null`). Setting either one in the override has no effect; set the output option instead.

`ModificationDate` describes the merge run, not the source document, so it is never taken from the policy or from `MetadataOverride`. Set it on the output options to fix the date for a whole plan.

A PDF can carry a second kind of metadata: an XMP packet, a small XML document stored in a stream rather than in the information dictionary. When the metadata policy is not `None`, the primary source's XMP packet is copied unchanged. The library does not regenerate it, so an XMP reader may see the primary source's title even after `MetadataOverride` changes the title in the information dictionary. If consistent XMP matters to your consumers, strip or rewrite it downstream.

Bookmarks, labels and annotations

These settings are independent, and turning one off does not turn off the others.

```C#
.WithOutput(options => options with
{
    OutlinePolicy = OutlinePolicy.Drop,       // no bookmarks at all
    PageLabelPolicy = PageLabelPolicy.FromPrimarySource,
    PreserveAnnotations = false,              // no links, highlights, stamps or widgets
    PreserveAcroForm = false,                 // no interactive form
    PreserveEmbeddedFiles = false,            // no attachments
})
```

Bookmarks. `Preserve` merges the outline trees of every source into one tree whose top-level items follow source order. Bookmarks are remapped to their page's new position; a bookmark that points at a page outside the source's selection is pointed at the first selected page instead. `Drop` writes no outline at all and is the right choice when source bookmarks are meaningless in the new document.

Page labels. Page labels are the displayed page numbers — roman numerals for a preface, "A-1" for an appendix. They are tied to absolute page indices, so they misalign as soon as sources are reordered or partially selected, which is why `Drop` is the default. `FromPrimarySource` copies the primary source's labels and applies them from the first output page, which is only accurate when the primary source starts at page 1 of the result.

Annotations. `PreserveAnnotations` controls every annotation on the merged pages: link annotations, highlights, notes, stamps, and the widget annotations that draw form fields. Turning it off is common for print-oriented output. Bookmarks are not annotations and are unaffected; the cost is that internal links stop working.

Forms. `PreserveAcroForm` copies the interactive form of the primary source only. Form fields from the other sources are not added to the form, but their widget annotations remain on the pages when annotations are preserved, so they stay visible without being fillable. Merging two independent forms into one is not attempted: field names would collide and the result would be ambiguous.

Attachments. `PreserveEmbeddedFiles` copies the embedded files of the primary source. Attachments from the other sources are not merged; if several sources carry attachments, choose the source with the complete set as the primary, or attach files to the result afterwards.

Reproducible output

`ModificationDate` does two jobs. It is written as the modification date, and it is the input from which the document identifier is derived. Supply a fixed value and, for unencrypted output, the same inputs produce the same bytes on every run. Leave it `null` and the current UTC time is used, so every run differs.

Reproducibility has limits worth stating plainly. Encrypted output uses fresh random material, so two encrypted merges of the same input are not byte-identical even with a fixed date (Chapter 7). The producer string contains the assembly version, so upgrading the library changes the bytes. And reproducible bytes are not a signature: they show that two files were produced the same way, not that the content is trustworthy.

```C#
var options = MergeOutputOptions.Default with
{
    ModificationDate = new DateTimeOffset(2025, 1, 1, 0, 0, 0, TimeSpan.Zero),
    Producer = "Contoso Report Pipeline",
};
```

7. Encryption and Permissions

A merged document is written in the clear unless you set `Encryption`. That is true even when every source was encrypted: a merge is a new document, and the passwords of its sources are not carried into it. Encryption is therefore a decision about the output, made once, with `PdfEncryptionOptions`.

```C#
var encryption = new PdfEncryptionOptions
{
    Algorithm = PdfEncryptionAlgorithm.AesWith256BitKey,
    UserPassword = "reader",        // opens the document with the permissions below
    OwnerPassword = "publisher",    // opens it with every permission
    Permissions = PdfPermissions.PrintOnly,
};

var plan = MergePlan.Create()
    .AddFile("report.pdf", password: "source-password")
    .WithOutput(options => options with { Encryption = encryption });

plan.EnsureValid();
```

Setting	Type	Default	Meaning
`Algorithm`	`PdfEncryptionAlgorithm`	`AesWith256BitKey`	The cryptosystem written into the security dictionary
`UserPassword`	`string`	`""`	Opens the document with `Permissions`
`OwnerPassword`	`string`	`""`	Opens it with every permission, ignoring `Permissions`
`Permissions`	`PdfPermissions`	`All`	What a reader who supplied the user password may do

Algorithms

Value	Security handler	Key	Minimum target version	Notes
`Rc4With40BitKey`	Revision 2	40-bit RC4	1.4 (any allowed)	Legacy compatibility only; not protection
`Rc4With128BitKey`	Revision 3	128-bit RC4	1.4 (any allowed)	Legacy; RC4 is broken
`AesWith128BitKey`	Revision 4	128-bit AES	1.5	AESV2 crypt filter
`AesWith256BitKey`	Revision 6	256-bit AES	1.7	AESV3 crypt filter, the default

`AesWith256BitKey` is the default for the same reason it is the default almost everywhere: AES-256 with the hardened revision 6 password hash is the strongest option, and every current reader supports it. Revision 5 of the standard security handler is deliberately not offered, because its single-pass SHA-256 password hash was replaced by revision 6 precisely because it was too cheap to attack. The RC4 algorithms exist for workflows that include a tool which cannot go beyond them; do not choose them for new documents.

The algorithm and the target version must agree. `Validate(TargetVersion)` reports AES-128 below PDF 1.5 and AES-256 below PDF 1.7, and `MergePlan.Validate()` includes those errors, so the mismatch is caught before anything is written rather than producing a file that readers reject.

Passwords

The two passwords have different jobs:

- The **user password** opens the document and is subject to `Permissions`.
- The **owner password** opens the document with every permission, whatever `Permissions` says.

The rules that follow from those jobs:

- **Empty user password.** The document opens without a prompt. This is the shape of a document that is only restricted against editing: anyone can read it, and only the owner password lifts the restrictions. A cooperating viewer still honours `Permissions` for the reader who did not supply the owner password.
- **Empty owner password.** The user password becomes the owner password as well. A document never needs more credentials than it was written with, and there is no separate owner secret to lose.
- **Both empty.** Encryption is still applied, with a key derived from empty passwords. Anyone can derive that key from the file, so the document is effectively unprotected. The library allows it because the difference between "no encryption" and "meaningless encryption" can matter to a downstream tool; it is not a sensible choice for a real document.
- **Truncation.** Each revision defines a maximum password length — 32 bytes for revisions 2 to 4 and 127 bytes for revision 6. A longer password is truncated by the format. `Validate` rejects a password whose UTF-8 encoding exceeds the limit, so you learn about the truncation at validation time instead of discovering that a shorter password opens the file.
- **Encoding.** Revisions 2 to 4 predate an agreement on how passwords are encoded. A password that survives a Latin-1 round trip is written as Latin-1 and anything else as UTF-8, which is what Adobe, qpdf and pypdf hash as well. Revisions 5 and 6 are UTF-8 by definition. SASL preparation is not applied, matching what readers accept.

Both passwords are checked by validation, so a password that is too long for the chosen algorithm is reported as a validation problem before the merge starts, rather than silently truncated.

Permissions

`PdfPermissions` is a flags enumeration whose values are the permission bits of the PDF encryption dictionary. Combine them with `|`.

Flag	Allows
`Print`	Printing
`PrintHighQuality`	Printing at a quality that reproduces the content faithfully
`ModifyContents`	Changing the pages and the document as a whole
`CopyContent`	Copying text and graphics out
`Annotate`	Adding and changing annotations, and filling in form fields
`FillFormFields`	Filling in form fields even when `Annotate` is not allowed
`ExtractForAccessibility`	Screen readers and similar tools extracting content even without `CopyContent`
`Assemble`	Inserting, rotating and deleting pages, and building bookmarks and thumbnails
`All`	Every flag above — a document that only needs a password to open
`PrintOnly`	`Print | ExtractForAccessibility` — readable and printable, nothing else
`None`	Nothing beyond viewing the pages

Permissions are **advisory, not enforcement**. They tell a cooperating viewer what to offer. A reader that ignores them, a tool that parses the file directly, or anyone with the owner password is not stopped. PDF encryption protects the bytes of strings and streams; it does not implement digital rights management, and it should not be described to users as if it did. For documents that must not be copied, the realistic control is who receives the file.

Two smaller consequences of the flags:

- Revision 2 does not define `FillFormFields` or the flags after it. They are still written, but a reader that predates them ignores those bits, so the extra actions you allowed are simply not offered. If you target the oldest readers, restrict yourself to `Print`, `ModifyContents`, `CopyContent` and `Annotate`.
- An empty user password with `PrintOnly` is the usual shape of a "read and print, do not edit" document: no password to open, an owner password for anyone who needs to change it, and a viewer that offers only the permitted actions.

Encrypted sources

The encryption of a source and the encryption of the result are unrelated. A source is opened with the password given to `PdfSource.Create`, `AddFile`, `AddBytes` or `AddStream`; that password may be either the user or the owner password, and `null` is correct for a document that needs none.

```C#
var plan = MergePlan.Create()
    .AddFile("confidential.pdf", password: "reader")
    .AddFile("public.pdf")
    .WithOutput(options => options with
    {
        Encryption = new PdfEncryptionOptions
        {
            UserPassword = "reader-2025",
            OwnerPassword = "publisher-2025",
            Permissions = PdfPermissions.Print | PdfPermissions.CopyContent,
        },
    });
```

Decryption happens while the source is read. The result is written from the parsed object model and then encrypted with the new settings, so the source passwords leave no trace in the output and do not have to be reused. It also means the result inherits nothing: an encrypted source merged without `Encryption` produces a clear document, and a clear source merged with `Encryption` produces an encrypted one.

When a source cannot be opened, the merger throws `PdfEncryptionException` with `RequiresPassword` telling you which problem you have:

- `RequiresPassword == true` — the password was missing or wrong. Ask for one and retry.
- `RequiresPassword == false` — the document uses a security handler the engine does not implement, or its encryption dictionary is malformed. No password will help.

Inspection never throws for an encrypted document (Chapter 8). It reports `RequiresPassword` and `SecurityHandlerRevision` on `PdfDocumentInfo` instead, which is how a caller can ask for a password before attempting a merge:

```C#
var info = await inspector.InspectAsync(new PdfFileSource("confidential.pdf"));

if (info.RequiresPassword)
{
    Console.WriteLine($"'{info.SourceName}' needs a password.");
}
```

Reproducibility and encryption

Encryption draws fresh random material for the file key, the password salts and every initialization vector. Two encrypted merges of identical input with identical settings are therefore not byte-identical, even with a fixed `ModificationDate` (Chapter 6). They are equivalent documents with different bytes; compare their content, not their hashes.

This is the expected behaviour, not a defect: reusing an initialization vector or a salt across documents would weaken the encryption. If you need reproducible bytes for caching or auditing, produce the unencrypted document reproducibly and encrypt it in a separate step whose output you do not compare byte for byte.

8. Inspection

`IPdfInspector` answers "what is this file?" without changing it and without producing a merge. It is the **summary** read side of the library: it describes the document, and the content reader later in this chapter reads what the pages carry. Both use the same reader as the merger, so what you inspect is what you will merge.

```C#
var info = await inspector.InspectAsync(
    new PdfFileSource("report.pdf"),
    password: null,
    cancellationToken: cancellationToken);
```

The method takes any `IPdfContentSource`, not just a file, so the same call works for a byte array or a stream. There is no inspection overload that takes a path directly; wrap the path in a `PdfFileSource`, which also supplies the display name used in messages.

The document model

`PdfDocumentInfo` is an immutable record. The properties you are most likely to use:

Property	Type	Meaning
`SourceName`	`string`	The display name of the source
`Version`	`PdfVersion`	The version from the document header
`PageCount`	`int`	Number of pages, or `0` when the page tree is unreadable
`IsReadable`	`bool`	Whether the structure could be read at all
`IsEncrypted`	`bool`	Whether the document declares encryption
`RequiresPassword`	`bool`	Whether a password is what stands in the way
`SecurityHandlerRevision`	`int?`	The revision of the security handler, or `null`
`Metadata`	`PdfMetadata`	Title, author, subject, keywords, dates and custom entries
`Pages`	`IReadOnlyList<PdfPageInfo>`	One entry per page, in document order
`HasOutlines`	`bool`	Whether the document has bookmarks
`OutlineItemCount`	`int`	Bookmarks, counting nested items
`EmbeddedFileCount`	`int`	Number of attachments
`HasAcroForm`	`bool`	Whether the document has an interactive form
`SourceLengthBytes`	`long?`	Size of the source when known
`Warnings`	`IReadOnlyList<PdfWarning>`	Non-fatal remarks about the document

Two computed properties save a loop: `FirstPageSize` is the first page's size or `null` for an empty document, and `HasUniformPageSize` is `true` when every page has the same size — convenient for deciding whether a merge will produce a uniform document.

`PdfPageInfo` describes one page:

Property	Type	Meaning
`Number`	`int`	The one-based page number
`Size`	`PdfPageSize`	The size as stored, before rotation
`Rotation`	`PageRotation`	How a viewer rotates the page before displaying it

Property	Type	Meaning
`HasContent`	`bool`	Whether the page has a content stream
`AnnotationCount`	`int`	Number of annotations on the page
`DisplaySize`	`PdfPageSize`	`Size` with the rotation applied

`PdfPageSize` is a value object with `WidthPoints` and `HeightPoints` as the source of truth, plus the millimetre and inch conversions, `IsLandscape`, `IsPortrait` and the predefined `A4`, `A5`, `A3`, `Letter` and `Legal` sizes. Use `DisplaySize`, not `Size`, when you care about what a reader sees: a landscape page is frequently stored as a portrait page plus a 90-degree rotation.

```C#
Console.WriteLine($"{info.SourceName}: PDF {info.Version}, {info.PageCount} page(s)");

if (info.FirstPageSize is { } size)
{
    Console.WriteLine($"First page: {size.WidthMillimeters:0} x {size.HeightMillimeters:0} mm");
}

if (!info.HasUniformPageSize)
{
    var sizes = info.Pages
        .Select(page => page.DisplaySize)
        .Distinct()
        .Select(size => size.ToString());

    Console.WriteLine($"Mixed page sizes: {string.Join(", ", sizes)}");
}

Console.WriteLine(info.HasOutlines
    ? $"{info.OutlineItemCount} bookmark(s)"
    : "No bookmarks");
```

What inspection is not

Inspection reads structure, not content. It does not read the text a page draws, render it, count words or check that a font is embedded — the content reader later in this chapter does that reading. `HasContent` says only that a content stream exists, not that it draws anything visible. `AnnotationCount` counts annotations, not the form fields they belong to.

It is also not free. The inspector parses the document, including its page tree, so the cost is close to reading the same document for a merge. Calling it on every request to display a page count doubles the work; if you need both, merge and read the count from `MergeResult.PageCount` instead, or cache the `PdfDocumentInfo` returned by one inspection.

Encrypted and damaged documents

Inspection has a different failure contract from merging, and the difference is deliberate.

- An **encrypted document whose encryption stayed closed** does not throw. You get a `PdfDocumentInfo` with `IsEncrypted = true`, `IsReadable = false`, `RequiresPassword` telling you whether a password would help, and a warning explaining what happened. `PageCount` is zero and `Pages` is empty, but `SourceName`, `SecurityHandlerRevision` and `Warnings` are populated.
- A **malformed document that is not a readable PDF** throws `PdfSourceFormatException`. There is no partial result to return, because the object structure could not be established.
- **A document that is readable but damaged** does not throw. The reader repairs what it can — rebuilds a broken cross-reference table, skips an unreadable page-tree branch, assumes A4 for a page without a media box — and reports each repair as a warning.

```C#
var info = await inspector.InspectAsync(new PdfFileSource(path), password);

if (!info.IsReadable)
{
    Console.WriteLine(info.RequiresPassword
        ? "The document is encrypted; supply the user or owner password."
        : "The document is encrypted with a security handler this library does not implement.");
}
```

`RequiresPassword` is about the *user* password. A document that opens without a password but is restricted against editing has `IsEncrypted = true`, `IsReadable = true` and `RequiresPassword = false`; its restrictions do not block inspection or merging. `PdfDocumentInfo` does not expose the permission flags themselves — inspect the encryption dictionary with another tool if you need to display them.

Reading what the pages contain

Inspection describes the document. `IPdfContentReader` reads the content it already carries — images, text, comments, the form, the bookmarks, the attachments and the XMP packet — and it changes nothing: no source is modified or disposed, and nothing is opened that you did not supply.

```C#
using SharpCorePDF.Application.Extensions;

var reader = provider.GetRequiredService<IPdfContentReader>();
var source = PdfFileSource.From("report.pdf");

var images = await reader.ReadImagesAsync(source, PageRange.FromTo(1, 10), cancellationToken: ct);

foreach (var image in images.Items)
{
    Console.WriteLine(
        $"image {image.Id} on page(s) {string.Join(", ", image.Pages)}: "
        + $"{image.Width}x{image.Height}, {image.ByteLength} bytes, inline {image.IsBuffered}");
}
```

Method	Answers
`ReadImagesAsync`	One entry per distinct image object the range paints: every page that draws it, the position of the first draw, geometry, filter, the soft or explicit mask beside the samples, and — when the payload was decoded — the pixels themselves as `DecodedPixels` in `Gray8`, `Rgb24` or `Cmyk32`.
`CopyImageDataAsync`	Writes the bytes of one image to a stream you supply.
`CopyImageMaskDataAsync`	Writes the bytes of one image's soft or explicit mask to a stream you supply; a colour key mask has no bytes and is refused by name.
`ReadAnnotationsAsync`	Every annotation with its page, subtype, rectangle, flags, text, author, date, appearance and, for a link, the resolved `Target`.
`ReadFormFieldsAsync`	Every field of the interactive form with its fully-qualified name (`person.name`), type, value, default value, flags and widgets.
`ReadOutlineAsync`	The bookmark tree in document order, with each item's title, open state, resolved destination and children.
`ReadEmbeddedFilesAsync`	One entry per attachment in document order, each with a distinct `Id`, its name, MIME type, relationship, date and length.
`CopyEmbeddedFileDataAsync`	Writes the bytes of one attachment to a stream you supply.
`ReadMetadataAsync`	The XMP packet byte-exact in `RawPacket`, its UTF-8 text, and the `dc:`, `pdf:` and `xmp:` values parsed into `Properties`.
`ReadTextAsync`	The text a page range draws: one `PdfTextBlock` per paragraph, in reading order, carrying its text, line count, bounding box and the words and glyphs it is made of.

Method	Answers
`ReadPageDetailsAsync`	An `IAsyncEnumerable<PdfPageDetail>`: one page at a time, carrying its images, annotations, links, form fields and own warnings.

The interface has exactly those eleven members and no overloads. The convenience members in `SharpCorePDF.Application.Extensions` sit beside it for the common shapes: `ReadImagesAsync(source, page)`, `ReadTextAsync(source, page)` and `ReadAnnotationsAsync(source, page)` for one page, `ReadImageDataAsync` / `ReadEmbeddedFileDataAsync` for a payload as a `ReadOnlyMemory<byte>`, `SaveImageDataAsync` / `SaveEmbeddedFileAsync` for a payload written to a file, and `ReadPageDetailAsync(source, page)` for a single `PdfPageDetail?`.

The result shape

Every read except `ReadMetadataAsync` and `ReadPageDetailsAsync` returns `PdfReadResult<T>`: `Items` (never null), `Warnings` (empty when nothing went wrong) and a derived `Count`. The two states a caller has to tell apart are:

- **nothing found** — `Items` is empty and `Warnings` is empty. The document genuinely has no image, no annotation, no attachment or no outline.
- **nothing could be read** — `Items` is empty and `Warnings` is not. Something was there and could not be returned; the kind says what.

A partial read is the normal case: one damaged image adds an `ImageDataUnreadable` warning and the other images of the range are still returned. `ReadMetadataAsync` returns `PdfXmpResult`, whose `Metadata` is `null` when the document has no packet, and whose malformed packet is a non-null `Metadata` with `RawPacket` intact, empty `Properties` and one `XmpPacketMalformed` warning. A page enumeration reports a damaged page through that page's own `Warnings`, so a bad page never abandons the pages after it.

Payloads and the inline budget

A record carries its bytes inline only when they fit `PdfEngineOptions.MaxInlineContentBytes`, 256 KiB by default (`0` means metadata only). Above the budget `Data` is `null`, `IsBuffered` is `false` and `ByteLength` still tells the truth — including when the payload was withheld rather than missing. No warning is raised: withholding bytes is the rule working, not a failure.

```C#
var large = images.Items.First(image => !image.IsBuffered);

await reader.CopyImageDataAsync(source, large, destinationStream, cancellationToken: ct);
```

`CopyImageDataAsync`, `CopyImageMaskDataAsync` and `CopyEmbeddedFileDataAsync` run through one bounded buffer, so the peak working set does not grow with the payload; they flush the destination and leave it open, because they did not open it. `ReadImageDataAsync` and `ReadEmbeddedFileDataAsync` materialise the payload and are only appropriate when its size is known to fit in memory; `SaveImageDataAsync` and `SaveEmbeddedFileAsync` are the copy route with the file opened for you. A `ReadOnlyMemory<byte>` that escapes a call never aliases a rented buffer.

Pixels

`Data` is the payload as the document stores it, which for most images is samples behind a compression filter. `DecodedPixels` is the same picture already unpacked — tightly packed, top row first, one byte per channel — so a caller that wants to look at an image does not write a decoder first. `PixelFormat` names the layout: `Gray8`, one channel, `Rgb24`, three interleaved, or `Cmyk32`, four device samples interleaved.

```C#
foreach (var image in images.Items)
{
    if (image.DecodedPixels is not { } pixels)
    {
        continue;                       // a payload, not pixels — see the table below for why
    }

    var channels = image.PixelFormat switch { "Gray8" => 1, "Rgb24" => 3, _ => 4 };
    var stride = image.Width!.Value * channels;
    var first = pixels.Span[..stride]; // the top row, one byte per channel
}
```

The pixels are present only when nothing is left to interpret: the library decoded the payload itself, the document states the width and the height, and the colour space is one whose samples mean what they are — `DeviceGray` or the abbreviation `G` for one channel, `DeviceRGB` or `RGB` for three, `DeviceCMYK` or `CMYK` for four, the array form of `CalGray` or `CalRGB`, an `ICCBased` space whose profile states one of those component counts, or an `Indexed` space over any of them. An image that carries no filter is the simplest case rather than an exception — its payload is its samples — so an inline image, which producers rarely compress, states its pixels the way a Flate one does. Nothing is left behind a filter either, because the filters an image realistically uses are decoded here — Flate and LZW, each with its predictor, and the two ASCII encodings — so an image that a viewer can show is an image whose samples this record states.

Two things are unpacked on the way out rather than handed over as they lie in the file. Samples narrower than a byte are widened to the same scale, so a bilevel image yields `0` and `255` rather than `0` and `1`, a four-bit sample steps by seventeen and a two-bit one by eighty-five; a row is read at its own padded length rather than as a continuous bit stream, so the padding of one row is not the start of the next. A palette is applied, so an `Indexed` image yields the colours its lookup table states rather than indices into it. Both leave `Data` untouched: the payload is always the bytes the document holds.

The length of the pixels is an invariant a caller can check instead of trusting — `Width × Height × channels` — and it is why an image whose geometry disagrees with its data, or whose palette does not reach as far as its largest index, is reported rather than reinterpreted. Three cases yield `null`, and the record always states which one it is:

Case	What the entry says	Why no pixels
A `DCTDecode` image	`IsDecodable` true, `Filter` `DCTDecode`, `PixelFormat` null	The bytes are already a JPEG, which a caller can read; unpacking it here would mean shipping a second decoder.
A filter the library passes through — `JPXDecode`, `JBIG2Decode`, `CCITTFaxDecode`, `RunLengthDecode`, or a name it does not know	`IsDecodable` false, `Filter` names it, and the read warns with `ImageFilterUnsupported`	Nothing was decoded, so there are no samples to hand back.
A `Separation`, `DeviceN` or `Lab` space, sixteen bits per sample, a palette whose samples run past its table or whose table holds nothing, a width or height of zero, or a geometry the document does not state	`BitsPerComponent` or `ColorSpace` states which	The first needs a function evaluated, the second a rounding rule, the third a decision the file does not support, and the last a rectangle that is not there. Pixels are eight bit and channel-ordered by contract, so converting or guessing would hide the mismatch from the caller.

Neither a colour profile nor a tint transform is applied: the samples are what the document stores. That is why `Cmyk32` exists rather than an RGB conversion — turning ink coverage into light is a rendering decision, not a fact of the file — and why an `Indexed` palette is applied but an `ICCBased` profile is not.

`DecodedPixels` is subject to the same inline budget as `Data`: when a payload is withheld, the pixels are `null` as well, and so are the pixels of a payload that fits the budget but would expand past it. Nothing is resampled, colour-converted or rotated here, and nothing in this library renders a page to a raster — Chapter 20's exports describe a page rather than paint it. This surface exists so that text extraction, an OCR pipeline or a checksum can see the samples without carrying a decoder of its own.

`ReadPageDetailsAsync` is how an index is built: it parses and decodes one page, yields its detail, and only then reads the next, so a 200-page document costs the pages consumed rather than the document.

```C#
await foreach (var page in reader.ReadPageDetailsAsync(source, PageRange.All, cancellationToken: ct))
{
    index.Add(page.PageNumber, page.DisplaySize, page.Links.Count);

    if (index.Count == 50)
    {
        break;   // abandonment stops the walk; the remaining pages are never read
    }
}
```

`DisplaySize` is `Size` with `Rotation` applied, which is the size a viewer shows. Abandoning the loop disposes the enumerator and with it the read, so a caller that has seen enough does not pay for the rest.

Reading the text

`ReadTextAsync` answers "what does this page say?" It returns one `PdfTextBlock` per paragraph, ordered the way a reader takes them: blocks down the page, words along the line. The order comes from where the glyphs land, not from the order the content stream draws them, so a page whose header is painted last still reads with the header first, and a stream that zig-zags across a page comes back in the order the eye follows rather than in the order of its operators.

```C#
var text = await reader.ReadTextAsync(source, PageRange.All, cancellationToken: ct);

foreach (var block in text.Items)
{
    Console.WriteLine($"page {block.PageNumber}, block {block.ReadingOrder}: {block.Text}");
}
```

Record	Carries
`PdfTextBlock`	`PageNumber`, `ReadingOrder` (one-based, restarting on every page), `Text`, `LineCount`, `BoundingBox` and `Words`
`PdfWord`	`Text`, `PageNumber`, `BoundingBox`, `Glyphs`, and from the first glyph its `FontName`, `FontSize` and `BaselineY`
`PdfGlyph`	The decoded `Text`, `PageNumber`, `X`, `Y`, `Width`, `FontSize`, `FontName` and `BaseFont` as drawn

`Text` is the block as plain text: its lines joined with a newline and each line's words joined with a single space — the form a search box, a translation call or a `LIKE` clause wants. `Words` and `Glyphs` carry the geometry for a caller that has to draw something back on the page or compare positions.

Three rules decide the grouping, all stated in ems so a 6pt footnote and a 30pt title are grouped alike:

- **glyphs to words** — a whitespace glyph ends the word, and so does a gap wider than a fifth of an em; a kerning overlap does not, so a word drawn one glyph at a time is still one word;
- **words to lines** — words share a line when their baselines are within a quarter of an em and their boxes reach across to each other; two runs on one baseline far apart stay two lines, which is how two table cells or two columns keep their places;
- **lines to blocks** — a line continues the block above when it drops by no more than 1.8 em and starts within 2 em of where that block started. A wider gap is the blank between paragraphs; a start further right is a column of its own.

That is a reading of the page, not a claim about its structure. Nothing here knows what a heading is, and a table comes back as one block per row. A two-column page comes back as blocks that alternate between the columns, one line at a time, because the lines of the two columns sit at the same heights. When a caller knows better, the glyph coordinates are enough to sort or merge the blocks again — no richer rule is attempted, because guessing would be a layout engine dressed as a reader.

The text state is followed in full: `Tf`, `Tc`, `Tw`, `Tz`, `TL`, `Ts`, `Td`, `TD`, `Tm`, `T*`, `TJ` and `q` / `Q`, with `cm` and a form's `/Matrix` applied on top, so a glyph lands where a viewer draws it inside a scaled, rotated or nested form. A `TJ` adjustment is a word break when it is wider than the same fifth of an em and a kern when it is narrower.

Characters come from the font, never from the raw code byte. A simple font is read through its `/Encoding` — `WinAnsiEncoding`, which is also the reading when none is stated, `StandardEncoding` or `MacRomanEncoding` — and its `/Differences`, which name a glyph for individual codes — the standard glyph names, including the Latin-1 accented set such as `eacute`, are understood. A composite font is read through its `/ToUnicode` map. Widths come from `/Widths` with `/FirstChar`, or from `/W` and `/DW` for a composite font, with `/MissingWidth` as the fallback. When none of these answers for a code, the glyph is skipped and the page gets one `TextUnreadable` warning naming the font: a document in an encoding this library cannot follow reads as fewer words plus that warning, never as invented characters, and the blocks from the fonts that did read are still returned. A page with nothing readable on it contributes no block, which keeps "this page has no text" and "this page's text could not be read" two different answers.

The read does not run OCR, does not reflow, and draws nothing. The only part of an embedded font program it reads is the clear-text header of a Type1 font — a run of <c>dup <code> /<glyph> put</c> lines or a statement that the encoding is <c>StandardEncoding</c> — and only when the font's dictionary states no encoding at all. A pdfTeX subset of a Computer Modern math or symbol font is the case that needs it: its encoding exists nowhere but there, so without it a formula reads as no text. The encrypted part of the program, which a renderer needs to draw, is never read. Beyond that the read reports the text the document draws and where it draws it; hyphenation, reading order across columns and what a table means are the caller's decisions, made on the geometry this read returns.

`PdfPageDetail` does not carry text, so an index that needs both text and images calls `ReadTextAsync` for the page beside the enumeration, rather than paying for a text read of the whole range.

Warnings

`PdfWarning` pairs a stable `Kind` with a human-readable `Message`. The kind is the contract; the text may be reworded in any release, so match on `Kind` and log `Message`. The set of kinds is open, which means a `switch` over `PdfWarningKind` must keep a `default` arm.

The kinds fall into five groups:

Group	Examples	What it means
Structure repaired	`CrossReferenceRebuilt`, `CrossReferenceEntryMismatch`, `ObjectStreamTruncated`	The reader recovered; the document may be damaged
Page tree	`PageTreeMissing`, `PageTreeCycle`, `PageSizeAssumed`	Pages were skipped, cut off, or described with a default size
Encryption	`EncryptionOpened`, `EncryptionHandlerUnsupported`, `CryptFilterUndefined`	The encryption was understood, opened, or could not be used
Merge decisions	`NamedDestinationsDropped`, `OutlinesDroppedByPolicy`, `BookmarksTruncated`	Content was intentionally not carried over
Content reading	`ContentStreamUnreadable`, `ImageFilterUnsupported`, `ImageDataUnreadable`, `XObjectResourceMissing`, `NamedDestinationUnresolved`, `EmbeddedFileUnreadable`, `EmbeddedFileNameCollision`, `XmpPacketMalformed`, `FormFieldUnreadable`, `FormFieldDepthExceeded`, `AnnotationUnreadable`, `TextUnreadable`	A page, an image, an attachment, a packet, a named destination, a form field or a font could not be read, or a form tree was cut off at its depth limit; the rest of the read is still returned

`EncryptionOpened` is a warning even though it describes success: it tells a caller that the document was protected and is not simply unprotected. `OutputVerified` is likewise a success signal, reported only when `PdfEngineOptions.VerifyOutput` is enabled (Chapter 10).

Warnings from a source are folded into `MergeResult.Warnings`, so a caller who only looks at the merge result still learns that a source had to be repaired. Identical warnings are not repeated; because the message names the source, two damaged sources normally produce two warnings rather than one.

A practical handling policy:

```C#
foreach (var warning in result.Warnings)
{
    logger.LogWarning("{Kind}: {Message}", warning.Kind, warning.Message);
}
```

Treat structural warnings as reasons to keep the source but investigate it, and treat merge-decision warnings as expected when the plan deliberately drops something. Do not fail a merge because warnings exist: if a condition was fatal, the merge would have thrown.

Using inspection as a preflight

The inspector is most useful immediately before a merge:

1. Inspect each source with its password.
2. Reject documents that are unreadable, and ask for a password when `RequiresPassword` is true.
3. Resolve page selections against `PageCount` so a bad range is a validation message rather than an exception halfway through writing (Chapter 5).
4. Warn the user about repairs, encryption and unusual page sizes.
5. Merge only the sources that passed.

This mirrors the order the merger itself uses — open, read structure, resolve selections, write — but gives you a place to interject a message, a prompt or a policy decision. It does not make the merge safer in the sense of catching every error: a file can change between inspection and merge, and the merger validates again. The gain is a better conversation with the user, not a substitute for handling `PdfException` (Chapter 9).

9. Errors, Exceptions and Warnings

SharpCorePDF separates three kinds of bad news:

- **Programmer errors** — a null argument, a stream that cannot be written, invalid page notation — throw the standard .NET exceptions (`ArgumentNullException` , `ArgumentException` , `FormatException`). They indicate a bug in the calling code, not a problem with a document.
- **Document problems** — a malformed source, a missing password, an unsupported compression filter, an impossible plan — throw a subclass of `PdfException` .
- **Non-fatal remarks** — a repaired cross-reference table, a bookmark that had to be skipped, a feature deliberately dropped by policy — are returned in `Warnings` and never throw.

Everything in the third group is reported on `PdfDocumentInfo.Warnings` when inspecting and on `MergeResult.Warnings` after a merge. Treating a warning as an error is a policy decision, not something the library forces on you.

The exception family

`PdfException` is abstract and is the base of every PDF-related error. Catching it catches all of them.

Exception	Meaning	Useful members
`MergePlanValidationException`	A plan invariant is broken, or a page selection does not fit its source	`Errors`
`PdfSourceFormatException`	A source is not a readable PDF, or its structure cannot be trusted	`SourceName` , `Detail`
`PdfEncryptionException`	A source is encrypted and could not be opened	`SourceName` , `RequiresPassword`
`PdfUnsupportedFeatureException`	A source uses a PDF feature the engine deliberately does not implement	`SourceName` , `Feature`

`MergePlanValidationException` is unusual in carrying more than one message. `Errors` lists every problem found in one pass, and `Message` joins them. Show the list if you are reporting to a developer; show the first item if you are reporting to a user.

`PdfSourceFormatException` and `PdfEncryptionException` name the offending source, which matters when a plan has ten sources. `PdfUnsupportedFeatureException` names the feature — for example an unsupported compression filter for an object stream — so the message can say what the tool would need to support rather than only that the file failed.

What throws when

The sequence inside `MergeAsync` explains where each exception comes from:

1. `plan.EnsureValid()` runs first. A plan with no sources, a primary source that is not in the plan, a blank display name, an undefined policy value or invalid encryption settings throws `MergePlanValidationException` before any file is opened.
2. Each source is opened and read. A missing file throws `IOException` or `UnauthorizedAccessException` from the content source; malformed content throws `PdfSourceFormatException` ; a source that stays encrypted throws `PdfEncryptionException` .
3. Each page selection is resolved against the real page count. A range that starts past the last page, or a document with no pages, throws `MergePlanValidationException` .
4. The engine writes pages. Unsupported filters and malformed objects discovered here still surface as `PdfException` subclasses, not raw index errors.
5. Cancellation is observed between sources and during writing and surfaces as `OperationCanceledException` , the standard .NET contract.

Because steps 1 and 3 happen before the engine writes, a bad plan or a bad page range does not produce a partial PDF in the destination stream. It does not protect an existing file, however: `MergeToFileAsync` creates and truncates the destination before it calls the merger. The section on writing files safely later in this chapter shows the pattern that avoids this.

Handling exceptions

Catch specific types from the most specific to the least, so each gets the response it deserves:

```C#
try
{
    plan.EnsureValid();
    var result = await merger.MergeToFileAsync(plan, destination, cancellationToken);
    Console.WriteLine(result);
}
catch (MergePlanValidationException exception)
{
    // The caller can fix this: bad range, empty plan, missing primary source.
    foreach (var error in exception.Errors)
    {
        Console.WriteLine($"plan: {error}");
    }
}
catch (PdfEncryptionException exception) when (exception.RequiresPassword)
{
    // Ask for a password for exception.SourceName and retry.
    Console.WriteLine($"'{exception.SourceName}' needs a password.");
}
catch (PdfEncryptionException exception)
{
    // Unsupported security handler: no password will help.
    Console.WriteLine(exception.Message);
}
catch (PdfSourceFormatException exception)
{
    Console.WriteLine($"'{exception.SourceName}' is damaged: {exception.Detail}");
}
catch (PdfUnsupportedFeatureException exception)
{
    Console.WriteLine($"'{exception.SourceName}' uses {exception.Feature}: {exception.Message}");
}
catch (PdfException exception)
{
    // Any PDF-related failure that the cases above did not name.
    Console.WriteLine(exception.Message);
}
catch (OperationCanceledException)
{
    Console.WriteLine("The merge was cancelled.");
}
catch (IOException exception)
{
    Console.WriteLine($"The file system refused the merge: {exception.Message}");
}
```

Three details are easy to get wrong:

- **`OperationCanceledException` is not a `PdfException`.** It derives from `SystemException` directly, so a `catch (PdfException)` will not see a cancellation. Catch it separately, and do not log it as a failure.
- **`IOException` is not a `PdfException` either.** A missing input file, a read-only destination or a full disk is a file-system problem. It can occur at any step, including while a source is read.
- **Order matters.** `PdfEncryptionException` and `PdfSourceFormatException` both derive from `PdfException`; catching the base first would hide the members that make them actionable.

`ArgumentException`, `ArgumentNullException` and `FormatException` are deliberately not caught in this example. They mean the code built the plan incorrectly — a null stream, an unwritable destination, a malformed page string — and the fix belongs in the code, not in a retry loop. Validate page text with `PageSelection.TryParse` before building the plan, so a typo becomes a validation message rather than an exception in the middle of a request.

Writing files safely

`MergeToFileAsync` is convenient and is the right call for a scratch file, but it creates the destination with `FileMode.Create` before the merge begins. If the merge then fails or is cancelled, the destination is left empty or partially written. Two consequences:

1. **Never merge a file onto itself.** If the destination is also a source path, the source is truncated before it is read. The merge fails and the original is gone.
2. **Do not treat a returned file as committed until the call returns successfully.** A watcher that processes the directory may see a partial file while the merge is still running.

For anything that matters, write to a temporary file in the same directory and move it into place only after success:

```C#
var fullPath = Path.GetFullPath(destination);
var directory = Path.GetDirectoryName(fullPath)!;
Directory.CreateDirectory(directory);

var temporary = Path.Combine(directory, $".{Path.GetFileName(fullPath)}.{Guid.NewGuid():N}.tmp");

try
{
    await merger.MergeToFileAsync(plan, temporary, cancellationToken);
    File.Move(temporary, fullPath, overwrite: true);
}
finally
{
    if (File.Exists(temporary))
    {
        File.Delete(temporary);
    }
}
```

The temporary file lives in the same directory, so the move stays on one volume, where it is atomic on Windows and on the common Unix file systems. The `finally` block removes the temporary file on failure and on cancellation; after a successful move the file is already gone. The destination is either the previous file or the complete new one, never a half-written document.

The path itself is caller input, and the library does not confine it. `MergeToFileAsync`, `SaveImageDataAsync` and `SaveEmbeddedFileAsync` resolve the destination with `Path.GetFullPath` and write there: a relative path becomes absolute against the process working directory, `..` is followed, a missing directory is created, and an existing file is truncated. No root is enforced, because the library never chose one — a `..` guard would break the relative paths callers build on purpose, and would not make the library a sandbox, since the same bytes can be written through a `FileStream` the caller opened. When a destination comes from an untrusted source, confine it before the call: decide which directory it must land in, resolve it against that directory, reject the segments you will not accept, and check the result.

`MergeToBytesAsync` has the same all-or-nothing property for a different reason: the buffer is local to the call, so a failure returns no bytes and leaves nothing behind. It is the simplest safe choice when the result fits in memory.

Warnings versus errors

A warning never changes the outcome: the merge completed and the result is usable. The distinction is worth preserving in your own API, because collapsing warnings into errors makes the library look less reliable than it is, and ignoring them hides real repairs.

A reasonable policy:

Warning group	Suggested response
Structure repaired (`CrossReferenceRebuilt` , `ObjectStreamTruncated`)	Log; keep the source; investigate if it recurs
Page tree (`PageTreeMissing` , `PageSizeAssumed`)	Log at warning level; compare the page count with expectations
Encryption (`EncryptionOpened`)	Log at information level; the password worked
Merge decisions (`OutlinesDroppedByPolicy` , `NamedDestinationsDropped`)	Log at debug level; expected when the plan drops features
`OutputVerified`	Log at information level; verification was requested and passed

If your product has a "strict" mode, implement it in your code by inspecting `result.Warnings` and deciding which kinds are unacceptable — do not expect the merger to throw for them. Chapter 8 lists the kinds and their meanings.

Reproducing a failure

When a merge fails, the report should contain the plan and the exception, not only the message:

```C#
catch (PdfException exception)
{
    logger.LogError(exception, "Merge failed. Plan: {Plan}", plan.ToString());
}
```

`MergePlan.ToString()` summarises the source count, target version and policies without printing page content. `PdfDocumentInfo.ToString()` and `MergeResult.ToString()` do the same for a document and a completed merge. None of them includes passwords: `PdfSource.ToString()` prints the display name and page selection only.

Do not log the plan *object* with structured logging, though. The properties are public, and `MergeOutputOptions` is a record whose generated `ToString()` would include the encryption options and their passwords. Log `plan.ToString()` or the individual non-secret fields.

10. Dependency Injection

The `SharpCorePDF.DependencyInjection` package registers the merger, the editor, the inspector and the engines behind their ports. It is a thin layer: one call, no auto-discovery, no background services and no logging of its own. The `SharpCorePDF` meta-package already depends on it, so the types are available after the installation described in Chapter 3.

One call

```C#
using SharpCorePDF.DependencyInjection;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSharpCorePDF();

var app = builder.Build();
```

That registers the services listed below as singletons. They are stateless; a merge or an edit creates its own context, so concurrent requests can use the same `IPdfMerger` or `IPdfDocumentEditor` instance without coordination.

Service	Lifetime	Notes
`PdfEngineOptions`	Singleton	`PdfEngineOptions.Default` unless replaced
`PdfDocumentReader`	Singleton	Concrete reader; registered for advanced use
`IPdfDocumentReader`	Singleton	Resolves to the same `PdfDocumentReader`
`PdfMergeEngine`	Singleton	Concrete merge engine
`IPdfMergeEngine`	Singleton	Resolves to the same `PdfMergeEngine`
`PdfEditEngine`	Singleton	Concrete edit engine
`IPdfEditEngine`	Singleton	Resolves to the same `PdfEditEngine`
`IPdfMerger`	Singleton	`PdfMergerService`, the merge use case to depend on
`IPdfDocumentEditor`	Singleton	`PdfDocumentEditorService`, the edit use case to depend on
`IPdfInspector`	Singleton	`PdfInspectorService`
`PdfPageContentReader`	Singleton	Concrete content reader
`IPdfPageContentReader`	Singleton	Resolves to the same `PdfPageContentReader`
`IPdfContentReader`	Singleton	`PdfContentReaderService`

Application code should depend on `IPdfMerger`, `IPdfDocumentEditor` and `IPdfInspector`, not on the concrete services. The concrete registrations exist so a test can replace one layer without replacing the whole stack, and so the engines can be exercised directly.

Authoring is not registered because it is not a service: `PdfDocumentBuilder` is an ordinary class with a static factory, and `Build()` returns an `IPdfContentSource` the merger already understands. A typeface is immutable, so a typeface read at start-up can be registered as a singleton and shared by every builder:

```C#
builder.Services.AddSingleton(PdfTypeface.FromTrueType(File.ReadAllBytes("fonts/DejaVuSans.ttf")));
```

```C#
app.MapPost("/merge", async (IPdfMerger merger, IFormFileCollection files) =>
{
    var plan = MergePlan.Create();

    foreach (var file in files)
    {
        await using var stream = file.OpenReadStream();
        using var buffer = new MemoryStream();
        await stream.CopyToAsync(buffer);
        plan = plan.AddBytes(buffer.ToArray(), file.FileName);
    }

    var bytes = await merger.MergeToBytesAsync(plan);
    return Results.File(bytes, "application/pdf", "merged.pdf");
});
```

Each `AddBytes` call creates a source with a fresh identity, so uploading the same file twice contributes its pages twice. The bytes are buffered before the merge because the merger reads its sources later; an `IFormFile` stream is not guaranteed to stay open until then. Chapter 4 covers the other factories.

Without the container

`AddSharpCorePDF()` only saves typing. The reader, the engine and the two services have public constructors, so the same graph can be built by hand:

```C#
using SharpCorePDF.Application.Services;
using SharpCorePDF.Domain.Ports;
using SharpCorePDF.Infrastructure;

var options = PdfEngineOptions.Default;
var reader = new PdfDocumentReader(options);

IPdfMerger merger = new PdfMergerService(reader, new PdfMergeEngine(options));
IPdfInspector inspector = new PdfInspectorService(reader);
```

The two routes are equivalent: the container builds exactly this graph and registers it as singletons. The `no-container` example in the repository runs the same plan through both and compares the bytes.

Three ways to configure the engine

The extension has three overloads; the configuration overload takes an optional section name, so these four shapes are possible:

```C#
services.AddSharpCorePDF();                                    // defaults
services.AddSharpCorePDF(PdfEngineOptions.Default with { ... }); // explicit options
services.AddSharpCorePDF(configuration);                       // the "SharpCorePDF" section
services.AddSharpCorePDF(configuration, "My:Pdf:Section");     // a section of your choosing
```

- **No argument** registers `PdfEngineOptions.Default` through `TryAddSingleton`, so an options instance you registered earlier is left in place.
- **An options instance** is registered with `Replace`, so it wins over an earlier registration.
- **A configuration** binds the named section and registers the bound value. The default section name is the `DefaultConfigurationSection` constant, `"SharpCorePDF"`.

One note matters only when the application is published trimmed or with Native AOT. The configuration overload uses the framework's binder, which reads the property names by reflection and creates its converters at run time, so it is annotated

`[RequiresUnreferencedCode]` and `[RequiresDynamicCode]` and a trimmed or AOT publish reports it as a warning. Such an application passes an options instance in code — the second shape above — and `docs/aot.md` records the proof that the rest of the library is trim- and AOT-clean.

The configuration overload does not fail when the section is missing: binding an absent section produces the default values. That is convenient in development and dangerous in production, where a typo in the section name silently disables a limit you thought you had set. Validate explicitly:

```C#
var engineOptions = builder.Configuration
    .GetSection(SharpCorePDFServiceCollectionExtensions.DefaultConfigurationSection)
    .Get<PdfEngineOptions>() ?? PdfEngineOptions.Default;

var problems = engineOptions.Validate();

if (problems.Count > 0)
{
    throw new InvalidOperationException(
        $"SharpCorePDF configuration is invalid: {string.Join(" ", problems)}");
}

builder.Services.AddSharpCorePDF(engineOptions);
```

`Validate()` checks the numeric limits and returns every problem it finds instead of throwing. It does not read any document, so it cannot catch a page selection that reaches past the last page; that surfaces when the plan runs (Chapter 9).

Engine options

`PdfEngineOptions` is an immutable record. Configure it with `with` expressions, and remember that the defaults are deliberately permissive:

Option	Default	Effect
`MaxSourceBytes`	512 MiB	Largest source read into memory; larger sources are rejected
`MaxNestingDepth`	64	Maximum dictionary/array nesting, page-tree depth and form field tree depth
`MaxPagesPerSource`	100 000	Declared page limit; see the note below
`ReconstructCrossReferenceOnFailure`	`true`	Rebuild a damaged or missing cross-reference table by scanning
`VerifyOutput`	`false`	Re-read the written document and check its page count
`WriteBinaryMarker`	`true`	Write the binary comment after the `%PDF` header

The source limit is enforced by the reader, and exceeding it throws `PdfSourceFormatException` with a detail that names the limit. It is not a streaming cap: the whole source is held in memory while it is processed, so the practical ceiling is your process's memory budget, not the number on the option.

`MaxNestingDepth` has four effects. It bounds how deep an object graph may be parsed, it bounds the page-tree walk, it bounds the form field tree, and it bounds the tree the signature search descends. A page tree that is cut off contributes a `PageTreeTooDeep` warning rather than failing the merge; a field tree that is cut off contributes `FormFieldDepthExceeded` together with the fields above the cut, and the search for signatures reports the same warning and adds a `PdfSignatureProblemKind.FormFieldTreeTooDeep` problem to the validations it returns.

`ReconstructCrossReferenceOnFailure` is the difference between "repair what you can" and "reject what is not well formed". Leaving it on produces a `CrossReferenceRebuilt` warning when repair happened. Turning it off is appropriate when a damaged file should be treated as an error, for example in a regulatory archive.

`VerifyOutput` is a development and test switch. It captures the complete output in memory, reads it back and compares the page count with what was written, then adds an `OutputVerified` warning on success. That doubles the work and holds the whole result in memory, which defeats the streaming design for large documents. Use it in integration tests, not in production.

`WriteBinaryMarker` should stay on. Some transfer tools inspect the first bytes to decide whether a file is binary; the marker costs a few bytes and prevents a class of corruption that is otherwise very hard to diagnose.

> `MaxPagesPerSource` is part of the options record but this version does not enforce it. Do not rely on it as a security boundary. For untrusted input, set `MaxSourceBytes`, inspect the document with `IPdfInspector` and reject it if `PageCount` exceeds your own limit, before building a plan.

Configuration file

With the configuration overload, the section names map directly to the property names:

```JSON
{
  "SharpCorePDF": {
    "MaxSourceBytes": 268435456,
    "MaxNestingDepth": 64,
    "MaxPagesPerSource": 50000,
    "ReconstructCrossReferenceOnFailure": true,
    "VerifyOutput": false,
    "WriteBinaryMarker": true
  }
}
```

Environment variables and command-line arguments follow the normal configuration rules, for example `SharpCorePDF__MaxSourceBytes`. Because the section is bound to an immutable record, a property that is missing keeps its default; a property with a value of the wrong type causes a binding exception at start-up, which is one of the few failures you want to be loud.

Logging and diagnostics

The library has no `ILogger` dependency and writes nothing to the console. That is deliberate: a class library that logs through a static logger cannot be tested cleanly and cannot respect a host's configuration. Everything you need to log is in the return values:

- `MergeResult` carries `PageCount`, `SourceCount`, `OutputBytes`, `OutputVersion`, `Duration`, `PrimarySourceId`, per-source summaries and `Warnings`.
- `PdfDocumentInfo` carries the same kind of facts for inspection, plus `IsReadable`, `IsEncrypted` and `RequiresPassword`.
- Exceptions carry `SourceName`, and `MergePlanValidationException` carries `Errors`.

Log them at the boundary of your application, where you know the request and the user. Chapter 9 shows the exception-handling shape, and Chapter 8 explains the warning kinds.

The producer string

When `MergeOutputOptions.Producer` is not set, the writer stamps the document with the product name and the assembly version of the SharpCorePDF domain assembly, for example `SharpCorePDF 1.5.0`. Override it with your own product name if the PDF is produced by your application rather than offered as a library feature:

```C#
var plan = MergePlan.Create()
    .AddFile("input.pdf")
    .WithOutput(options => options with { Producer = "Contoso Report Server 4.2" });
```

There is no separate "application name" setting in `PdfEngineOptions`; the producer belongs to the output document, not to the engine, and therefore lives on `MergeOutputOptions` (Chapter 6).

Testing with the container

For integration tests, register the real stack and turn verification on:

```C#
var services = new ServiceCollection();
services.AddSharpCorePDF(new PdfEngineOptions { VerifyOutput = true });
services.AddSingleton<MyReportService>();

await using var provider = services.BuildServiceProvider();
var report = provider.GetRequiredService<MyReportService>();

var bytes = await report.BuildAsync();
Assert.True(bytes.Length > 0);
```

An `InvalidOperationException` from the merge in this configuration means the engine wrote a document it could not read back — a defect worth reporting, not a problem with the input. Ordinary input problems still appear as `PdfException` subclasses.

`ServiceCollection` and `BuildServiceProvider()` come from `Microsoft.Extensions.DependencyInjection`, which a test project adds for itself: the registration extends `IServiceCollection` from `...Abstractions` and references no more than that (Chapter 3 has the one line it costs).

To test the layer above, replace `IPdfMerger` or `IPdfInspector` with a fake after the registration. Because the concrete services are registered separately, replacing only the interface leaves the engine available for tests that need it.

11. Architecture

SharpCorePDF is built as four layers around a pure domain model. This chapter explains the shape and the reasons behind it, so that you can judge where a change belongs and what a failure means.

The layers

Project	Contains	Depends on
`SharpCorePDF.Domain`	`MergePlan`, `PdfEditPlan`, `PdfSource`, value objects, policies, read models, warnings, exceptions, `IPdfContentSource`, `IPdfDocumentEditor`	Nothing but the BCL
`SharpCorePDF.Application`	`PdfMergerService`, `PdfDocumentEditorService`, `PdfInspectorService`, `IPdfDocumentReader`, `IPdfMergeEngine`, `IPdfEditEngine`, extension methods	Domain
`SharpCorePDF.Infrastructure`	The PDF reader, writer, object model, filters, copier, engine, content sources, and the authoring builder with its font and PNG writers	Domain, Application
`SharpCorePDF.DependencyInjection`	`AddSharpCorePDF()` and the service registrations	Domain, Application, Infrastructure
`SharpCorePDF` (meta-package)	No assembly; depends on the four above	—

Dependencies point inward:

```text
SharpCorePDF (meta)
        │
        ├── DependencyInjection ──┬── Infrastructure ──┐
        │                         ├── Application ─────┤
        │                         └── Domain ◄─────────┘
        └── (transitively, all four)
```

The domain does not know what a file is. It defines plans, page selections, output options and results, but the only way it reaches the outside world is `IPdfContentSource`, which is a stream factory with a display name. That is why a plan can be built, validated and tested without touching the disk.

The application layer orchestrates but does not parse PDF syntax. It validates a plan, opens each source, asks `IPdfDocumentReader` what is inside, resolves the page selections against the real page counts, and hands a `MergeEngineRequest` to `IPdfMergeEngine`. The edit service follows the same shape with one source: it validates the `PdfEditPlan` and hands it to `IPdfEditEngine`. The application layer also owns the lifetime of every stream it opens, so a failure in the middle does not leak a handle.

The infrastructure layer is the PDF engine. It reads bytes, parses objects, decrypts, walks page trees, copies object graphs, writes a cross-reference table and encrypts the result. It has no opinion about where a document came from or which pages the caller wanted; the request states both. The edit engine reuses that reader, copier and writer, flattening the page tree and applying the edit operations before the document is written.

Authoring lives in this layer too, as a pure factory rather than a port: `PdfDocumentBuilder` holds the pages and writes them with the same `PdfWriter` a merge uses, then returns the bytes as a `PdfByteArraySource`. No service registration is involved, and the domain gains only two value objects — `PdfColor` and `StandardFont` — because a colour and a standard face are facts about a document, not about the engine. The font parser, the subset writer and the PNG reader are internal to the infrastructure assembly, the same way the object parser and the filters are.

The dependency injection project is the composition root. It knows the concrete classes, and nothing else does.

The life of a merge

A call to `IPdfMerger.MergeAsync` passes through these stages:

1. **Argument checks.** A null plan or destination is a programmer error.
2. **Plan validation.** `EnsureValid()` checks the invariants that do not need a document: at least one source, unique identities, a primary source that belongs to the plan, defined enum values, usable encryption settings.
3. **Opening.** Each `IPdfContentSource` is asked for a readable stream. The streams stay open until the merge ends.
4. **Inspection.** `IPdfDocumentReader` reads each source, opens its encryption if a password was given, walks its page tree and reports `PdfDocumentInfo`. The application layer turns an unreadable encrypted source into `PdfEncryptionException` here, while it still knows which source it was.
5. **Resolution.** `PageSelection.Resolve` maps the one-based ranges onto real page numbers, collapses duplicates and preserves order; the engine skips a closed range's tail past the last page.
6. **Writing.** The engine receives the resolved sources and the output options. It copies the objects that the selected pages and retained features reach, renumbers them, builds one page tree, merges catalog-level features according to the policies, and writes the result to the destination.
7. **Release.** The application disposes every source stream in a `finally`, including on cancellation and on failure.

The engine never mutates the plan or the request. Everything it needs while running lives in `PdfMergeContext`, a per-merge object; that is what makes the registered services safe to share.

Why the plan is immutable

`MergePlan` is an immutable record. `AddSource`, `WithOutput` and the rest return new instances, and a source identity is created once and preserved by every operation on `PdfSource`. Three consequences are deliberate:

- A plan can be validated and then executed without a window in which another thread changes it.
- `MergePlanValidationException.Errors` describes a specific plan value, not a transient state.
- A plan is safe to log, cache or pass across layers. Its `ToString()` is stable and excludes passwords.

The cost is allocation on every edit. Building a plan of a hundred sources allocates a hundred plans; that is negligible next to reading the sources, and it buys predictable behaviour.

What the engine writes

The result is a new PDF, not an incremental update of a source. In particular:

- The merged document uses a classic cross-reference table, even when a source used cross-reference streams or object streams. Objects stored in object streams are written as ordinary objects.
- Only reachable objects are copied. A source's unused fonts, unused images or long-deleted revisions do not travel into the result.
- The object graph of each source is copied in two phases — reserve, then write — so cycles such as a form field pointing at its widget and back are handled without recursion loops.
- A single page tree node lists the selected pages in merge order. Page labels, outlines, annotations, form fields and attachments are handled by the output policies described in Chapter 6.
- The document is written as it is produced; there is no temporary full copy in the engine unless `VerifyOutput` is enabled, which captures the whole output so it can be re-read and checked.

The memory model

Two facts determine the memory profile, and both are worth knowing before processing large files:

1. **Each source is read completely into memory.** `PdfByteReader` reads a stream into a pooled buffer, and parsing resolves objects from those bytes on demand. `MaxSourceBytes` is the guard.
2. **Every source is held until the merge finishes.** The application inspects each source once, then the engine reads it again, and the engine keeps a reader per source while it copies objects from all of them. Peak memory is therefore roughly the sum of all source sizes, plus the destination's own buffering.

The destination is written in a streaming fashion, so a large result does not need a second full copy in memory. Verification is the exception: `VerifyOutput` deliberately captures the whole output.

For very large jobs, split the work into separate output documents, or merge subsets to temporary files and keep the intermediates on disk rather than in byte arrays. Staging reduces the work of each step, but the final merge still holds every intermediate it is given; it cannot lower the peak below the total size of all sources for one output document.

Boundaries and extension points

The ports are the seams:

Port	Implemented by	Replace it when
`IPdfContentSource`	`PdfFileSource`, byte and stream sources	Documents come from a database, blob store or network
`IPdfDocumentReader`	`PdfDocumentReader`	You need a different inspection strategy in tests
`IPdfMergeEngine`	`PdfMergeEngine`	You want to stub the writer while testing orchestration
`IPdfEditEngine`	`PdfEditEngine`	You want to test the editor without PDF work
`IPdfMerger`	`PdfMergerService`	You want to test a caller without any PDF work
`IPdfDocumentEditor`	`PdfDocumentEditorService`	Same, for editing
`IPdfInspector`	`PdfInspectorService`	Same, for inspection

A content source is small enough to write for any backing store. Each call must return an independent, readable stream positioned at the start, and `DisplayName` should identify the document in errors:

C#

```
using SharpCorePDF.Domain.Ports;

public sealed class StreamFactorySource(
    string displayName,
    Func<CancellationToken, ValueTask<Stream>> open) : IPdfContentSource
{
    public string DisplayName { get; } = displayName;

    public long? Length => null;

    public ValueTask<Stream> OpenReadAsync(CancellationToken cancellationToken = default) =>
        open(cancellationToken);
}
```

C#

```
var source = PdfSource.Create(new StreamFactorySource(
    "report.pdf",
    cancellationToken => storage.OpenReadAsync("report.pdf", cancellationToken)));
```

The merger takes ownership of the returned stream and disposes it, so the factory must return a fresh stream on every call. If the backing store can report a length cheaply, return it from `Length`; the current reader buffers from the stream's own `Length` when it is seekable, so the property is a declaration of what you know rather than something the engine relies on today.

What is *not* pluggable is equally deliberate: PDF filters, encryption handlers and the object model live inside the infrastructure layer. Adding a filter means changing the engine, not registering a service. The supported feature set is a

compatibility promise, and a plugin point would make it impossible to say what the library can read.

Concurrency

Every registered service is a singleton and stateless. Concurrent merges share the services but not the data: each call has its own request, context, readers and writer. Two rules follow for callers:

- **Do not share a destination stream between concurrent merges.** The stream itself is not synchronised, and interleaved PDF output is corrupt by construction.
- **Do share plans.** An immutable plan can be executed repeatedly and concurrently; the sources open a fresh stream per merge.

The content source is the only collaborator a caller supplies, and the contract requires `OpenReadAsync` to be safe to call more than once. A source backed by a non-seekable one-shot network stream cannot honour that contract; buffer it first, as the upload example in Chapter 10 does.

12. Authoring a Page

SharpCorePDF can start a document from nothing: `PdfDocumentBuilder` adds pages, draws lines of text with a standard or an embedded font, places PNG, JPEG, BMP and GIF images and draws shapes. It writes the document in memory, and the result is an ordinary PDF, so it can be inspected and merged like any other source. The use it was built for is a cover page: a logo, an address block and a few lines of text in front of a document that already exists. Measuring those lines, and wrapping a paragraph into lines, is Chapter 16; this chapter draws what a layouter hands it. The shape surface — paths, fill and stroke, clipping and transforms — has its own chapter, Chapter 15, and the formats an image can be read from, and what each one is refused for, are Chapter 17.

A page from nothing

```C#
var cover = PdfDocumentBuilder.Create()
    .AddPage(PdfPageSize.A4, page => page
        .DrawImage(logoPng, x: 40, y: 30, width: 152, height: 87)
        .DrawText("Veuillez trouver ci-joint le décompte", 150, 400,
            PdfTypeface.Standard(StandardFont.Helvetica), 12))
    .Build("cover.pdf");                    // a PdfByteArraySource, usable as a merge source

var plan = MergePlan.Create()
    .AddSource(PdfSource.Create(cover))
    .AddFile(@"C:\documents\statement.pdf");

await merger.MergeToFileAsync(plan, @"C:\documents\with-cover.pdf");
```

`Build()` writes the document and returns its bytes as a content source. `ToArray()` returns the same bytes, and `SaveAsync(Stream)` writes them to a stream you supply. Nothing touches the file system, so a cover can be produced, merged and discarded inside one request.

The builder collects pages and writes them when one of those three members is called. Adding a page afterwards does not change bytes that were already returned, and a builder is not thread safe: build one document on one thread. A typeface is immutable and may be shared by any number of builders.

Coordinates

The page uses the coordinate system a layout is written in, not the one PDF stores:

- the origin is the **top-left** corner of the page;
- **x** grows to the right, **y** grows downwards, in points (1 point = 1/72 inch);
- `DrawText` places the **baseline** of the text at `y`, not its top edge;
- `DrawImage` places the **top-left** corner of the image at `(x, y)`.

For an A4 page, `page.Width` is 595.276 and `page.Height` is 841.89. A line at `y = 400` sits 400 points below the top edge. The conversion to PDF's bottom-left origin is done for you, so nothing in an authored document needs the `page.Height - y` arithmetic.

Every call is independent: five lines of text are five calls at five `y` values. A line break inside one string is refused, because `DrawText` draws the baseline it is given and nothing else; a paragraph is laid out into lines first — Chapter 16 — and those lines are drawn, either one at a time or with `DrawTextLayout`.

Text and fonts

```C#
var heading = PdfTypeface.Standard(StandardFont.Helvetica);
var body = PdfTypeface.FromTrueType(File.ReadAllBytes("fonts/DejaVuSans.ttf"));

PdfDocumentBuilder.Create().AddPage(PdfPageSize.A4, page => page
    .DrawText("Décomptes SPRL", 330, 180, heading, 12, PdfColor.FromHex("#1F3A5F"))
    .DrawText("Veuillez trouver ci-joint le décompte pour l'immeuble 12345", 150, 400, body, 12)
    .DrawText("Sending 1/3", 150, 550, body, 12));
```

A **standard font** is one of the twelve faces every reader supplies: Helvetica, Times and Courier, each in four styles. It is written as a `/Type1` font with `/WinAnsiEncoding`, which needs no font file and keeps the document small. The encoding is Windows-1252: Latin-1 plus the typographic characters (curly quotes, en and em dashes, the euro sign). A character outside it — an arrow, a CJK ideograph, a ligature — is refused when the document is written, with the character and its code point in the message. That refusal is deliberate: writing `?` would silently change the text.

An **embedded font** is a TrueType program you supply as bytes. `PdfTypeface.FromTrueType` copies and parses it immediately, so a malformed program is reported where it is handed over rather than at the first page that draws with it. The font is embedded as a subset: the outlines of the glyphs the document draws are kept, the outlines of every other glyph are left out, and the glyph numbers of the source are preserved. A cover that draws two lines of text with the full 738 KB DejaVu Sans is a 58 KB document: the outlines of every other glyph are gone, and what remains is the per-glyph bookkeeping — glyph locations, advance widths and the character map — which keeps its size because the numbering is preserved. A `/ToUnicode` map travels beside it, so `é`, `è`, `à`, `ç`, `ô` and `ï` can be extracted again by `pypdf`, `pdftotext` or `IPdfContentReader.ReadTextAsync`.

Nothing is looked up in the operating system's font folder. The bytes you pass are the only source of the face, which is what makes an embedded font work in a container that has no fonts installed. The program must use TrueType (`glyf`) outlines; an OpenType font with PostScript (`CFF`) outlines is refused with an explanation.

Each rune of the text is mapped through the font's `cmap`. A code point the font does not cover is refused, and the message names the character, so a missing glyph becomes a build error rather than a blank space on a customer's statement.

The width of a line is not guessed: `PdfTypeface.Measure` returns it in points from the face's own data, and `PdfTypeface.Metrics` states the ascent, descent and line gap the face recommends. Chapter 16 covers both, together with `PdfTextLayouter`, which uses them to wrap a paragraph into lines that fit a column. The example above keeps a fixed step of 14 points between baselines because a cover needs no more than that.

Images

```C#
// A file opened once and drawn wherever the document needs it.
var logo = PdfImage.Open(File.ReadAllBytes("logo.jpg"));

page.DrawImage(logo, x: 40, y: 30, width: 152, height: 87); // explicit size in points
page.DrawImage(logo, x: 40, y: 30);                         // natural size

// The same two calls take the bytes directly, which reads them on every draw.
page.DrawImage(pngBytes, x: 40, y: 30, width: 152, height: 87);
```

`PdfImage.Open` decides the format from the signature of the bytes, never from a file name. `FromPng`, `FromJpeg`, `FromBmp` and `FromGif` do the same read with the format stated, and refuse a file whose signature disagrees rather than guessing at it. `Format`, `WidthPixels`, `HeightPixels`, `ColorSpace`, `BitsPerComponent`, `HasAlpha`, `NaturalWidth`, `NaturalHeight` and `FrameCount` report what the file holds. Two images opened from the same bytes state the same `Key` and become one image object in the document, so a logo drawn on forty pages is stored once.

Each format has its own policy — what is read, what is refused by name, and what the document stores — and Chapter 17 is its reference. `1.2` embeds all four: the JPEG, BMP and GIF readers and the colour profile a PNG may state arrived with this

release, and `1.1.0` embedded PNG images only. In short: a PNG is read at eight bits in its five colour types, where an alpha channel, a palette `tRNS` and a colour key each become the mask PDF defines for it, and an `iCCP` profile is declared as `/ICCBased` rather than dropped; a JPEG is passed through as its own bytes under `DCTDecode`; a BMP and a GIF are decoded, because PDF has no filter for their pixels, and a GIF contributes its first frame. Interlaced and sixteen bit PNGs, a JPEG whose EXIF orientation is not the identity, and the BMP compressions this reader does not decode are refused with the name of what was found.

The natural size of a PNG comes from its `pHYs` chunk, of a JPEG from its JFIF density and of a BMP from its pixels-per-metre fields, so a 300 dpi logo of 152 × 87 pixels is drawn 36.48 × 20.88 points. A file that states no resolution, which every GIF does, is drawn at one point per pixel.

SVG is not read at all, and a caller with another format re-saves it as one of the four.

Metadata, identity and reproducible output

```C#
var options = PdfAuthoringOptions.Default with
{
    CreationDate = new DateTimeOffset(2026, 1, 1, 0, 0, 0, TimeSpan.Zero),
    DocumentId = "00112233445566778899AABBCCDDEEFF",
};

var document = PdfDocumentBuilder.Create(options)
    .WithMetadata(new PdfMetadata
    {
        Title = "Décompte 12345",
        Author = "Décomptes SPRL",
        Creator = "Billing service",
    })
    .AddPage(PdfPageSize.A4, page => ...);
```

`WithMetadata` writes the document information dictionary. `Producer` is an authoring option, like it is an output option of a merge, and defaults to `SharpCorePDF 1.5.0`.

The document identifier is derived from the content and the creation date, so two builds of the same pages at the same moment are the same file. Supplying both `CreationDate` and `DocumentId` removes the last two sources of variation and makes the output reproducible byte for byte, which is what a test that compares files needs. `DocumentId` must be 32 hexadecimal digits.

An XMP packet, bookmarks and imported pages

```C#
var document = PdfDocumentBuilder.Create()
    .WithXmp(xmpPacket)                       // the catalog's /Metadata stream, byte for byte
    .WithOutline(
    [
        new PdfOutlineItem { Title = "Couverture", Destination = new PdfDestination { PageNumber = 1 } },
        new PdfOutlineItem
        {
            Title = "Sommaire",
            Destination = new PdfDestination { PageNumber = 2 },
            IsOpen = false,
            Children =
            [
                new PdfOutlineItem { Title = "Totaux", Destination = new PdfDestination { PageNumber = 2 } },
            ],
        },
        new PdfOutlineItem { Title = "Site", Destination = new PdfDestination { Uri = "https://example.com" } },
    ])
    .AddPage(PdfPageSize.A4, page => ...)
    .AddPage(PdfPageSize.A5, page => ...)
    .ImportPage(PdfFileSource.From("annexe.pdf"), 1);
```

`WithXmp` takes a packet as bytes, or as the `PdfXmpMetadata` a reader returned, and writes it to the catalog's `/Metadata` stream exactly as it was given and uncompressed — a validator compares the bytes it reads with the ones a producer signed, and PDF/A-1 forbids compressing them. Bytes that contain neither an `x:xmpmeta` nor an `rdf:RDF` element are refused at the call rather than written as metadata. It is how a caller carries a profile or a schema the library does not build itself.

`WithOutline` writes the bookmark tree, nesting included. A destination is a one-based page number of *this* document, a URL, or nothing; `IsOpen` states whether a viewer shows the children. Because pages can still be added after the outline is set, the page numbers are checked when the document is written: a bookmark that names a page the document does not have is refused there instead of being written dangling. A destination that states an `Area` becomes `/XYZ` at its top-left corner with the zoom left to the viewer; an external file destination, or a name that no table in the document resolves, is refused with `PdfUnsupportedFeatureException`.

`ImportPage` and `ImportPages` copy pages out of another document: the same copy a merge performs, on one page or one range. The source is read when the call is made, so a document that cannot be read, needs a password, or has no such page fails there and not when the result is written. The copy keeps its content, its resources, its annotations, its rotation and its geometry, and states on itself the attributes it inherited from its own page tree, because the authored document has a single flat one. A link or a bookmark that points at a page that was not imported points at the imported page rather than at nothing.

What it deliberately does not do

This is an authoring API for fixed-position content, not a page-flow engine. Text wraps, aligns and justifies inside a column — Chapter 16 — but there is no table, no list, no page break, no column set, no text shaping (kerning and ligatures come from the font program, not from this library), no form authoring, no redaction and no encryption of the authored page. SVG is not embedded, and neither is any format beyond the PNG, JPEG, BMP and GIF described in Chapter 17: a caller that has another format re-saves it. A document that needs any of those is better produced by a reporting library; SharpCorePDF can still merge the result.

Fixed-position **vector drawing** is part of this surface: lines, rectangles, ellipses, arbitrary paths of move/line/curve/close segments, filled or stroked with a colour and an alpha, dashed strokes, clipping and a saved graphics state. What it is not is a shape model with a scene graph: you place shapes at coordinates. Chapter 15 covers that surface in full — coordinates, style, clip, transforms and the state. Every shape is written into the page's content stream in the order you call it, so later shapes paint over earlier ones.

The authored page is written as PDF 1.7 with a classic cross-reference table, uncompressed content streams and Flate-compressed image and font streams. It is not validated against PDF/A or PDF/UA.

The surface at a glance

Member	What it does
`PdfDocumentBuilder.Create(options?)`	Starts a document; the options control the date, the identifier and the producer.
`WithMetadata(metadata)`	Sets title, author, subject, keywords, creator, dates and custom entries.
`WithXmp(packet)` / `WithXmp(xmpMetadata)`	Writes an XMP packet into the catalog exactly as given.
`WithOutline(bookmarks)`	Writes the bookmark tree, with nesting, open state and page or URL destinations.
`AddPage(size, draw?)`	Adds a page; `draw` receives a `PdfPageCanvas`, or `null` leaves the page blank.
`ImportPage(source, pageNumber)` / `ImportPages(source, range)`	Copies pages out of another document, with their content, resources, annotations and geometry.
`PdfPageCanvas.DrawText(text, x, y, typeface, size, color?)`	Draws one line at a baseline, in points, top-left origin.

Member	What it does
`PdfPageCanvas.DrawTextLayout(layout, origin)`	Draws every line of a `PdfTextLayout`, the top-left corner of its box at `origin`.
`PdfImage.Open(bytes)`	Reads a PNG, JPEG, BMP or GIF file; the `FromPng`, `FromJpeg`, `FromBmp` and `FromGif` factories state the format.
`PdfPageCanvas.DrawImage(image, x, y, width, height)`	Draws an image — bytes or a `PdfImage` — at an explicit size.
`PdfPageCanvas.DrawImage(image, x, y)`	Draws an image at the size the file's resolution states, one point per pixel when it states none.
`PdfPageCanvas.DrawLine(from, to, stroke)`	Draws one stroked segment.
`PdfPageCanvas.DrawRectangle(rect, fill?, stroke?)`	Fills and/or strokes a rectangle.
`PdfPageCanvas.DrawEllipse(bounds, fill?, stroke?)`	Fills and/or strokes an ellipse inside a box.
`PdfPageCanvas.DrawPath(path, fill?, stroke?)`	Fills and/or strokes a path of `PdfPathSegment` moves, lines, curves and closes.
`PdfPageCanvas.SetClip(path, fillRule?)`	Restricts everything drawn afterwards to a path.
`PdfPageCanvas.SaveState()` / `RestoreState()`	Brackets a change of fill, stroke, clip or transform.
`PdfTypeface.Standard(font)`	Selects one of the twelve standard fonts.
`PdfTypeface.FromTrueType(bytes)`	Embeds and subsets a TrueType program.
`PdfTypeface.Measure(text, size)` / `.Metrics`	The width of a line in points, and the face's vertical metrics in font units (Chapter 16).
`PdfTextLayouter.Layout(text, style, options)`	Wraps a paragraph into a `PdfTextLayout` that fits a width (Chapter 16).
`PdfColor.FromRgb(...)` / `FromHex(...)`	Selects a text colour.
`PdfPaint(color, alpha?, fillRule?)` / `PdfStroke(paint, width, …)`	States a fill, a stroke, their transparency and their dash pattern.
`ToArray()` / `SaveAsync(stream)` / `Build(name)`	Writes the document to bytes, to a stream, or as a merge source.

13. Cookbook

Short, complete recipes for the shapes that come up most often. Each one assumes `IPdfMerger` and `IPdfInspector` are available, as described in Chapter 10, and that the destination path is not one of the input files.

Merge every PDF in a folder, in a defined order

`EnumerateFiles` returns files in an unspecified order on most platforms. Sort explicitly, and sort numerically if your files are numbered without zero padding.

```C#
var files = Directory
    .EnumerateFiles(folder, "*.pdf", SearchOption.TopDirectoryOnly)
    .OrderBy(path => Path.GetFileName(path), StringComparer.OrdinalIgnoreCase)
    .ToArray();

if (files.Length == 0)
{
    throw new InvalidOperationException($"No PDF files found in '{folder}'.");
}

var plan = MergePlan.Create();

foreach (var file in files)
{
    plan = plan.AddFile(file);
}

await merger.MergeToFileAsync(plan, Path.Combine(folder, "merged.pdf"));
```

`merged.pdf` itself matches `*.pdf`; on a second run the plan would include the previous result and the destination would truncate it before the merge reads it. Filter the output name out, write outside the folder, or use the temporary-file pattern from Chapter 9.

For `report-2.pdf` before `report-10.pdf`, the default comparator is not enough: it compares text, so `report-10.pdf` sorts before `report-2.pdf`. Zero-pad the names, or sort with a natural-order comparer that understands embedded numbers.

Ask for a password only when one is needed

Inspection never throws for an encrypted document it cannot open; it reports it.

```C#
var content = new PdfFileSource(path);
var info = await inspector.InspectAsync(content, cancellationToken: cancellationToken);

string? password = null;

if (!info.IsReadable)
{
    if (!info.RequiresPassword)
    {
        throw new InvalidOperationException(
            $"'{info.Metadata.Title ?? Path.GetFileName(path)}' cannot be opened, and a password would not
help.");
    }

    password = PromptForPassword(path);

    if (password is null)
    {
        return;
```

```
    }

    info = await inspector.InspectAsync(content, password, cancellationToken);

    if (!info.IsReadable)
    {
        throw new InvalidOperationException("The password did not open the document.");
    }
}

var plan = MergePlan.Create().AddFile(path, password: password);
await merger.MergeToFileAsync(plan, destination, cancellationToken);

static string? PromptForPassword(string path)
{
    Console.Write($"Password for '{Path.GetFileName(path)}': ");
    return Console.ReadLine();
}
```

Two details matter. A document that is only restricted against editing opens with no password at all, so `password` stays `null` and the merge succeeds — asking for a password there would be confusing. And `InspectAsync` must be repeated with the password if you want to use the page count for a range; `info.PageCount` from the unreadable result is not trustworthy.

`Console.ReadLine` echoes the password. In a real client, use the password control of your UI or a masked console helper; the branching logic stays the same.

Extract a page range

Extraction is a merge of one source with a selection:

C#

```
var plan = MergePlan.Create()
    .AddFile("manual.pdf", pages: "3-7,12")
    .WithOutput(options => options with
    {
        OutlinePolicy = OutlinePolicy.Preserve,
        MetadataPolicy = MetadataPolicy.FromPrimarySource,
    });

await merger.MergeToFileAsync(plan, "excerpt.pdf");
```

The result contains six pages in the order written: 3, 4, 5, 6, 7, 12. With `OutlinePolicy.Preserve` the bookmarks of the source are carried over and renumbered onto the pages of the excerpt; the policy is the default. Page labels are dropped by default because a label such as "iv" no longer describes the same position in the new document. Set `PageLabelPolicy = PageLabelPolicy.FromPrimarySource` only when the copied labels still make sense from the first output page, as they do when the selection starts at page 1.

Encrypt the result

```C#
var options = new PdfEncryptionOptions
{
    Algorithm = PdfEncryptionAlgorithm.AesWith256BitKey,
    UserPassword = "reader-secret",
    OwnerPassword = "owner-secret",
    Permissions = PdfPermissions.PrintOnly,
};

var plan = MergePlan.Create()
    .AddFile("internal.pdf")
    .WithOutput(output => output with
    {
        TargetVersion = PdfVersion.V1_7,
        Encryption = options,
    });

await merger.MergeToFileAsync(plan, "internal-protected.pdf");
```

AES-256 requires PDF 1.7, which is the default target, so the explicit version is only there to show the dependency. The reader who supplies `reader-secret` may print but not copy text or modify the document; the owner password unlocks every permission. If you omit `OwnerPassword`, the user password acts as the owner password too — convenient, but then anyone who can open the document can also remove the restrictions.

Merge uploads in an ASP.NET Core endpoint

`IFormFile` gives you a stream; `AddStream` accepts one and reads it exactly once at merge time. Merging to bytes keeps the response simple because a byte array has a known length:

```C#
[HttpPost("merge")]
[RequestSizeLimit(100_000_000)]
public async Task<IActionResult> MergeAsync(
    [FromForm] List<IFormFile> files,
    CancellationToken cancellationToken)
{
    if (files.Count == 0)
    {
        return BadRequest("Upload at least one PDF.");
    }

    var plan = MergePlan.Create();

    foreach (var file in files)
    {
        plan = plan.AddStream(file.OpenReadStream(), file.FileName);
    }

    var merged = await merger.MergeToBytesAsync(plan, cancellationToken);

    return File(merged, "application/pdf", "merged.pdf");
}
```

The streams belong to the request and are read before the action returns, so no upload outlives the request. A total request limit is not optional: each source is buffered in full, so an unlimited upload is an unlimited allocation. If the result may be large, write it to a temporary file and return a `PhysicalFile` or `FileStreamResult` instead of holding a second copy in memory.

`MergeToBytesAsync` is all-or-nothing: an exception leaves no partial response body to send, because the body is only produced after the merge succeeded. Streaming the merge directly to `Response.Body` would be cheaper but sends a truncated document when a later source fails.

Report what was written

The result is meant to be logged as-is. Do not parse the text of a warning for control flow; branch on its `Kind`, which is the stable part of the contract.

```C#
var result = await merger.MergeToFileAsync(plan, destination, cancellationToken);

logger.LogInformation(
    "Merged {SourceCount} source(s) into {PageCount} page(s), {OutputBytes} bytes, PDF {Version} in {Duration}
ms.",
    result.SourceCount,
    result.PageCount,
    result.OutputBytes,
    result.OutputVersion,
    result.Duration.TotalMilliseconds);

foreach (var source in result.Sources.Where(source => !source.IsComplete))
{
    logger.LogInformation(
        "{SourceName}: {PagesContributed} of {SourcePageCount} pages selected.",
        source.SourceName,
        source.PagesContributed,
        source.SourcePageCount);
}

foreach (var warning in result.Warnings)
{
    logger.LogWarning("{WarningKind}: {WarningMessage}", warning.Kind, warning.Message);
}
```

`SourceCount` counts the sources of the plan, not distinct files: adding the same file twice produces two entries. `OutputBytes` is the size of the document the engine wrote; for `MergeToFileAsync` the file has already been flushed when the call returns, so it matches the file length.

How many warnings are too many is a policy decision. A production service usually logs known-benign kinds at information level and the rest at warning level, and keeps the warning kinds in its metrics so a rise in repaired documents is visible before users report it.

Override metadata and fix the modification date

`MetadataOverride` replaces only the fields you set. Everything else still follows `MetadataPolicy`, which is `FromPrimarySource` by default:

```C#
var created = new DateTimeOffset(2026, 1, 1, 0, 0, 0, TimeSpan.Zero);

var plan = MergePlan.Create()
    .AddFile("cover.pdf")
    .AddFile("body.pdf")
    .WithOutput(options => options with
    {
        MetadataOverride = new PdfMetadata
        {
            Title = "Annual Report 2026",
            Author = "Contoso",
            Subject = "Results for the financial year",
            Keywords = "annual report, 2026, results",
        },
        Producer = "Contoso Report Server 4.2",
        ModificationDate = created,
    });

await merger.MergeToFileAsync(plan, "annual-report.pdf");
```

The primary source is the first source unless `WithPrimarySource` says otherwise, so the title you did not override, the custom properties and the creation date come from `cover.pdf`. Viewer preferences, page layout and page mode are also taken from the primary source, but they are catalog entries rather than document information and are not affected by `MetadataPolicy`. To drop the metadata fields you did not override, set `MetadataPolicy = MetadataPolicy.None`; the policy starts from an empty information dictionary and the override is still applied on top. Chapter 6 lists the policies.

`Producer` and `ModificationDate` are output options: every merge writes them, and setting them pins the values that would otherwise describe the merge run. Neither can be set through `MetadataOverride`; `PdfMetadata.Producer` and `PdfMetadata.ModificationDate` are ignored for the result. Fixing the modification date removes one source of run-to-run difference. The output is reproducible only when the inputs, the modification date and the producer are the same; Chapter 6 explains the rest.

Split a large job into batches

Every source is held in memory until the merge ends, so a hundred large files can exhaust a process even though each one is below `MaxSourceBytes`. Merging in batches makes each intermediate job smaller and isolates a failure to one batch, but it does not make the final merge hold one batch: every batch file is a source of that merge too.

C#

```
static async Task MergeInBatchesAsync(
    IPdfMerger merger,
    IReadOnlyList<string> files,
    string destination,
    int batchSize,
    CancellationToken cancellationToken)
{
    if (files.Count == 0)
    {
        throw new ArgumentException("There must be at least one file to merge.", nameof(files));
    }

    if (batchSize < 1)
    {
        throw new ArgumentOutOfRangeException(nameof(batchSize), "The batch size must be at least one.");
    }

    var workDirectory = Path.Combine(Path.GetTempPath(), $"sharpcorepdf-{Guid.NewGuid():N}");
    Directory.CreateDirectory(workDirectory);

    try
    {
        var batches = new List<string>();

        for (var offset = 0; offset < files.Count; offset += batchSize)
        {
            var plan = MergePlan.Create();

            foreach (var file in files.Skip(offset).Take(batchSize))
            {
                plan = plan.AddFile(file);
            }

            var batchPath = Path.Combine(workDirectory, $"batch-{batches.Count:D4}.pdf");
            await merger.MergeToFileAsync(plan, batchPath, cancellationToken);
            batches.Add(batchPath);
        }

        var final = MergePlan.Create();

        foreach (var batch in batches)
        {
            final = final.AddFile(batch);
        }

        await merger.MergeToFileAsync(final, destination, cancellationToken);
    }
```

```
    finally
    {
        if (Directory.Exists(workDirectory))
        {
            Directory.Delete(workDirectory, recursive: true);
        }
    }
}
```

The cost is reading and writing the bytes twice and losing some document-level features twice: each intermediate merge applies the default policies, so page labels are already gone before the final merge. The final merge still reads and holds every batch, so peak memory is the total size of the batch files, not the size of one batch. If that total is too large, produce several smaller output documents, select fewer pages per document, or give the process more memory; the engine cannot write one merged document without holding all of its sources. If document-level features matter, keep the batch count low enough that the final merge is acceptable.

Accept a document only after checking it

A preflight gate turns a class of runtime failures into a clear rejection:

C#

```
const int MaximumPages = 2_000;

var content = new PdfFileSource(candidatePath);
PdfDocumentInfo info;

try
{
    info = await inspector.InspectAsync(content, cancellationToken: cancellationToken);
}
catch (PdfSourceFormatException exception)
{
    return Reject($"not a readable PDF: {exception.Detail}");
}

if (!info.IsReadable)
{
    return Reject(info.RequiresPassword
        ? "the document needs a password"
        : "the document uses an unsupported security handler");
}

if (info.PageCount == 0)
{
    return Reject("the document has no pages");
}

if (info.PageCount > MaximumPages)
{
    return Reject($"the document has {info.PageCount} pages; the limit is {MaximumPages}");
}

if (info.Warnings.Any(warning => warning.Kind is
        PdfWarningKind.PageTreeMissing or PdfWarningKind.PageTreeCycle))
{
    return Reject("the page tree is damaged; fix the source before uploading it");
}

return Accept(info);
```

A few warnings deserve rejection, most do not. `PageTreeMissing` means the document contributes no pages; `PageTreeCycle` means a page tree pointed back at itself. A repaired cross-reference table, an assumed page size or a truncated object stream on an otherwise readable document is usually a reason to log, not to refuse the file. Chapter 8 lists every kind with its meaning.

Notice that `PdfSourceFormatException` is caught even though inspection is supposed to report rather than throw. It still throws when the bytes are not a PDF at all or the limits are exceeded — those are not "encrypted but closed" conditions that a `PdfDocumentInfo` can describe.

Extract the text a document carries, and keep its coordinates

A search needs the words; a highlight needs where they are. One read returns both:

```C#
var reader = provider.GetRequiredService<IPdfContentReader>();
var source = PdfFileSource.From("report.pdf");

var text = await reader.ReadTextAsync(
    source,
    PageRange.FromTo(1, 20),
    cancellationToken: cancellationToken);

foreach (var block in text.Items)
{
    hits.AddRange(
        block.Words
            .Where(word => word.Text.Contains(term, StringComparison.OrdinalIgnoreCase))
            .Select(word => (block.PageNumber, word.Text, word.BoundingBox)));
}

foreach (var warning in text.Warnings)
{
    logger.LogWarning("{Kind}: {Message}", warning.Kind, warning.Message);
}
```

Blocks come in reading order — down the page, then along the line — so `block.Text` is a paragraph a search index or a translation call can take as it is, while `block.Words` carries the rectangle of every word for a viewer that has to draw a highlight. A `PdfWarningKind.TextUnreadable` warning is the one to expect on a scan, or on a font this library cannot decode: the words that could be read are still returned, and the warning names the font that was in the way. Text drawn inside a form is included, because the read follows the form's matrix and the resources it inherits (Chapter 8).

This is reading, not conversion: nothing is reflowed, and a two-column page comes back as blocks that alternate between the columns. When a caller knows the layout, `BoundingBox.Left` is enough to sort the blocks again before merging them.

Write a cover page and put it in front of a document

A batch of statements needs one cover per document: a logo, the addressee and two lines of text. The cover is authored in memory and merged as the first source, so no temporary file exists at any point.

C#

```
public async Task<string> AddCoverAsync(
    string statementPath,
    string logoPath,
    string fontPath,
    CancellationToken cancellationToken)
{
    // Both are read once per process in a service; a typeface is immutable and can be shared.
    var logo = await File.ReadAllBytesAsync(logoPath, cancellationToken);
    var body = PdfTypeface.FromTrueType(await File.ReadAllBytesAsync(fontPath, cancellationToken));

    var cover = PdfDocumentBuilder.Create(PdfAuthoringOptions.Default with
    {
        CreationDate = new DateTimeOffset(2026, 1, 1, 0, 0, 0, TimeSpan.Zero),
    })
        .WithMetadata(new PdfMetadata { Title = "Décompte 12345", Creator = "Billing" })
        .AddPage(PdfPageSize.A4, page => page
            .DrawImage(logo, x: 40, y: 30, width: 152, height: 87)
            .DrawText("Décomptes SPRL", 330, 180, PdfTypeface.Standard(StandardFont.Helvetica), 12)
            .DrawText("Rue de l'Église 1", 330, 194, PdfTypeface.Standard(StandardFont.Helvetica), 12)
            .DrawText("1000 Bruxelles", 330, 208, PdfTypeface.Standard(StandardFont.Helvetica), 12)
            .DrawText("Veuillez trouver ci-joint le décompte pour l'immeuble 12345", 150, 400, body, 12)
            .DrawText("pour la période du 01/01/2025 au 31/12/2025.", 150, 414, body, 12)
            .DrawText("Sending 1/3", 150, 550, body, 12))
        .Build("cover.pdf");

    var destination = Path.ChangeExtension(statementPath, ".with-cover.pdf");

    var plan = MergePlan.Create()
        .AddSource(PdfSource.Create(cover))
        .AddFile(statementPath);

    await merger.MergeToFileAsync(plan, destination, cancellationToken);
    return destination;
}
```

The coordinates are points from the top-left corner, and the `y` of `DrawText` is the baseline. The `PdfTypeface.Standard` calls create the same typeface every time, which is cheap but needless: hoist one instance next to the embedded face in a service. Keep the creation date fixed if the cover is compared between runs, and leave it out when the real creation moment matters (Chapter 12).

The text round-trips: `ReadTextAsync` gives back `Veuillez trouver ci-joint le décompte…` with its accents, because the embedded font carries a `ToUnicode` map. The standard font covers the address block because it is Windows-1252 text; a character outside that encoding would have been refused when the cover was written, and an embedded face is the answer for anything else.

When the paragraph comes from a template

Two hand-placed lines are fine while the sentence is fixed. When the wording comes from a template, the lines have to be computed, and Chapter 16 is the surface for it (added in `1.2`; `1.1.0` drew one line per call). Put the first baseline where the hand-placed line was, so the cover does not have to be re-measured:

C#

```
// bodyText is the paragraph the template produced; body is the embedded face from above.
var style = new PdfTextStyle(body, 12, leading: 15);
var paragraph = PdfTextLayouter.Layout(bodyText, style, new PdfTextLayoutOptions(maxWidth: 295));

var metrics = body.Metrics;
var firstBaseline = metrics.Ascent * style.FontSize / metrics.UnitsPerEm;
var top = 400 - firstBaseline;          // the box corner that puts the first baseline back at 400
```

`295` is the A4 width minus a 150-point margin on each side. The block is drawn with one call, and the next block is placed at `top + paragraph.Bounds.Height`:

```C#
page.DrawTextLayout(paragraph, new PdfPoint(150, top));
page.DrawText("Sending 1/3", 150, top + paragraph.Bounds.Height + 20, body, 12);
```

`Bounds.Height` runs from the top of the box to the descender of the last line, so it is the space the paragraph actually needs — not the number of lines times the leading. A template that overflows the column throws at `Layout`, naming the word that did not fit, unless `BreakWords` or a `Clip` / `Continue` policy is stated.

14. Editing an Existing Document

A merge builds one document from several. An edit changes one source document and writes the result as a new document. The source is only read: an edit never modifies it, needs no write access to it and leaves no temporary file beside it.

Editing here means changing the **assembly** of the document — which pages it has, in what order, with what rotation — and its document information. It does not mean changing what a page draws; Chapter 2 states that boundary.

One plan, applied in order

```C#
using SharpCorePDF.Domain.Aggregates;
using SharpCorePDF.Domain.ValueObjects;
using SharpCorePDF.Infrastructure.Sources;

var plan = PdfEditPlan.Create()
    .RotatePage(1, PageRotation.Clockwise90)
    .DeletePage(4)
    .MovePage(3, 1)
    .DuplicatePage(2, 4)
    .SetMetadata(new PdfMetadata { Title = "Revised statement" });

await using var output = File.Create("statement-edited.pdf");
var result = await editor.ApplyAsync(
    new PdfFileSource("statement.pdf"), plan, output);
```

`editor` is an `IPdfDocumentEditor`, registered by `AddSharpCorePDF()` and constructible by hand as `new PdfEditEngine()` (Chapter 10). The plan above is read against a four page source like this:

Operation	Pages after it
source	`1 2 3 4`
`RotatePage(1, Clockwise90)`	`1 2 3 4`, page 1 rotated
`DeletePage(4)`	`1 2 3`
`MovePage(3, 1)`	`3 1 2`
`DuplicatePage(2, 4)`	`3 1 2 1` — a copy of page 2 appended
`SetMetadata(...)`	`3 1 2 1`, information overridden

Page numbers are one-based and refer to the state produced by every preceding operation. That is the whole ordering rule: a plan reads from top to bottom, and changing one operation changes what the next one addresses.

Every method returns a new plan. A plan is immutable, so it can be built once, validated once, shared between requests and applied concurrently without a lock.

The operations

Method	What it does
`RotatePage(pageNumber, rotation)`	Adds a quarter turn to the page's declared rotation. Rotation is additive and normalised: two `Clockwise90` operations make `Clockwise180`. `Clockwise270` is allowed.
`DeletePage(pageNumber)`	Removes one page. An edit that would leave the document with no page is refused.
`MovePage(from, to)`	Moves a page so that it ends up at position `to` in the state that exists before the move.
`DuplicatePage(pageNumber, insertAt)`	Copies a page and inserts the copy before `insertAt`. `insertAt` may be one past the current count, which appends. Both copies draw the same objects; the page entries are distinct.

Method	What it does
`CropPage(pageNumber, cropBox)`	Sets the visible area of a page inside its media box. A box that reaches outside the media box is clamped to the part that overlaps it and reported as `PdfWarningKind.PageGeometryNormalized`; a box with no overlap is refused.
`ResizePage(pageNumber, size, mode)`	Changes the page size and makes the content follow it. `PdfPageResizeMode.Scale` scales the content to the new size; `PdfPageResizeMode.Preserve` keeps it at its original size and centres it. The original content streams are wrapped, not rewritten.
`SetMetadata(metadata)`	Merges document information into the source values. Fields left `null` keep their source value; fields set are replaced. Several metadata operations merge in plan order.
`SetXmp(xmp)`	Replaces the XMP packet with the bytes given. The packet must be well-formed XML; nothing in it is rewritten.
`SetOutline(outline)`	Replaces the bookmarks. Page numbers are the ones the edit leaves behind; `PdfOutline.Empty` removes the panel.
`SetPageLabels(labels)`	Replaces the page labels with explicit ranges; `PdfPageLabeling.None` removes them.
`SetViewerPreferences(preferences)`	Replaces what a viewer shows when it opens the document: window chrome, panel, page layout, binding side and print scaling.
`Overlay(overlay, pageNumber, options)`	Records one page of another document over a page.
`Underlay(underlay, pageNumber, options)`	Records one page of another document under a page.
`AddAnnotation(pageNumber, annotation)`	Draws a comment, a stamp or a link over a page without touching its content (chapter 18).
`UpdateAnnotation(pageNumber, id, annotation)`	Replaces the annotation that carries an identifier.
`RemoveAnnotation(pageNumber, id)`	Removes the annotation that carries an identifier.
`SetForm(form)` / `SetFormField(...)`	Writes a form, or fills the fields of the one the document carries, by name (chapter 19).
`ResetForm()`	Puts every field back to the value the document states as its default.
`FlattenForm()` / `FlattenForm(pageNumber)`	Draws the widgets into the page and removes the form.

The operation objects are public values (`PdfRotatePageOperation`, `PdfDeletePageOperation`, `PdfMovePageOperation`, `PdfDuplicatePageOperation`, `PdfCropPageOperation`, `PdfResizePageOperation`, `PdfSetMetadataOperation`, `PdfOverlayOperation`, `PdfSetXmpOperation`, `PdfSetOutlineOperation`, `PdfSetPageLabelsOperation`, `PdfSetViewerPreferencesOperation`, `PdfAddAnnotationOperation`, `PdfUpdateAnnotationOperation`, `PdfRemoveAnnotationOperation`, `PdfSetFormOperation`, `PdfResetFormOperation`, `PdfFlattenFormOperation`), but the methods on `PdfEditPlan` are the intended construction route: they check the invariant that can be checked without a document, and they keep the plan immutable.

The two surfaces that write over a page — annotations and forms — have chapters of their own, because each carries a model a caller has to understand before the plan makes sense: chapter 18 for the annotation kinds and their appearance streams, chapter 19 for the field kinds, the merge by name and the flatten.

Overlays and underlays

An overlay or underlay records one page of another document on a page of the one being edited: a stamp, a watermark, a letterhead.

```C#
var plan = PdfEditPlan.Create()
    .Underlay(new PdfFileSource("letterhead.pdf"), 1)
    .Overlay(new PdfFileSource("approved.pdf"), 2, new PdfOverlayOptions
    {
        Opacity = 0.5,
        Placement = PdfOverlayPlacement.Fit,
        BlendMode = PdfOverlayBlendMode.Multiply,
    });
```

The layer is content, not an annotation. It is written as a form XObject that keeps the resources of the document it came from, so a name it uses for a font or an image can never collide with a name the page already means something else by. The page's own streams are copied unchanged: an overlay appends a stream that draws the form, an underlay prepends one, and the bytes the page carried stay in between, wrapped in the `q` / `Q` pair the contract states.

Option	Effect
`Placement`	`Fit` scales the layer until it fits the visible area and centres it, keeping the aspect ratio. `Stretch` fills the area in both directions, so the aspect ratio may change. `OriginalSize` scales nothing and anchors the layer at the lower-left corner.
`Opacity`	From `0` (invisible) to `1` (opaque). It is written through an `ExtGState` of the layer's own, so it applies to the layer alone.
`BlendMode`	`Normal`, `Multiply`, `Screen`, `Overlay`, `Darken` or `Lighten`; also written through that `ExtGState`.
`Clip`	Clips the layer to the visible area of the page. Defaults to `true`; an overlay that reaches past the page would otherwise paint outside it.

The document a layer comes from must hold exactly one page. The visible area is the page's `CropBox` when it has one and its `MediaBox` otherwise, after any crop or resize that preceded the layer. The layer travels with the page it was applied to: `MovePage` carries it, `DeletePage` removes it with the page, and `DuplicatePage` makes both copies draw the same form.

`PDF/A-1b` is built on a version of PDF that has no transparency, so a plan written for it refuses a layer with an opacity below `1` or a blend mode other than `Normal` rather than writing the layer opaque without saying so. The validation happens when the plan is checked, before the source is read.

Document-level edits

Four operations state a value of the document rather than of a page. Each replaces what the source carried, a later operation of the same kind replaces an earlier one, and none of them changes the pages:

- `SetOutline` writes the bookmark tree as stated. Page numbers are one-based in the order the edit leaves the pages in; a bookmark that names a page the result does not have is refused with `PdfEditPlanValidationException` before anything is written. `PdfOutline.Empty` writes no `/Outlines` entry at all, which removes the panel the source carried.
- `SetPageLabels` writes one range per numbering style — `Decimal`, `UpperRoman`, `LowerRoman`, `UpperLetters`, `LowerLetters` or `NoNumber` — each with an optional prefix and a start number. A range that starts past the last page of the result is refused. `PdfPageLabeling.None` removes the labels, so the edit reports no `PageLabelsDropped` warning for a change the caller made itself.
- `SetViewerPreferences` writes `HideToolbar`, `HideMenubar`, `HideWindowUi`, `FitWindow`, `CenterWindow`, `DisplayDocTitle`, `PageLayout`, `PageMode`, `ReadingDirection` and `PrintScaling`. The page layout and the page mode are catalog entries; the rest is written to `/ViewerPreferences`.
- `SetXmp` writes the packet as given, as UTF-8 XML, and never rewrites it. The plan refuses a packet that is not well-formed, and one set on a document written for a PDF/A profile, whose own packet declares conformance.

Output options

`PdfEditOutputOptions` controls the document that is written:

Property	Effect
`TargetVersion`	The version in the `%PDF-` header. PDF 1.4 to 2.0; the default is 1.7.
`Metadata`	Information values applied after every `SetMetadata` operation. It is the last override, not a replacement for the source values.
`ModificationDate`	The value written as `ModDate`. When unset, the moment the edit ran is used.
`Producer`	The value written as `Producer`. It defaults to the library's producer string; the source value is never kept.
`DocumentId`	An optional identity for the result. It participates in the trailer identifier.

`Producer` and `ModDate` always describe the document that was just written, never the document it was made from. The other information fields follow the source values and the overrides.

Supply the options when the plan is created, or replace them later:

```C#
var plan = PdfEditPlan.Create()
    .RotatePage(2, PageRotation.Clockwise180)
    .WithOutput(PdfEditOutputOptions.Default with
    {
        TargetVersion = PdfVersion.V2_0,
        ModificationDate = new DateTimeOffset(2026, 4, 1, 12, 0, 0, TimeSpan.Zero),
        DocumentId = "statement-2026-04",
    });
```

A fixed `ModificationDate` and `DocumentId` make repeated edits of the same source byte-identical. The trailer identifier is derived from the output version, the modification date, the page count, the document identifier and the operations that were applied.

What survives an edit

Every surviving page keeps its content streams, resources, fonts, images and annotations. Stream bytes are copied verbatim; nothing is re-compressed and no page is re-rendered. The page tree is flattened, so inherited `MediaBox`, `CropBox`, `Resources` and `Rotate` are materialised onto each page.

Document-level features are carried over from the source. Two of them cannot be carried over truthfully after some edits, so they are dropped and reported rather than left to contradict the result — unless the plan states them itself, in which case the stated value is written:

- **Page labels** are dropped when an operation changes the page order or count — delete, move or duplicate. A rotation alone does not change the labels, so they survive it. The result carries `PdfWarningKind.PageLabelsDropped`; `SetPageLabels` states them instead, and `PdfPageLabeling.None` removes them without the warning.
- **The XMP packet** is dropped when the information dictionary is overridden, whether by `SetMetadata` or by `PdfEditOutputOptions.Metadata`. Keeping an old packet would leave two descriptions of the same document that disagree. The result carries `PdfWarningKind.XmpPacketDropped`; the information dictionary holds the new values, and `SetXmp` replaces the packet instead of dropping it.

Bookmarks, named destinations and form widgets that name a page are copied with their references redirected to the first surviving occurrence of that page, or to the first page of the result when the page did not survive. A bookmark whose page was deleted therefore still opens a real page, and a duplicate does not produce a second target for the same reference.

Validation and failure

A plan checks what it can without opening a document: page numbers must be positive, a rotation must be a quarter turn, a crop box must enclose an area, a resize mode must be `Scale` or `Preserve`, and the output options must name a supported version and non-empty strings where a value is set. `Validate()` returns every problem; `EnsureValid()` throws a single `PdfEditPlanValidationException` that lists them.

The rest is checked when the source has been read, because only then is its page count known. A page number beyond the last page at that point, an insert position past the end, or an operation that would delete the last page fails with the same exception type. These failures happen before the destination is written, so a failed edit does not leave a half document behind.

The source itself can still fail for the reasons any read can fail: a malformed file raises `PdfSourceFormatException`, and an encrypted file that cannot be opened with the supplied password raises `PdfEncryptionException`. Both are described in Chapter 9.

What an edit cannot change

An edit copies pages; it does not re-render them. There is no operation that replaces text or an image, removes or edits an annotation, fills a form field, redacts content, signs the result or adds a digital signature. `CropPage` and `ResizePage` change the geometry a viewer uses, but they do not change the bytes of the content streams they show. The source can be encrypted and is opened with the `password` argument of `ApplyAsync`; the result is written in the clear, because output encryption is not part of this editing release.

The writer produces a complete new document rather than an incremental update. That is what keeps earlier revisions and signatures out of a document whose pages have changed; it also means a copied signature is not a signature of the edited document.

Reading the result

`ApplyAsync` returns a `PdfEditResult`:

```text
edited into 4 page(s), 18,204 bytes, PDF 1.7 in 6 ms
```

`PageCount`, `BytesWritten`, `OutputVersion` and `Elapsed` describe the write. `Operations` lists the operations in plan order, formatted for diagnostics, and `Warnings` holds the remarks described above. `ApplyToArrayAsync` performs the same edit and returns the bytes; use it only when the result fits in memory, and use the stream overload for a large document.

15. Drawing

Chapter 12 starts a page from nothing: text in a standard or an embedded font, PNG, JPEG, BMP and GIF images, and a document written to bytes. This chapter is the rest of that surface — the shapes. Lines, rectangles, ellipses and arbitrary paths, the paint and stroke style applied to them, clipping, transforms and the saved graphics state all shipped in `1.1.0`; the colour-space selection in *Painting in a colour space of your own* arrived in `1.5.0`.

Every call appends operators to the page's content stream in the order you make it, so the calls read like the drawing they produce: the background first, the shapes that cover it after. There is no scene graph, no retained shape and no second layout pass. What you draw is what the stream contains.

Coordinates, and the one name that misleads

A canvas measures from the **top-left** corner of the page, with **y** growing downwards, in points. Page space — the coordinates the PDF file itself stores — has its origin at the bottom-left corner with y growing upwards. The conversion happens when the page is written, so a caller never writes `page.Height - y`.

`PdfRectangle` keeps PDF's edge names: `left`, `bottom`, `right`, `top`, where `bottom` is the edge with the smaller y and `top` the one with the larger y. On a canvas that means `bottom` **is the edge nearer the top of the page**:

```C#
// A 100 x 50 point box whose upper edge is 20 points below the top of the page.
var box = new PdfRectangle(left: 40, bottom: 20, right: 140, top: 70);

box.Width;      // 100
box.Height;     // 50
box.IsEmpty;    // false
```

If the names get in the way, read the constructor as "left, smaller y, right, larger y" — that pair is always in that order. `PdfRectangle.FromCorners(x1, y1, x2, y2)` builds the same rectangle from any two opposite corners and normalises them, so `(140, 70, 40, 20)` describes the same box. A `right` smaller than `left`, a `top` smaller than `bottom`, or a coordinate that is not a finite number is an `ArgumentOutOfRangeException`, raised where the value is created rather than when the page is written.

Lines

```C#
page.DrawLine(
    new PdfPoint(72, 120),
    new PdfPoint(523, 120),
    new PdfStroke(PdfPaint.Black, width: 0.75));
```

A line is one open subpath: a move, a line, and the paint. The `Cap` of the stroke decides what the two ends look like, and a single segment has no join, so `Join` and `MiterLimit` are written but have nothing to act on. `DrawLine` takes a `PdfStroke` and no fill: filling an open line is nothing a reader will show you.

Rectangles

```C#
page.DrawRectangle(
    new PdfRectangle(40, 20, 140, 70),
    fill: new PdfPaint(PdfColor.FromHex("#1F3A5F"), alpha: 0.08),
    stroke: new PdfStroke(PdfPaint.Black, width: 0.5));
```

The rectangle is a closed subpath. Pass a fill, a stroke, or both; supplying neither is an `ArgumentException`, because a path that is constructed and never painted is a mistake rather than a no-op. A box with no width or no height is refused for the same reason. When both are supplied the shape is painted with one operator (`B`, or `B*` under the even-odd rule), so no other drawing can slip between its fill and its outline.

Ellipses

`DrawEllipse(bounds, fill?, stroke?)` fits an ellipse inside a box. The box does not have to be square, and an ellipse in a wide box is a wide ellipse. The curve is four cubic Béziers with the control-point constant every PDF producer uses — 0.5522847498 × the radius — which leaves a worst-case radial error of about 0.03% of the radius: a thousandth of a point on a four-centimetre radius, below what a printer resolves. A caller who needs an exact curve draws it with `DrawPath` and as many segments as the job requires.

Paths

`PdfPath` is a list of `PdfPathSegment` records and a flag. The five segment kinds are `MoveTo`, `LineTo`, `CubicTo` (two control points and an end), `QuadraticTo` (one control point and an end) and `Close`:

```C#
var arrow = new PdfPath(
    [
        new PdfPathSegment.MoveTo(new PdfPoint(100, 400)),
        new PdfPathSegment.LineTo(new PdfPoint(220, 400)),
        new PdfPathSegment.LineTo(new PdfPoint(220, 388)),
        new PdfPathSegment.LineTo(new PdfPoint(244, 406)),
        new PdfPathSegment.LineTo(new PdfPoint(220, 424)),
        new PdfPathSegment.LineTo(new PdfPoint(220, 412)),
        new PdfPathSegment.LineTo(new PdfPoint(100, 412)),
    ],
    isClosed: true);

page.DrawPath(arrow, new PdfPaint(PdfColor.Black), stroke: null);
```

Four rules, all checked when the path is constructed:

- the segments may not be empty, and the first must be a `MoveTo` — a path that does not start somewhere is ambiguous about where its first segment begins;
- a null segment is refused rather than skipped;
- `Close` closes the current subpath back to its own start point, which matters in a path with several subpaths, while `isClosed` closes after the last segment. Use one or the other, not both;
- a `QuadraticTo` is converted to the equivalent cubic when it is written, because PDF has no quadratic operator. The curve is the same curve, not an approximation of it.

A second `MoveTo` starts a new subpath. When the path is filled, the fill rule decides what a subpath inside another subpath means: `PdfFillRule.NonZero` (the default) fills the *union*, so an inner subpath drawn the same way round does not cut a hole; `PdfFillRule.EvenOdd` alternates, so it does. The rule is a property of the paint and reaches the stream as `f*` / `B*` instead of `f` / `B`. It has no effect on a stroke.

Fill and stroke

Two immutable values describe how a shape is painted. `PdfPaint` states the colour, its opacity, the fill rule and the device colour space. `PdfStroke` states a paint plus the line style:

```C#
var fill = new PdfPaint(
    PdfColor.FromRgb(0x1F, 0x3A, 0x5F),
    alpha: 0.35,                              // 0 to 1, through an /ExtGState
    fillRule: PdfFillRule.EvenOdd,
    colorSpace: PdfColorSpace.DeviceRGB);     // DeviceGray, DeviceRGB (default) or DeviceCMYK

var stroke = new PdfStroke(
    paint: PdfPaint.Black,
    width: 1.2,
    cap: PdfLineCap.Round,
    join: PdfLineJoin.Miter,
    miterLimit: 4,
    dashArray: [6, 2, 1, 2],                  // dash, gap, dot, gap
    dashPhase: 1,                             // one point into the pattern
    strokeAdjustment: PdfStrokeAdjustment.Enabled);
```

Member	Default	What it states	Written as
`PdfPaint.Color` / `.ColorSpace`	black / `DeviceRGB`	The device colour, and the space its components are written in.	`rg` , `g` or `k` (fill); `RG` , `G` or `K` (stroke)
`PdfPaint.Alpha`	`1.0`	One opacity for the whole shape.	`/ca` and `/CA` in an `/ExtGState` , selected with `gs`
`PdfPaint.FillRule`	`NonZero`	What counts as inside when filling.	`f` vs `f*` , `B` vs `B*`
`PdfStroke.Width`	`1.0`	The line width in points.	`w`
`PdfStroke.Cap`	`Butt`	The end of an open subpath: `Butt` , `Round` or `Square` .	`J` (`0` , `1` , `2`)
`PdfStroke.Join`	`Miter`	Where segments meet: `Miter` , `Round` or `Bevel` .	`j` (`0` , `1` , `2`)
`PdfStroke.MiterLimit`	`10.0`	How far a miter may run before it is cut to a bevel.	`M`
`PdfStroke.DashArray`	empty	Dash and gap lengths **in points** — not multiples of the width, which is the PostScript convention.	`d`
`PdfStroke.DashPhase`	`0.0`	How far into the pattern the line starts.	`d`
`PdfStroke.StrokeAdjustment`	`null`	Whether the viewer may snap strokes to device pixels.	`/SA` in an `/ExtGState`

Alpha is not a fourth colour component: the viewer composites it, so it applies to the shape as a whole rather than to each pixel of a photograph, and it costs one entry in the page's graphics-state resources. The writer keeps one entry per distinct combination of fill alpha, stroke alpha and stroke adjustment, so a thousand translucent shapes share one `/GS1` ; a paint whose alpha is `1` needs no entry at all.

The refusals are the ones a caller is likely to hit, and they arrive before anything is recorded: a width that is not a positive finite number, a miter limit below 1, a dash phase that is not finite, a dash array with a negative or non-finite value or one whose values are all zero (PDF would draw nothing), and an enum value the engine does not define. The dash array is **copied** when the stroke is created, so mutating the list you passed in cannot reach a stroke that is already recorded.

The writer emits the path, then the style, then the painting operator. That order is legal — a viewer takes the style at the moment the path is painted — and it has a consequence worth knowing: the stream cannot be read as "set the style, then draw", and a shape that has already been painted cannot be restyled.

Painting in a colour space of your own

A `PdfPaint` carries a device colour, so it cannot state a separation, a device-N space or a pattern. For those the canvas exposes the selection operators and a way to paint with what they selected:

```C#
canvas
    .SetFillColor("DeviceCMYK", [0.10, 0.20, 0.30, 0.40])   // cs then scn
    .FillPath(square)                                        // f, with no colour operator of its own
    .SetStrokeColor("/Cs1", [0.25])                          // CS then SCN — a name the resources define
    .StrokePath(line);                                       // S
```

`SetFillColor` and `SetStrokeColor` write `cs` / `CS` and `scn` / `SCN`; `FillPath` and `StrokePath` paint without writing a colour, so the selection stands. Every other drawing call states its own paint and therefore wins over a selection — `DrawRectangle(rectangle, paint: somePaint)` after `SetFillColor` paints `somePaint`, because no API can know which of the two you meant. The name is written as you state it, so it must already be a legal PDF name: a resource name such as `Cs1`, or `DeviceGray`, `DeviceRGB`, `DeviceCMYK`. A name that would need `#` escaping is refused rather than escaped, and so is a colour with no component or a component that is not finite.

Clipping

`SetClip` restricts everything drawn afterwards to the inside of a path — the usual way to crop a photograph, to confine a table to a column, or to keep an overlay inside a rounded panel:

```C#
var panel = new PdfPath(
    [
        new PdfPathSegment.MoveTo(new PdfPoint(40, 40)),
        new PdfPathSegment.LineTo(new PdfPoint(555, 40)),
        new PdfPathSegment.LineTo(new PdfPoint(555, 300)),
        new PdfPathSegment.LineTo(new PdfPoint(40, 300)),
    ],
    isClosed: true);

page.SaveState()
    .SetClip(panel, PdfFillRule.EvenOdd)
    .DrawImage(photographPng, 0, 0, page.Width, page.Height)
    .RestoreState();
```

A clip is part of the graphics state, not of the operation that follows it. It stays in force for every later drawing until it is restored, and it can only be undone by `RestoreState`, never by setting an empty clip. A clip set outside a save therefore applies to the rest of the page, which is occasionally what you want and usually a mistake. The fill rule decides which side of the path is inside, exactly as it does for a fill, and the call paints nothing itself.

Transforms

`SetTransform` concatenates a matrix onto the current transform. The matrix is stated in canvas coordinates — the same top-left, y-down system as every coordinate in this chapter — and the conversion to page space is folded into the `cm` operator, so a translation of `(10, 20)` moves content ten points right and twenty points down the page:

```C#
page.SaveState()
    .SetTransform(PdfMatrix.Scale(2).Multiply(PdfMatrix.Translate(60, 40)))
    .DrawRectangle(new PdfRectangle(0, 0, 100, 50), fill, stroke: null)
    .RestoreState();
```

`Multiply` follows PDF's row-vector convention: `left.Multiply(right)` applies `left` first and `right` afterwards, so the example scales about the origin and then moves the result. Compose a group in that order — scale, then rotate, then translate — or a rotation will turn the placement as well as the content, because `Translate(60, 40).Multiply(Rotate(-15))` rotates the point the group was moved to. The other factories are `Scale(x, y)` and `Rotate(degrees)`, and any matrix can be built directly with the six-argument constructor `new PdfMatrix(a, b, c, d, e, f)`.

A rotation deserves one sentence of care, because the sign surprises almost everyone the first time. The canvas y axis points **down**, so `PdfMatrix.Rotate(90)` maps a point ten points to the right of the origin to a point ten points *below* it: the content turns clockwise on the page. Use `Rotate(-90)` for a counter-clockwise turn. The same reason makes `Scale(1, -1)` a flip about the horizontal line through the origin rather than about the bottom of the page.

A transform applies to everything drawn after it, not only to shapes: text baselines placed with `DrawText` or `DrawTextLayout`, and images placed with `DrawImage`, are all written through the current transform. That is what makes a rotated stamp — text and logo together — one `SetTransform` rather than arithmetic on every coordinate. Bound it with `SaveState` and `RestoreState` unless you intend the rest of the page to move.

The saved graphics state

`q` saves the graphics state and `Q` restores it. On the canvas the pair is `SaveState()` and `RestoreState()`, and what travels in it is everything the sections above change: the fill paint, the stroke style, the alpha pair, the clipping path and the transform.

```C#
page.SaveState()                                    // q
    .SetTransform(PdfMatrix.Rotate(-90))            // cm
    .SetClip(corner)                                // W n
    .DrawText("DRAFT", 20, 20, typeface, 48)        // BT ... ET
    .RestoreState();                                // Q
```

Two refusals keep the stream balanced. `RestoreState()` on a canvas with no matching save throws `InvalidOperationException` where you call it, so the mistake is reported at its own line rather than as a corrupt page. A page still holding a saved state when the document is written is refused the same way: `ToArray()`, `SaveAsync()` and `Build()` throw rather than emit a stream that leaves a clip or a transform behind.

What it deliberately does not do

This is a drawing surface, not a vector graphics library with a document model:

- **No scene graph and no retained shapes.** There is no object to move after it has been drawn, no z-order beyond paint order and no hit testing. Coordinates are the model.
- **No gradients, patterns, shadings or spot colours** through the canvas. `PdfPaint` carries a device colour — grey, RGB or CMYK — and one alpha. A gradient is not expressible, and neither is a tiling pattern.
- **No blend modes.** `PdfBlendMode` exists in the model behind the content writer, but the canvas exposes opacity only: a shape is composited normally, with alpha.
- **No stroke-to-outline conversion, no text on a path, no clipping to text.** Text clipping is a reader feature; this surface writes.
- **No SVG, and no image format beyond the PNG, JPEG, BMP and GIF of Chapter 12.** These are the formats the authoring side reads; anything else is a file the caller re-saves.
- **No validation against PDF/A or PDF/UA.** What is written is plain content-stream operators in an authored page; Chapter 24 covers the archival output that is a separate path.

Everything is emitted in call order, so painting over what came before is how you stack: an opaque rectangle over a photograph hides it, and a translucent one tints it.

The surface at a glance

Member	What it does
`DrawLine(from, to, stroke)`	One stroked segment; the cap applies at both ends.
`DrawRectangle(rect, fill?, stroke?)`	A closed box, filled, stroked, or both in one operator.

Member	What it does
`DrawEllipse(bounds, fill?, stroke?)`	An ellipse fitted to a box as four cubic Béziers.
`DrawPath(path, fill?, stroke?)`	Any path of move, line, cubic, quadratic and close segments.
`SetClip(path, fillRule?)`	Restricts everything afterwards to the inside of a path.
`SetFillColor(name, components)` / `SetStrokeColor(name, components)`	Selects a colour space by name (`cs` / `CS`) and sets its colour (`scn` / `SCN`).
`FillPath(path)` / `StrokePath(path)`	Paints a path with the colour the selection stated, writing no colour of its own.
`SetTransform(matrix)`	Concatenates a canvas-space transform; affects paths, text and images.
`SaveState()` / `RestoreState()`	Brackets any change of paint, stroke, clip or transform.
`PdfPaint(color, alpha?, fillRule?, colorSpace?)`	A device colour, its opacity, the fill rule and the space.
`PdfStroke(paint, width?, cap?, join?, miterLimit?, dashArray?, dashPhase?, strokeAdjustment?)`	A paint plus the line style written with it.
`PdfPath(segments, isClosed?)` / `PdfPathSegment.*`	The path model: `MoveTo` , `LineTo` , `CubicTo` , `QuadraticTo` , `Close` .
`PdfRectangle(left, bottom, right, top)` / `FromCorners(...)`	A box in page space, normalised on the way in.
`PdfPoint(x, y)` / `PdfMatrix.Translate/Scale/Rotate`	The two other values shapes are built from.

Where to go next

- To start a page, draw text and place images, read Chapter 12.
- To measure a line, wrap a paragraph and draw the result, read Chapter 16.
- For complete methods that combine shapes with a merge — a stamped header, a watermark panel, a caption under an image — read the cookbook, Chapter 13.

16. Text Layout

> **Availability.** This chapter documents the text-layout surface `1.2` adds: a typeface that measures, a layouter that turns a paragraph into lines, and a canvas call that draws the result. `1.1.0` drew one line per call and measured nothing, so this is new API rather than a change to the call Chapter 12 describes — which is why the surface has a chapter of its own instead of a second section there.

Chapter 12 draws one line at a baseline at a time. Everything above that — how wide a line is, where the lines of a paragraph break, and where each line starts inside a column — is this chapter. Three types do the work: `PdfTypeface`, which measures; `PdfTextLayouter`, which wraps; and `PdfPageCanvas.DrawTextLayout`, which draws.

Measuring a line

`PdfTypeface.Measure` returns the advance width of a string in points, using the same bytes the page will be written with: the AFM widths of a standard font, or the `hmtx` table of the embedded program.

```C#
var heading = PdfTypeface.Standard(StandardFont.Helvetica);

var width = heading.Measure("Décomptes SPRL", 12);        // 12 point text
var x = (page.Width - width) / 2d;                        // centre it on the page
page.DrawText("Décomptes SPRL", x, 180, heading, 12);
```

The overload with two extra arguments measures the line as it will be drawn once character spacing and word spacing are set, because those are PDF text state the writer applies and the face cannot guess:

```C#
var spaced = heading.Measure("Ref: 12345", 12, characterSpacing: 0.4, wordSpacing: 1.2);
```

That number is *before* horizontal scaling. `PdfTextStyle.HorizontalScale` is a percentage the writer sets in the content stream, so a caller that scales text has to multiply the measurement by it — the layouter does exactly that and you do not have to.

Two refusals are worth knowing, because they happen where the string is measured rather than where it is drawn:

- A **standard font** is written with `/WinAnsiEncoding`, so a character outside Windows-1252 — an arrow, a CJK ideograph, a ligature — throws `ArgumentException` naming the character and its code point. A width is not guessed for a glyph the font cannot show.
- An **embedded font** throws the same exception for a code point its `cmap` does not map.

A size that is not a positive finite number is an `ArgumentOutOfRangeException` in both cases. Nothing is read from the operating system: the same string measures the same on every machine and both target frameworks, which is what lets a document be rebuilt byte for byte.

Metrics, and the line height they imply

`PdfTypeface.Metrics` returns a `PdfTextMetrics` in the face's own **font units**, not in points:

Member	What it states
`UnitsPerEm`	Font units in one em: 1000 for a standard font, the program's own value otherwise (2048 for DejaVu Sans).
`Ascent`	Distance from the baseline to the top of the ascent; positive.
`Descent`	Distance from the baseline to the bottom of the descent; normally negative.

Member	What it states
`LineGap`	The extra space the font recommends between lines.
`CapHeight`	Height of capital letters; from `OS/2`, or from the outline of `H` when the table says nothing.
`XHeight`	Height of lowercase letters; from `OS/2`, or from the outline of `x`.

`PdfTextMetrics.LineHeight(size)` converts them to points — `(Ascent - Descent + LineGap) * size / UnitsPerEm` — which is the default distance between baselines:

```C#
var metrics = PdfTypeface.Standard(StandardFont.Helvetica).Metrics;

metrics.UnitsPerEm;             // 1000
metrics.Ascent;                 // 718
metrics.Descent;                // -207
metrics.LineHeight(12);         // 11.1 points
```

That 11.1 is a *tight* line height: it is what the font states, not what a page designer would choose. A statement set at 12 points with 15 points between baselines states `leading: 15` on the style, and this is where that number belongs.

Laying a paragraph out in a box

`PdfTextLayouter.Layout` takes the text, the style it will be drawn with, and a `PdfTextLayoutOptions` that describes the column. The result is a `PdfTextLayout`: the lines, each line's start point and runs, and the box they occupy.

```C#
var style = new PdfTextStyle(PdfTypeface.Standard(StandardFont.Helvetica), 12, leading: 15);

var layout = PdfTextLayouter.Layout(
    "Veuillez trouver ci-joint le décompte pour l'immeuble 12345. "
    + "Nous restons à votre disposition pour toute question.",
    style,
    new PdfTextLayoutOptions(maxWidth: 240, alignment: PdfTextAlignment.Justify));

PdfDocumentBuilder.Create()
    .AddPage(PdfPageSize.A4, page => page.DrawTextLayout(layout, new PdfPoint(72, 400)))
    .Build("statement.pdf");
```

The short overload, `Layout(text, style, maxWidth)`, is the same call with the defaults below: left aligned, refusing what does not fit. `text` may contain line breaks and any number of paragraphs.

Option	Default	What it decides
`MaxWidth`	required	The width of the column, in points. Must be finite and greater than zero.
`Alignment`	`Left`	Where a line sits inside the column: `Left`, `Center`, `Right` or `Justify`.
`LineHeight`	`null`	The distance between baselines. `null` falls back to the style's `Leading`, then to `Metrics.LineHeight(FontSize)`.
`BreakWords`	`false`	Whether a word wider than the column may be split into pieces that fit.
`Overflow`	`Refuse`	What happens to a word that does not fit: `Refuse`, `Clip` or `Continue`.
`Direction`	`LeftToRight`	The reading direction. `RightToLeft` is refused today — see below.

An undefined enum value, a width that is not a width or a line height that is not a line height is an `ArgumentOutOfRangeException` from the options constructor, before any text is looked at.

What wrapping does

The rules are few, and they are deterministic:

- A **line break** (`\n` , `\r\n` or `\r`) starts a new line and is never wrapped away. An empty paragraph stays a line, so a blank line in the input is a blank line in the output.
- Lines are built out of **words**, and words are separated by spaces. A run of spaces between two words collapses to the single space the line is rebuilt from; a tab, a non-breaking space and a hyphen are part of the word, so none of them is a place where a line may break.
- Wrapping is **greedy**: a word is added to the current line while it and the space before it fit `MaxWidth` , and the next word starts a new line. No line is balanced against the next one, and nothing tries to avoid a short last line.
- The width a word is measured at is the width it will be drawn at: the style's character spacing, word spacing and horizontal scale are included.

Alignment

`Left` , `Center` and `Right` place each line inside the column, measured from its left edge. A line is never moved outside the column and never broken again because of its alignment.

`Justify` stretches the **gaps between words** so that both edges of the column are used. It applies to a line that is not the last line of its paragraph and has at least two words; the last line of a paragraph is placed left, because stretching it would add space where the reader expects none. The words themselves are never stretched and no gap is narrowed, so a line whose natural width already fills the column is left as it is.

Overflow

One word can be wider than the column — a URL, a long identifier, a number in a narrow field. Four policies decide what happens, in this order:

Setting	What happens to the word
`Overflow = Refuse` (default)	`ArgumentException` naming the word, its width and the column width. Nothing is drawn and nothing is guessed.
`BreakWords = true`	The word is split by character into the longest pieces that fit. Each piece but the last takes a line of its own; the last shares a line with whatever follows.
`Overflow = Clip`	The word is cut at the column edge: the longest leading part that fits is kept and the rest of the word is dropped.
`Overflow = Continue`	The whole word is placed, and the line is allowed to be wider than `MaxWidth` .

A single **character** wider than the column follows the same policy: it is refused, dropped or placed alone on its line. `Clip` and `Continue` are the caller's stated choice, so neither reports a warning — if dropping characters silently is not acceptable, leave the policy at `Refuse` and catch the exception.

Direction

`PdfTextDirection.RightToLeft` throws `PdfUnsupportedFeatureException` with the feature `right-to-left text layout` . The engine writes glyphs in the order the string states them and does not reorder or shape them, so a right-to-left paragraph would be drawn backwards; refusing it names the gap instead of producing a document that looks wrong. Text in a right-to-left script can still be drawn with `DrawText` when the string is already in visual order.

Drawing the layout

`DrawTextLayout(layout, origin)` draws every line of the layout at the point you give it:

```C#
page.DrawTextLayout(layout, new PdfPoint(72, 400));
```

`origin` is the **top-left corner of the box**, in the page coordinates Chapter 12 describes. The first baseline is one ascent below that corner — 8.6 points for 12 point Helvetica — and each later baseline is a line height below the one before. A block asked for at 400 therefore has its first baseline at about 408.6, and its capitals start at about 400, where the box starts.

The engine writes each **run** as one text operation, with the style's character spacing, word spacing, horizontal scale, rise and colour in the text state, so the characters can be extracted again by `IPdfContentReader`, `pypdf` or `pdftotext`. A paragraph that was justified is drawn as one operation per word-plus-gap, because that is what a stretched line is; a plain line is one operation, and an empty paragraph draws nothing at all.

`Bounds` is a description of the box, not a second coordinate space: the call adds `origin` to each run's start point and takes no offset from `Bounds`. A layout whose `Bounds.Height` is 71 points is a layout that is 71 points tall — drawing it does not move it up by that much.

The layout result

`PdfTextLayout` is immutable and can be drawn more than once, on more than one page, and measured before it is drawn.

Member	What it holds
`Style`	The `PdfTextStyle` the lines were measured with; `DrawTextLayout` draws with it.
`Lines`	The `PdfTextLine` records, in reading order.
`Bounds`	`PdfRectangle (0, 0, MaxWidth, height)`, where the height runs from the top of the box to the last line's descender.
`Baseline`	The y coordinate of the first line's baseline — what `DrawText` would need to draw the first line itself; zero for a layout constructed with no lines.
`LineHeight`	The distance between baselines that was used, after the option/leading/font fallback.
`LineCount`	The number of lines, blank lines included.

Each `PdfTextLine` states `Text`, `Runs`, `Start` (`PdfPoint`), `Width` and `Baseline`. Each `PdfTextRun` states `Text`, `Start` and `Width`. A run's `Start` is relative to the box, so `Start.X` is the indent of that word inside the column on a justified line and `0` (or the alignment offset) elsewhere.

Both records have public constructors, so a layout can also be assembled by hand — a table cell, a caption placed at a computed offset — and drawn with the same call:

```C#
var amount = "42.00";
var width = heading.Measure(amount, 12);
var line = new PdfTextLine(amount, new PdfPoint(40 - width, 8.6), width, 8.6);
var cell = new PdfTextLayout(style, [line], new PdfRectangle(0, 0, 40, 11.1), 11.1);

page.DrawTextLayout(cell, new PdfPoint(page.Width - 60, 700));   // the line ends at x = 40 in the box
```

A layout of an empty string is one empty line: `LineCount` is 1, `Lines[0].Text` is `""` and nothing is inked. That keeps "the text was empty" and "the layout was never built" distinguishable.

Determinism

Laying text out is a pure function of the string, the style and the options. Widths come from the AFM data of a standard font or the tables of the embedded program, never from the operating system, the culture or a font-fallback chain, so the same call produces the same lines on Windows, Linux and both target frameworks. The layouter is static and takes no shared state; a `PdfTextLayout` is immutable, so one may be cached and drawn on many pages.

Combined with a fixed `CreationDate` and `DocumentId` (Chapter 12), that makes a laid-out page reproducible byte for byte — which is what a test that compares two builds, or a document that must be reissued identically, needs.

A character the face cannot encode is refused when the text is *measured*, so a layout cannot succeed and then fail at `Build()`, and a document is never written with a character missing from a line.

What it deliberately does not do

This is a text layout, not a typesetting engine:

- **No hyphenation and no dictionary.** A line breaks at spaces only; a word that does not fit is refused, broken by character, clipped or continued, as the options say.
- **No shaping and no bidirectional algorithm.** Ligatures and kerning come from the font program; Arabic, Hebrew and Indic reordering do not happen, and a right-to-left layout is refused rather than drawn backwards.
- **One style per layout.** A bold word inside a sentence needs two layouts, or one layout per run of style, placed by hand. A line's `Runs` exist to place and measure words — notably a justified line — not to carry per-word styling.
- **No flow.** There are no tables, columns, page breaks, footnotes, floats or text wrapped around an image. A block that does not fit a page is a block that runs off it; the caller decides where the next block starts.
- **No vertical text and no rotation.** `DrawTextLayout` writes horizontal text; a rotated block needs the page canvas transform, which is a drawing concern.
- **No last-line justification** and no shrinking of a gap: justification only ever adds space between words.
- **No overflow warning.** `Clip` and `Continue` are policies the caller states, so they are not reported as warnings; the refusal is the exception.

The surface at a glance

Member	What it does
`PdfTypeface.Metrics`	The face's vertical metrics in font units, from its AFM data or its own tables.
`PdfTypeface.Measure(text, size)`	The advance width of a string in points.
`PdfTypeface.Measure(text, size, characterSpacing, wordSpacing)`	The width as it will be drawn with that text state.
`PdfTextMetrics.LineHeight(size)`	Ascent, descent and line gap at a size, in points.
`PdfTextLayouter.Layout(text, style, maxWidth)`	Lays text out in a width, left aligned, refusing what does not fit.
`PdfTextLayouter.Layout(text, style, options)`	The same, with alignment, line height, overflow and direction stated.
`PdfTextLayout.Lines` / `Bounds` / `Baseline` / `LineCount` / `LineHeight`	The result: lines, box, first baseline and the leading that was used.
`PdfTextLine.Text` / `Start` / `Width` / `Baseline` / `Runs`	One line, in the layout's own space.
`PdfTextRun.Text` / `Start` / `Width`	One contiguous piece of a line, at its own start point.
`PdfPageCanvas.DrawTextLayout(layout, origin)`	Draws every run at `origin` plus its own start point.
`new PdfTextLayoutOptions(maxWidth, alignment, lineHeight, breakWords, overflow, direction)`	Describes the column and what to do when the text does not fit it.

17. Image Formats

> **Availability.** This chapter documents the image surface `1.2` adds: the JPEG pass-through, the BMP and GIF readers, and the colour profile a PNG may state. `1.1.0` embedded PNG images only — Chapter 12 describes the authoring surface as it stands today — so every reader here is new API, and the format a caller draws is now a choice rather than a restriction.

A page shows a picture by naming a file, and everything after that — the bytes the document stores, the colour space it declares, the mask that makes part of it transparent — is decided by the format of that file. This chapter is the reference for those decisions: what SharpCorePDF reads out of a PNG, a JPEG, a BMP and a GIF, what it refuses by name, and why a logo drawn on forty pages is stored once.

Chapter 12 covers the drawing surface a page offers and how an image is placed in it; this chapter is about the file behind the call.

One call, four formats

```C#
var logo = PdfImage.Open(File.ReadAllBytes("logo.png"));   // the format comes from the signature

page.DrawImage(logo, x: 40, y: 30, width: 152, height: 87); // an explicit size in points
page.DrawImage(logo, x: 40, y: 30);                          // the size the file states

page.DrawImage(pngBytes, x: 40, y: 30, width: 152, height: 87); // the same, from bytes
```

`PdfImage.Open` reads the first bytes and never the file name: a `.png` whose content is a JPEG is read as a JPEG, and a file whose content is neither is refused with those first bytes in the message. `FromPng`, `FromJpeg`, `FromBmp` and `FromGif` do the same read with the format stated, so a caller that can vouch for a file gets a refusal when the file disagrees instead of a picture read as something it is not.

An opened image reports what the file holds:

Member	What it states
`Format`	`Png`, `Jpeg`, `Bmp` or `Gif`, as read from the signature.
`WidthPixels`, `HeightPixels`	The pixel size of the samples.
`ColorSpace`	The colour space of the samples: `Gray`, `Rgb`, `Cmyk`, `Indexed` or `IccBased`.
`BitsPerComponent`	Eight for every format this release reads.
`HasAlpha`	Whether the file states transparency, as a soft mask or a colour key.
`NaturalWidth`, `NaturalHeight`	The size in points the file's resolution metadata implies.
`FrameCount`	The number of frames: one for every format but GIF.
`Key`	A stable identifier of the prepared bytes, used to store one image object per picture.

Opening the same bytes twice produces the same `Key`, so the document carries one image object however many pages and however many calls draw it. The key covers everything that changes what a reader shows — the samples or the encoded bytes, the filter, the colour space, the mask, the decoding and any colour profile — so two pictures that differ in a way a reader can see are never folded together.

PNG

A PNG is read at eight bits per sample, without interlacing, of colour type 0 (greyscale), 2 (RGB), 3 (palette), 4 (greyscale and alpha) or 6 (RGBA). Sixteen bit samples and Adam7 interlacing are refused with a message that says which one the file uses

and what to do about it, because decoding them halfway would show a picture nobody asked for.

Transparency arrives in three shapes, and each becomes the structure PDF defines for it:

- an alpha channel of colour type 4 or 6 is split out of the samples into a `/SMask` stream;
- a palette whose `tRNS` chunk makes entries transparent is expanded to RGB, because PDF transparency is per sample and not per palette entry, and the alpha becomes a `/SMask` as well;
- a `tRNS` colour key on a greyscale or RGB image becomes the `/Mask` array — a minimum and a maximum for each component, which is how PDF states "this colour is transparent" — and the samples are stored exactly as the file holds them.

Compressed data is kept when it does not have to be decoded. An image without transparency keeps its `IDAT` bytes and is embedded with `FlateDecode` and a PNG predictor, so a 67 KB logo is a 67 KB stream in the document and nothing is inflated and deflated again. Only an image whose samples have to be split — an alpha channel, an expanded palette — is decoded and re-compressed.

The `pHYs` chunk states the resolution: with the metre unit it becomes the natural size in points, so a PNG at 11811 pixels per metre draws 300 dots to the inch.

Colour profiles in a PNG

A PNG may state the colour space its samples are measured in, in an `iCCP` chunk. That profile is declared rather than dropped: it is written as its own Flate-compressed stream, the image's colour space becomes `[/ICCBased …]`, and `/N` states the number of components the profile describes.

```text
13 0 obj   % the image
<< /Type /XObject /Subtype /Image /Width 152 /Height 87
   /ColorSpace [/ICCBased 14 0 R] /BitsPerComponent 8 /Filter /FlateDecode … >>
14 0 obj   % the profile, exactly as the file carried it
<< /N 3 /Alternate /DeviceRGB /Filter /FlateDecode >>
```

Leaving such a picture as device values would show it through whatever profile the viewer happens to use, which is a colour shift no one is told about — the silent substitution this library refuses. The alternate space is stated too, so a reader that cannot use the profile still knows what the samples are meant to be, and `PdfImage.ColorSpace` reports `IccBased` so a caller can see the picture is managed.

A profile that does not describe the samples is refused by name rather than written: a `GRAY` profile on an RGB image, or a `CMYK` profile in a PNG, would describe the picture wrongly. A `sRGB` chunk needs no work, because an unlabelled PNG is already read as sRGB — the same values, stated twice. `gAMA` and `cHRM` chunks are not used: they describe a space the samples would have to be converted from, and this release does not convert.

JPEG

A JPEG is passed through. The file's own bytes become the image stream, decoded by `DCTDecode`, so no entropy-coded data is walked, decoded or re-encoded, and nothing is lost between the file and the page.

What the file states	What the document holds
One component	`/ColorSpace /DeviceGray`
Three components	`/ColorSpace /DeviceRGB`
Four components with an Adobe `APP14` transform	`/ColorSpace /DeviceCMYK` and `/Decode [1 0 1 0 1 0 1 0]`

The `/Decode` array is there because Adobe wrote those samples inverted; without it a CMYK photograph would come out as its negative.

`SOF0` (baseline), `SOF1` (extended sequential) and `SOF2` (progressive) frames are read; arithmetic coding, lossless, differential and hierarchical frames are refused by name, as is a sample precision other than eight bits. An EXIF orientation

other than the identity is refused too: honouring it means rotating or mirroring the samples and re-encoding them, which is a different operation from placing a picture. A four-component file with no Adobe transform is refused, because whether the samples are inverted cannot be known; five or more components are outside what PDF's device colour spaces can name.

The JFIF density states the natural size, in dots per inch or per centimetre.

BMP

A BMP is a pixel format PDF has no filter for, so it is decoded: eight bit samples through a palette, or twenty-four or thirty-two bit samples, with `BITMAPINFOHEADER` or a later, longer header, and with either row order. The palette is expanded to RGB, the rows are written top row first, and the samples are compressed with `FlateDecode` and a PNG predictor.

The fourth byte of a thirty-two bit pixel is padding, not transparency: a BMP cannot state an alpha channel this reader would honour, and inventing one would make an opaque picture partly invisible.

`BI_RLE8`, `BI_RLE4`, `BI_BITFIELDS`, the headers that embed another image, one, four and sixteen bit samples, and the OS/2 core header are refused with the name of what was found. The pixels-per-metre fields state the natural size.

GIF

A GIF is read frame by frame, of which the first is drawn — PDF has no animation. The LZW codes are decoded into palette indices, a global or a local palette is expanded to RGB, an interlaced frame is put back into row order, and a graphic control extension that names a transparent palette entry becomes a `/SMask`.

`FrameCount` states how many frames the file holds; the others are counted, not decoded. That count is how a caller learns that a picture it drew is an animation the document cannot carry — the authoring side has no warning channel, unlike the read side, where the same fact arrives as a `PdfWarning`.

A GIF states no resolution, so it is drawn at one point per pixel unless a size is given.

Natural size, placement and stretching

Without a size, an image is drawn at the size its own metadata implies: `pHYs` for a PNG, the JFIF density for a JPEG, the pixels-per-metre fields for a BMP, and one point per pixel for everything else. A logo of 152 × 87 pixels at 300 dots to the inch covers 36.48 × 20.88 points.

With a width and a height, the image fills that box. The scale is applied to each axis separately, so a call that gives a box of another proportion stretches the picture; nothing crops it and no part of it is placed outside the box. The top-left corner sits at `(x, y)` in canvas coordinates, and a transform set on the canvas moves the image like it moves a path — see Chapter 15.

What a draw costs

The first draw of a file parses it and copies what the document will hold; a later draw of the same bytes parses them again and finds the same key, so the document still carries one object. A caller that draws one picture many times should open it once and draw the `PdfImage`, which parses it once.

For a PNG and a JPEG the bytes the document stores are compressed already, so a draw costs a copy and a hash. For a BMP and a GIF the samples are decoded, filtered and compressed, which is where the cost of those two formats comes from.

Refusals

Every refusal happens at the call, before anything is written, and names what was found:

Refusal	Exception	The message names
The signature is none of the four	`ArgumentException`	The first eight bytes, as hexadecimal

Refusal	Exception	The message names
A PNG is interlaced or sixteen bit	`NotSupportedException`	Adam7, or the bit depth
A PNG states another colour type	`NotSupportedException`	The colour type and the supported ones
A PNG profile does not describe the samples	`NotSupportedException`	The profile's colour space and the components written
An `iCCP` chunk holds no profile	`ArgumentException`	The missing `acsp` signature
A JPEG uses an unsupported frame or precision	`PdfUnsupportedFeatureException`	The frame marker or the precision
A JPEG states an EXIF orientation	`PdfUnsupportedFeatureException`	The orientation
A BMP uses a compression this reader does not decode	`NotSupportedException`	`BI_RLE8`, `BI_RLE4`, `BI_BITFIELDS` or the header

A refused call leaves the canvas unchanged: no half-drawn picture, and no image registered for a file that was not read.

What this chapter does not do

- No SVG, TIFF, WebP, JPEG 2000 or animated GIF: a caller with another format re-saves it as one of the four this chapter describes.
- No EXIF rotation: a photograph that expects its orientation to be applied is refused, not turned.
- No colour conversion: a CMYK JPEG stays CMYK, and a profiled PNG keeps its profile instead of being converted to sRGB. A reader shows both correctly; only a converter would change the samples.
- No profile validation: an `iCCP` chunk is checked for its signature and its component count, and what is inside the profile is the profile's business.
- No resampling on the page: the image is drawn into a box, and how fine it looks is the viewer's business — an image drawn larger than its pixels is soft, and nothing here guesses an interpolation.
- No pixel access on this side: `PdfImage` describes the file it read and draws it, and the samples a *read* hands back are `PdfImageInfo.DecodedPixels` in `Gray8`, `Rgb24` or `Cmyk32` — Chapter 8. A JPEG has no pixels there either, for the same reason it is passed through here: it is already a format the caller can decode.
- No PDF/A image rules here: Chapter 24 covers the profiles, which refuse an image they cannot describe.

18. Annotations

An annotation is something a viewer draws **over** a page: a highlight, a sticky note, a stamp, a link. It is not page content. That distinction is the whole design of this chapter, and it is why adding a comment to a document does not change a byte of the page's content stream — only the page's `/Annots` array, which is the list of annotations the page carries.

Annotations are written through the same immutable plan as every other edit (chapter 14), so a plan that names a kind this library cannot draw, a link with both a page and a URL, or a rectangle outside the page is refused before anything is written.

Release: the annotation writer shipped in `1.1.0`, so everything this chapter describes is in that package and in every one since.

Adding one

```C#
var plan = PdfEditPlan.Create()
    .AddAnnotation(1, PdfAnnotations.Highlight(
        [new PdfRectangle(72, 700, 320, 716)],
        PdfColor.FromHex("#FFF176")) with { Name = "summary-highlight" })
    .AddAnnotation(1, PdfAnnotations.FreeText(
        new PdfRectangle(72, 640, 340, 672),
        "Reviewed against the checklist.",
        new PdfStandardTextStyle(StandardFont.HelveticaOblique, 11)))
    .AddAnnotation(1, PdfAnnotations.Link(
        new PdfRectangle(72, 520, 300, 540),
        "https://github.com/MPCoreDeveloper/SharpCorePDF"));
```

The factories cover the kinds a document realistically carries:

Factory	Kind	What it draws
`PdfAnnotations.Text`	`Text`	A sticky note, folded corner and all
`PdfAnnotations.FreeText`	`FreeText`	The text itself, wrapped and clipped to the box
`PdfAnnotations.Stamp`	`Stamp`	A frame and a caption
`PdfAnnotations.Highlight`	`Highlight`	The quads, filled with the colour; set `Opacity` to make it translucent
`PdfAnnotations.Underline` / `StrikeOut`	`Underline`, `StrikeOut`	A rule under, or through, each quad
`PdfAnnotations.Square` / `Circle`	`Square`, `Circle`	A border, or an ellipse inscribed in the box
`PdfAnnotations.Line`	`Line`	One stroked segment, `LineFrom` to `LineTo`
`PdfAnnotations.Ink`	`Ink`	Every stroke as a polyline
`PdfAnnotations.Link`	`Link`	Nothing: a link is an active area

`PdfAnnotation` is a record, so everything the factories do not state is set with a `with` expression: `Contents`, `Color`, `Opacity`, `Border`, `Flags`, `TextStyle`, `IconName`. Setting a value the kind cannot draw is refused by `Validate`, which the plan calls before the source is opened.

Appearance streams

Every annotation the editor creates gets an appearance stream — the small form XObject a viewer draws instead of synthesising the mark. That is what makes a highlight visible to a printer, which never synthesises anything, and to a viewer that does not either.

The stream is written in the annotation's own space: the lower-left corner of the rectangle is the origin, so the operators are small numbers that do not depend on where on the page the annotation sits.

A caller that wants a different face supplies one:

```C#
var stamp = PdfAnnotations.Stamp(new PdfRectangle(420, 560, 560, 610), "DRAFT")
    with
    {
        Appearance = new PdfAnnotationAppearance(
            Content: "q 1 0.9 0.2 rg 0 0 140 50 re f Q",
            BoundingBox: new PdfRectangle(0, 0, 140, 50)),
    };
```

`PdfAnnotationAppearance` carries the operators, the box they are drawn in, and the font they select as `/Helv`. Only an operator stream is accepted: an appearance never carries a font program, because the editor's fonts are the twelve standard ones.

A link has no appearance at all. A link is an active area with nothing to draw, and a viewer that drew one would paint over the page.

Updating and removing

```C#
var plan = PdfEditPlan.Create()
    .UpdateAnnotation(1, "summary-highlight", PdfAnnotations.Highlight(
        [new PdfRectangle(72, 700, 320, 716)],
        PdfColor.FromHex("#FFE082")) with { Contents = "Reviewed" })
    .RemoveAnnotation(1, "old-note");
```

Both refer to the identifier the annotation was written with, which is its `/NM` entry. An annotation that states no name of its own is written as `scpdf-N`, where `N` is the index of the operation, so a plan can always name what it added. An update or a removal that names an annotation the page does not carry is refused, and the refusal happens before a byte is written.

The page's array is rebuilt rather than mutated: a page that shares an annotation with another page keeps it there, and an annotation the plan removes simply stops being reached — and therefore stops being written at all.

What the editor reports

Warning	When
`AnnotationClipped`	The rectangle reached outside the page and was clipped, or lay entirely outside and was not written
`AnnotationAppearanceRegenerated`	The page carries an appearance stream the engine drew, so a viewer that does not synthesise one still shows the marks
`AnnotationDestinationUnresolved`	A link points at a page the same edit removed, so the link was not written
`AnnotationUnreadable`	A source annotation could not be resolved while reading

Two behaviours are worth stating outright, because they are choices rather than consequences:

- **A link to a removed page is dropped, not redirected.** Silently pointing it at a different page would make the document lie about where it goes.
- **The geometry is clipped to the crop box, not to the media box.** What a reader sees is what the annotation is kept inside of.

Where it is tested

`tests/SharpCorePDF.Tests/PdfAnnotationTests.cs` covers the factories, the flags and the refusals; `PdfDocumentEditorTests.Annotations.cs` writes marks through the plan and reads them back with

`ReadAnnotationsAsync`. The `annotate` example produces `annotated.pdf`, which `docs/tools/validate-pdf.py` reports as `qpdf check: no problems`.

19. Forms

An AcroForm is a set of named fields that a viewer makes editable. This chapter writes them: creating the fields, filling the ones a document already carries, putting them back to their default values, and flattening them so the page shows the values to a reader that knows nothing about forms.

All of it goes through the same immutable plan as the rest of the editor (chapter 14), and all of it is AcroForm: XFA is not a format this library writes, and a document that carries one loses it with a warning the moment an edit writes a field (see *XFA* below).

Release: the field model shipped in `1.1.0`; the writer this chapter documents — filling, reset and flatten — is new public API in `1.2`, not a behaviour change to an existing one.

The model

A field is a record with a name, a kind and a rectangle, and the properties that kind can honour:

```C#
var field = PdfFormFields.Text(1, "person.name", new PdfRectangle(72, 700, 320, 724), "Ada Lovelace")
    with { IsRequired = true, MaxLength = 64 };
```

The page comes first, because a field is placed before it is described — the same order every other page operation uses.

Factory	Kind	What a caller can put in it
`PdfFormFields.Text`	`Text`	A value; `IsMultiline`, `IsPassword`, `MaxLength`, `Alignment`
`PdfFormFields.CheckBox`	`CheckBox`	`IsChecked`, and the `OnState` name the value is exported under
`PdfFormFields.RadioButton`	`RadioButton`	The same, drawn as a dot in a circle
`PdfFormFields.Choice`	`Choice`	`Options` and one value from them; `IsCombo` for a combo box, otherwise a list box
`PdfFormFields.PushButton`	`PushButton`	A `Caption` and no value at all

`Flags` is computed, not stated: the bits are derived from the properties above, so a caller cannot ask for a flag the writer does not honour — a JavaScript action, a rich-text value, a comb field. The bits that are written are read-only, required, multiline, password, radio, push button, combo, editable and multi-select.

Two rules are checked before the source is opened, and both are refusals rather than repairs:

- a name is unique within a form, and two operations of one plan cannot write the same name;
- a value the kind cannot hold is refused with the reason — a choice value that is not an option, a button that carries a value, a text field that is both multiline and a password, a value longer than `MaxLength`.

Writing a form

```C#
var plan = PdfEditPlan.Create().SetForm(
    PdfForm.Create(
        PdfFormFields.Text(1, "person.name", new PdfRectangle(72, 700, 320, 724), "Ada Lovelace"),
        PdfFormFields.CheckBox(1, "agree", new PdfRectangle(72, 636, 90, 654), isChecked: true),
        PdfFormFields.Choice(1, "country", new PdfRectangle(72, 600, 260, 624), ["NL", "BE", "DE"], "NL",
isCombo: true),
        PdfFormFields.PushButton(1, "submit", new PdfRectangle(72, 560, 200, 588), "Send"))
    .WithDefaultStyle(new PdfStandardTextStyle(StandardFont.Helvetica, 10)));
```

`PdfForm.Create` takes the fields in the order they are written, and that order is the order a viewer walks them: tab order follows it. `WithDefaultStyle` states the style a field that carries none of its own is drawn in;

`PdfStandardTextStyle.Default` — Helvetica 12 — is the fallback.

A dotted name is written as a tree, which is what a PDF means by `person.name`: one node per segment, each stating its part, with the widget on the last one. That is also why a name is only unique once its ancestors are joined to it.

`SetForm` writes the fields a document does not carry yet and **fills the ones it does**, by name. Nothing else in the form is touched, so ten `SetFormField` calls fill ten fields and leave the rest alone:

```C#
var fill = PdfEditPlan.Create()
    .SetFormField(1, PdfFormFields.Text(1, "person.name", new PdfRectangle(0, 0, 1, 1), "Grace Hopper"));
```

The rectangle of a field the document already carries is ignored: the field is filled where it stands, with the widget and the ancestors the document gave it. The rectangle matters only for a field the plan creates.

Appearances

Every widget the editor writes gets an appearance stream — the form XObject a viewer draws inside the rectangle. That is what makes a filled field visible to a printer, which never synthesises one, and it is why the form the engine writes states `NeedAppearances false`.

- **Text and combo boxes** are drawn in the field's style; a value that does not fit a single line shrinks to fit, and a multiline value wraps at the spaces.
- **A password field is drawn masked**, one `*` per character: the appearance is what a reader shows, and writing the characters into it would undo the reason the field is masked.
- **A list box** draws its options and highlights the selected one.
- **A check box and a radio button** get both states, `Off` and the field's on state, and the widget states the one it is in.
- **A push button** draws its frame and its caption.
- **A field that states an appearance of its own** keeps it: the caller's operators are written, with the box and the font from `PdfAnnotationAppearance`.

The form also carries a `/DR` with every font it names and a `/DA` that selects the first one, which is what a viewer uses if it decides to draw a field itself.

Reset

```C#
var plan = PdfEditPlan.Create().ResetForm();
```

A reset restores each field's `/DV`, the value the document stated when it was designed, and empties the fields that state none. A read-only field is reset too: the reset restores what the document said, and refusing it would leave a filled field that the caller asked to empty. The appearances are regenerated, so what a reader shows and what the field holds cannot disagree.

Flatten

```C#
var plan = PdfEditPlan.Create().FlattenForm();          // every page
var one = PdfEditPlan.Create().FlattenForm(2);          // one page
```

Flattening draws each widget's appearance into the content of the page that carries it, so the value is part of the page: a viewer without form support — and a printer — still shows it. The widgets leave the page's `/Annots` array, the fields leave `/Fields`, and a form whose last widget is gone is removed from the catalog rather than written empty.

The appearance is placed by the widget's rectangle, not by its default appearance: what the reader saw before the flatten is what the page shows after it. Where a widget states no appearance at all, one is generated from the value its field carries — a form in the wild often relies on the viewer for that, and a flatten cannot copy a stream that is not there.

Two widgets deliberately survive a flatten:

- **a field that is drawn nowhere** has no widget to draw, so there is nothing to flatten;
- **a signature field** has no static appearance this writer can draw, so it is left interactive rather than dropped without a mark.

Refusals

Condition	Result
A field the document states as read-only	Refused with the field's name
A signature field	Refused: signing is a separate surface
A name that would sit inside an existing one	Refused with both names
Duplicate field name in a form, or two operations writing one name	Refused by the plan, before the source is opened
A value that is not one of the field's options	Refused with the value and the list
A page the field names does not exist	Refused with the page number

What the engine reports

Warning	When
`FormAppearanceGenerated`	The fields of a page carry an appearance the engine drew
`XfaFormRemoved`	The source carried an XFA packet, which cannot be kept beside fields this editor writes
`FormFieldActionPreserved`	A field carries an action; it is copied as it stands and never executed
`FormFieldUnreadable`	A field of the source could not be resolved while reading, or the tree repeated an object
`FormFieldDepthExceeded`	The field tree is deeper than `PdfEngineOptions.MaxNestingDepth` and was cut off

The field tree is followed to `PdfEngineOptions.MaxNestingDepth` levels — the same limit the page tree and the content reads use. A tree that is cut off contributes `FormFieldDepthExceeded` together with the fields read above the cut, so a form too deep to finish is reported rather than read as a shorter one. The search for signature fields is bounded the same way, which is why the same warning also accompanies the `PdfSignatureProblemKind.FormFieldTreeTooDeep` problem that the signature report carries (chapter 23).

XFA

An XFA form is a form of its own, written as an XML packet beside the fields. A viewer that honours it shows the XFA fields instead of the AcroForm ones, so writing AcroForm fields into a document that carries both would produce a form nobody sees. The packet is removed, and `XfaFormRemoved` says so — a warning rather than a refusal, because the alternative is refusing to fill a document that a lot of tools produce.

Reading the result back

The read side is the same field tree a caller inspects:

```C#
var fields = await reader.ReadFormFieldsAsync(new PdfFileSource("form-filled.pdf"));

foreach (var field in fields.Items)
{
    Console.WriteLine($"{field.Name} ({field.FieldType}) = {field.Value}");
}
```

`PdfFormFieldInfo` states the fully-qualified name, the document's own type token (`Tx`, `Btn`, `Ch`, `Sig`), the value and the default value, the flags, and every widget with the page and rectangle it occupies. A value the document does not state is `null`, never an empty string, so "absent" and "empty" stay distinguishable.

Where it is tested

`tests/SharpCorePDF.Tests/PdfFormTests.cs` covers the model: the factories, the derived flags, the refusals. `PdfFormEditorTests.cs` covers the engine: `Fills_and_reads_back_form_values`, `Fills_a_field_the_document_already_carries`, `Refuses_a_read_only_field`, `Refuses_an_unsupported_field_type`, `Flattens_a_form_without_a_form_viewer`, `Resets_the_form_to_its_default_values` and `Removes_the_xfa_form_and_says_so`.

The `fill-form` example writes `form-created.pdf`, `form-filled.pdf` and `form-flattened.pdf`:

- `docs/tools/validate-pdf.py` reports `qpdf check: no problems` for all three, with page 1 carrying five annotations and a 62-byte content stream before the flatten, and no annotations and 226 bytes after it;
- `pypdf` reads `person.name = Grace Hopper`, `agree = /Off` and `country = BE` from the filled form, and no fields at all from the flattened one;
- PyMuPDF renders the flattened page and reads `Grace Hopper` as **page text**, which is what "visible without a form viewer" means in practice.

What this chapter does not cover

No JavaScript is executed, no calculation or validation script runs, and no action on a field is fired: `/AA` dictionaries are copied as they stand and reported. XFA is not written, a signature field is not filled by this surface (chapter 23 signs), redaction is not offered, and rich-text and comb fields are not modelled because the writer would not honour them.

20. Extraction

Extraction is the read that answers "what does this page say, and where does it say it". It is the same walk of the page tree, the same text state and the same font decoding that the merger uses, so a page you extract is a page you could have merged, and the two cannot disagree about what it draws.

Two ports offer it. `IPdfContentReader.ReadTextAsync` reads blocks of text — the summary read of Chapter 8. `IPdfLayoutReader` reads the same page at three levels — letters, words and blocks — and writes the result out in the four formats a repository ingests. Both take an `IPdfContentSource`, a `PageRange` and an optional password, and both return a `PdfReadResult<T>`. A caller that will ask more than one question about a document opens a session and keeps one parse open for all of them — *One open, several questions* below.

```C#
using SharpCorePDF.Domain.Ports;
using SharpCorePDF.Domain.ValueObjects;

var blocks = await contentReader.ReadTextAsync(
    new PdfFileSource("report.pdf"),
    PageRange.All,
    cancellationToken: cancellationToken);

foreach (var block in blocks.Items)
{
    Console.WriteLine($"p{block.PageNumber} #{block.ReadingOrder}: {block.Text}");
}
```

The shape of a read

`PdfReadResult<T>` is the same wrapper for every read in the library:

Member	Type	Meaning
`Items`	`IReadOnlyList<T>`	What was read, in reading order
`Count`	`int`	`Items.Count`
`Warnings`	`IReadOnlyList<PdfWarning>`	What could not be read, and what had to be assumed
`Segmentation`	`PdfPageSegmentationStrategy?`	The rule that grouped the words — set by the block reads only
`ReadingOrderStrategy`	`PdfReadingOrderStrategy?`	The rule that ordered the blocks — set by the block reads only

The two strategy members are `null` on a read that does not group anything, and they are filled in even when the page turned out to be empty: an empty result still names the rule that found nothing, so a caller comparing two runs never has to recall which options produced which. Chapter 21 explains what the strategies mean and how to choose one.

A read reports a page it could not decode rather than throwing: a content stream that cannot be walked produces a `ContentStreamUnreadable` warning and no blocks for that page, the other pages still read, and a font whose encoding cannot be decoded yields fewer letters rather than invented text (`TextUnreadable`). The guard is the same one Chapter 9 states for merges — treat the presence of a warning as information, and never as a failed read unless the exception itself was thrown.

Letters

A letter is the smallest thing a page draws that carries text. It has its own box, the baseline it sits on, the size it was drawn at and the font it came from:

```C#
var letters = await layoutReader.ReadLettersAsync(
    source,
    PageRange.Single(1),
    cancellationToken: cancellationToken);

foreach (var letter in letters.Items.Take(5))
{
    Console.WriteLine(
        $"{letter.Value,-2} box {letter.BoundingBox} baseline {letter.StartBaseLine} "
        + $"{letter.FontSize:0.0}pt {letter.FontName ?? "(no name)"}");
}
```

Property	Type	Meaning
`Value`	`string`	The character the letter draws
`PageNumber`	`int`	The one-based page it was drawn on
`BoundingBox`	`PdfRectangle`	The area it covers, in page space
`StartBaseLine`, `EndBaseLine`	`PdfPoint`	The baseline it sits on, from its start to its end
`FontSize`	`double`	The size it was drawn at, in points
`FontName`	`string?`	The font's name, when the document states one
`TextDirection`	`PdfTextDirection`	Left to right or right to left
`IsWhitespace`	`bool`	Whether the value is whitespace
`Width`	`double`	The length of the baseline

Whitespace letters are reported. A caller that segments text itself measures the gaps between words, and a read that silently dropped the spaces would force it to measure the gaps between letters instead.

`Width` is measured along the baseline, not from the box: a letter in a rotated run has a box that says nothing about the advance, while its baseline always does.

Words

`ReadWordsAsync` groups letters into words by geometry, with two thresholds stated in ems rather than in points — a fraction of the size of the letters being grouped — so one rule serves a 6pt footnote and a 30pt title:

```C#
var words = await layoutReader.ReadWordsAsync(
    source,
    PageRange.All,
    new PdfWordExtractionOptions
    {
        MaxHorizontalGap = 0.25,
        MaxVerticalGap = 0.5,
        IncludeWhitespace = false,
    },
    cancellationToken: cancellationToken);
```

Option	Default	What it decides
`Strategy`	`NearestNeighbour`	Whether letters are taken in the page's own order or in the order the content stream drew them
`MaxHorizontalGap`	`0.25` em	How wide a gap may be and still sit inside one word; a space is about a quarter of an em
`MaxVerticalGap`	`0.5` em	How far two baselines may drift apart and still be one line; half an em tolerates a superscript
`IncludeWhitespace`	`false`	Whether a space belongs to the word it follows, so concatenating the words reproduces the drawn text

`PdfWord` carries `Text`, `PageNumber`, `BoundingBox`, the `Letters` it was built from, the raw `Glyphs` behind those letters, and three conveniences derived from them: `FontName`, `FontSize` and `BaselineY`. The glyphs are the document's own units — position, width, base font — and the letters are the same run restated with baselines and a direction, which is what a caller measuring lines needs.

A word never spans a line and never spans a page, so the `BoundingBox` of every word in a result is one rectangular area with a single baseline through it.

Blocks, and the exports

A block is a run of words that belong together: a paragraph, a heading, a table cell, a caption. It is what a caller indexes when it wants prose rather than geometry.

Property	Type	Meaning
`PageNumber`	`int`	The one-based page the block sits on
`ReadingOrder`	`int`	The block's position on its page, from 1
`Text`	`string`	The block's text, with a newline between lines
`BoundingBox`	`PdfRectangle`	The area the block covers
`Words`	`IReadOnlyList<PdfWord>`	The words the block was built from
`LineCount`	`int`	The number of lines in `Text`

`ExportAsync` writes that same read out in one of four formats. It does not produce a second reading: the blocks are read once, with the options you pass, and the format decides only the vocabulary they are written in.

```C#
await using var destination = File.Create("report.alto.xml");

var written = await layoutReader.ExportAsync(
    source,
    PageRange.All,
    PdfLayoutFormat.Alto,
    destination,
    cancellationToken: cancellationToken);

Console.WriteLine($"{written.Count} block(s), {written.Warnings.Count} warning(s)");
```

`PdfLayoutFormat`	What it writes
`Alto`	One ALTO document, a `Page` per page with its text blocks, lines, words and print space
`PageXml`	One PAGE XML document, a text region per block, a text line per line and the reading order that names them
`Hocr`	One XHTML document of `ocr_page`, `ocr_carea`, `ocr_par`, `ocr_line` and `ocrx_word` elements, each with its box
`SvgText`	One SVG document, a group per page and a `text` element per word, placed on its baseline

Every format carries the same three things — the text, the area each word covers and the order the page is read in — and differs only in what its consumer already parses. Three properties are worth relying on:

- **Coordinates are flipped for you.** The read reports page space with the origin at the bottom left, as the PDF format stores it. All four formats are written with the origin at the top left and the y axis running down, which is the convention of the authoring surface (Chapter 12). Each export flips the y coordinate against the height of the page; nothing else about a box changes.
- **The bytes are reproducible.** No timestamp, random identifier or machine name is written, so the same document read with the same options produces the same file on every run. That is what makes an export diffable in a test.
- **An export never renders the page.** Nothing produces an image, embeds a font or copies the bytes of the source. ALTO's measurement unit is `pixel` in the sense ALTO gives it — one unit of the page's own coordinate space, a point

— because there is no raster for a physical measurement to refer to. A consumer that needs a length divides by 72 to get inches.

An export writes to a stream you own, so the caller decides the file name, the encoding wrapper and the lifetime. `ExportAsync` validates the format and the destination before it opens the document: a format this release does not write throws `ArgumentOutOfRangeException`, and a destination that cannot be written throws `ArgumentException`, both at the call rather than after a full read.

One open, several questions

Release: `OpenAsync` and `IPdfLayoutSession` arrived in `1.5.0`; the one-shot members this chapter opened with are unchanged, and a caller that asks once keeps using them.

Every member above is a one-shot read: it opens the source, answers, and forgets the parse. A caller that already knows it will ask more than once — the letters of one range, then the words of another, then an export — pays for the cross-reference table and the page tree on every call. `OpenAsync` returns a session that keeps one parse open for all of them:

```C#
await using var session = await layoutReader.OpenAsync(
    new PdfFileSource("report.pdf"),
    cancellationToken: cancellationToken);

var letters = await session.ReadLettersAsync(PageRange.Single(1), cancellationToken);
var words = await session.ReadWordsAsync(PageRange.All, cancellationToken: cancellationToken);
var blocks = await session.ReadBlocksAsync(
    PageRange.All,
    new PdfLayoutOptions { ReadingOrder = PdfReadingOrderStrategy.RowBands },
    cancellationToken);

await using var alto = File.Create("report.alto.xml");
await session.ExportAsync(PageRange.All, PdfLayoutFormat.Alto, alto, cancellationToken: cancellationToken);
```

Member	What it answers
`SourceName`	The display name of the source the session was opened from; the warnings name the same source
`ReadLettersAsync(range)`	One entry per glyph, in content-stream order — the read of the *Letters* section above
`ReadWordsAsync(range, options?)`	The words, grouped by the options the call states
`ReadBlocksAsync(range, options?)`	The blocks, in the reading order the options select
`ReadHyperlinksAsync(range)`	The links the pages carry
`ExportAsync(range, format, destination, options?)`	The blocks, written as ALTO, PAGE XML, hOCR or SVG text

The options belong to the call rather than to the session, so one session can read the same range twice with two different reading orders — which is what the comparison in Chapter 21 does. A `null` options argument takes the same defaults a one-shot read would, and a range past the last page answers for the pages that exist rather than throwing.

A session keeps the read of its **two most recent ranges**, so asking for the letters and then the words of the same pages walks the content once: the second answer groups glyphs that are already in memory. Two ranges are what it holds, so a third evicts the oldest and a long-lived session cannot grow without end.

Three rules about lifetime, because they are the ones a caller gets wrong:

- **The session owns the parse, not your source.** `OpenAsync` reads the source into the session; the stream is released before the call returns, and disposing the session disposes neither the source nor the file it named. The document itself stays in memory for the life of the session, which is the cost the saved opens are paid against.

- **A disposed session refuses every question** with `ObjectDisposedException`, so a caller that has finished with a document cannot keep reading it by accident. Disposing twice is harmless.
- **A session is not thread-safe.** One caller owns it from open to dispose; open a second session for a second concurrent reader of the same source. The service class is still safe to share, because a one-shot read holds no state between calls.

A session is a cost trade rather than a switch: it is worth opening when the caller asks at least twice, and it is what the competitive harness measures as its warm extraction row (`docs/competitive-scoreboard.md`). The recorded cost of one open is in `docs/measurements.md`.

What an extraction is not

- **It is not OCR.** A page that carries only an image of text has no letters to read and produces no blocks. Nothing here invents text from pixels.
- **It is not a layout engine.** `LineCount` and `BoundingBox` are what the page draws; a table, a column or a heading is the caller's interpretation of the geometry. Chapter 21 is about the groupings the library does offer, and where it stops.
- **It is not a conversion to an editable document.** The four exports describe a page; they do not rebuild one, and re-importing them is out of scope (ADR-001).

The `extract-layout` example runs the whole of this chapter against the sample document: blocks in reading order, the letters of page 1, and all four exports written to disk with their sizes reported.

What an export says about itself

A PAGE XML document states two times its schema requires, `Created` and `LastChange`. They are written at a fixed instant (the epoch) because an export has to stay byte-for-byte reproducible: two reads of one page must compare equal, which is a property the export tests pin. A caller that needs the real timestamps writes them itself; the exporter will not invent the moment it happened to run.

21. Layout Analysis

Chapter 20 read a page and reported what it found. This chapter is about the two decisions that turn the letters of a page into prose a caller can index: **what a block is** and **which block is read first**. They are separate decisions, made by two named strategies, and both travel with the result.

Neither strategy changes what a letter says. The glyph read is the same one every other read of the content uses — same page tree walk, same text state, same font decoding — and the grouping happens above it. A docstring in the engine states the rule exactly: *a strategy never changes the glyph data; it only groups it.*

Why the two decisions are separate

A page can be segmented correctly and still be listed in the wrong order. A two-column page whose left column is drawn after its right one is the ordinary case: the blocks are right, and the sequence a caller reads them in is not. Fixing one variable at a time is also what makes a comparison meaningful — vary the segmentation against a fixed reading order and the difference you see is segmentation.

```C#
var blocks = await layoutReader.ReadBlocksAsync(
    source,
    PageRange.All,
    new PdfLayoutOptions
    {
        Segmentation = PdfPageSegmentationStrategy.RecursiveXYCut,
        ReadingOrder = PdfReadingOrderStrategy.ColumnAware,
    },
    cancellationToken: cancellationToken);

Console.WriteLine($"segmentation: {blocks.Segmentation}");
Console.WriteLine($"order:        {blocks.ReadingOrderStrategy}");
```

`PdfLayoutOptions` validates itself when it is created: a value that is not a strategy this release implements throws `ArgumentOutOfRangeException` at the call site, before a document is opened. A read never pays for a document it cannot segment.

Segmentation

`PdfPageSegmentationStrategy`	What it does	When to choose it
`Raw`	Groups the words in the order the content stream draws them, starting a block wherever the drawing stops following a line down the page	To reproduce a producer exactly — it is the baseline a comparison is made against, not a reading of the page
`NearestNeighbour`	Groups words by their place on the page: lines top to bottom, paragraphs as the runs of lines that follow one another with the leading of a paragraph	A single-column page, or the grouping Chapter 20's text read uses
`RecursiveXYCut`	Splits the page recursively along the whitespace that separates two groups of words — the widest vertical gutter first, which separates columns, then the widest horizontal one — until a region admits no split	The default, and the right first attempt: it finds columns and headings without knowing what a column is
`Docstrum`	Groups lines by the distance between them: lines closer together than the page's own line spacing become one block, and a gutter wider than that spacing keeps two blocks apart	A page of uneven leading, or a two-column page whose columns do not share lines

`RecursiveXYCut` is the default because it separates columns and headings before it looks at line spacing, which is the order a page usually states. `Docstrum` reads a page of irregular leading better than a fixed threshold and a two-column page only when the columns do not share lines — that is the trade, and it is why both exist rather than one.

The words a segmentation groups come from the word read Chapter 20 described, and the segmentation decides which of its strategies is used: `Raw` segmentation groups letters in content order, every other segmentation groups them in the page's order. A caller does not state that twice.

Reading order

`PdfReadingOrderStrategy`	What it does
`Raw`	Reports the blocks in the order the segmentation produced them, which for a raw segmentation is the order the content stream draws them
`ColumnAware`	Reports the columns of a page left to right, and the blocks of a column top to bottom. A heading that spans the columns is read before the columns it spans
`RowBands`	Reports the page as horizontal bands: the blocks that overlap vertically are read left to right before the band below them. It is the order a table is read in

`ColumnAware` is the default because it is the order a reader takes a page in. `Raw` is the one a caller wants when it compares a read against the producer, or when the file's drawing order is itself the thing under investigation. `RowBands` arrived in `1.5.0`; it is a deliberate choice for a document known to be tabular: the same two-column page read by bands would interleave its columns line by line, which is why it is never the default. A table page read with `RowBands` gives its cells row by row — `Item, Quantity, Price` before `Widget, 2, 10.00` — while `ColumnAware` gives a column at a time.

Keeping ink that is not prose

Two flags on `PdfLayoutOptions` describe the parts of a page that draw something without drawing a sentence:

Option	Default	What it decides
`IncludeDecorations`	`true`	Whether a block that carries no letter and no digit is kept — a rule drawn as a run of hyphens, a lone bullet, a separator
`IncludeImages`	`true`	Reserved: image regions are not part of a block, and this release's exports describe text only, so the flag adds nothing today

A caller that indexes prose sets `IncludeDecorations = false` and gets only what a reader reads. A caller that reproduces a page's text as a reader would see it keeps the default, because a separator between two sections is part of what the page says.

`IncludeImages` is stated now and honoured later. It is in the released API so that the meaning of a layout option does not change under a caller when image regions arrive; today it changes no result.

Comparing two readings

The result names the rules that produced it, which is what makes a comparison self-describing. Read a page twice and print what each read says about itself:

```C#
async Task<PdfReadResult<PdfTextBlock>> ReadAsync(PdfPageSegmentationStrategy segmentation) =>
    await layoutReader.ReadBlocksAsync(
        source,
        PageRange.Single(1),
        new PdfLayoutOptions { Segmentation = segmentation },
        cancellationToken: cancellationToken);

var raw = await ReadAsync(PdfPageSegmentationStrategy.Raw);
var cut = await ReadAsync(PdfPageSegmentationStrategy.RecursiveXYCut);

Console.WriteLine($"raw:         {raw.Segmentation}, {raw.Count} block(s)");
Console.WriteLine($"recursive:   {cut.Segmentation}, {cut.Count} block(s)");

// The text of the two reads is the same text. A difference between them is a difference in grouping.
var sameText = raw.Items.Select(block => block.Text.Replace("\n", " ", StringComparison.Ordinal)).SequenceEqual(
    cut.Items.Select(block => block.Text.Replace("\n", " ", StringComparison.Ordinal)));
```

That equality is the property worth testing against a new document: a segmentation may move a word from one block to another, but it may not change, lose or add a word. It is also why `Raw` is kept in the released API — without a baseline there is nothing for the other three strategies to be compared against.

Two further properties hold for every strategy:

- **The read is deterministic.** The same document and the same options produce the same blocks in the same order on every run, on every machine. There is no random seed, no time input and no culture-dependent comparison.
- **The strategy sets are open.** A later release may add a segmentation or an order, so a `switch` over either enum must keep a `default` arm (Chapter 9's rule for warnings applies here too).

Where the analysis stops

The library groups glyphs; it does not interpret a document. What that means in practice:

- **No table model.** A table is a set of blocks that happen to line up. Merging cells, identifying a header row or relating two columns as columns is the caller's work, on the geometry the read returns.
- **No semantic roles.** A block is not marked as a heading, a caption or a footnote. Size and position say something about that; nothing else does.
- **No font or style model beyond the drawing.** `PdfWord.FontName` and `PdfLetter.FontName` name the font the page used. Weight, emphasis and language are not derived.
- **No rendering, so no confidence.** A read that finds no letters on a page says so and returns no blocks; it does not guess from pixels (ADR-001).

The cost of a call is one walk of the pages you selected and one pass over their content streams; the grouping is a pass over glyphs already in memory. A one-shot call also opens the source, so a tool that shows several readings of the same pages opens a session once and asks it several times — one open, and a walk per range the first time that range is asked for, because a session keeps the glyphs of its two most recent ranges (Chapter 20, *One open, several questions*). The grouping itself is recomputed for each question, which is what makes two strategies comparable: only the rule changes, never the glyphs. The recorded cost of a read is in `docs/measurements.md`.

22. The Object Graph

Chapters 20 and 21 read what a page **means**. This chapter reads what the file **is**: one node per object the cross-reference table lists, and one entry per operator a page's content stream states. It is the surface to reach for when a document misbehaves, when a producer writes something unexpected, or when an audit has to show that nothing was hidden.

`IPdfObjectInspector` offers the two reads and writes nothing:

```C#
using SharpCorePDF.Domain.Ports;

var objects = await objectInspector.ReadObjectsAsync(
    new PdfFileSource("suspect.pdf"),
    cancellationToken: cancellationToken);

Console.WriteLine($"{objects.Count} object(s), {objects.Warnings.Count} warning(s)");
```

Reading the graph resolves every object the file lists. That is what makes it the diagnostic surface: a document that merges badly is a document whose graph you can look at.

A node per object

`PdfObjectView` is an immutable record. The shape is deliberately small — it says what an object is, not what it is for:

Property	Type	Meaning
`Kind`	`PdfObjectViewKind`	The type of the object: null, boolean, integer, real, string, name, array, dictionary, stream or reference
`Key`	`string?`	The key this node holds in its parent dictionary, when it has one
`ObjectNumber`	`long?`	The object number, for the roots of the graph
`Generation`	`int`	The generation of that object number
`Scalar`	`object?`	The value of a scalar node
`Reference`	`PdfObjectReference?`	The object number and generation a reference names
`IsResolved`	`bool`	Whether the document actually contains what this node names
`Children`	`IReadOnlyList<PdfObjectView>`	The entries of an array or a dictionary, in document order
`StreamLength`	`long?`	The encoded length of a stream
`Filter`	`string?`	The filters a stream declares

The nodes come back in ascending object number, one per object the file lists. A walk is depth first, and `Key` is what makes a projection readable: `/Root` → `/Pages` → `/Kids` is visible in the result rather than inferred from a position.

References are reported, never followed

This is the one rule that makes the graph safe and finite:

- A reference is a node of kind `Reference` that states the number and generation it names. It is not replaced by the object it points at, and it is not walked into.
- An object that points back at itself — a page whose `/Parent` chain returns to it — therefore produces a graph that terminates, without a visited set and without a depth limit on the graph read.
- A reference to an object the file does not list stays in the graph, with `IsResolved` cleared and one `DanglingReference` warning for that target. The page that pointed at it is still visible where it was named.

An object the file **lists** but cannot open — a broken offset, a stream that will not decode — is also a node, of kind `Null` and unresolved. That is the difference between "there is no object 42" (no node) and "object 42 is there but cannot be read" (an unresolved node): a caller sees a gap in the file rather than a gap in the answer.

```C#
// A reference that the file does not satisfy is where a document usually breaks first.
var dangling = objects.Items
    .SelectMany(Flatten)
    .Where(node => !node.IsResolved)
    .Select(node => node.Reference?.ObjectNumber ?? node.ObjectNumber)
    .Distinct()
    .ToList();
```

`Flatten` is the reader's own four-line depth-first walk — `Children` is the whole traversal API, and there is no visitor, callback or cursor to learn.

Streams without their bytes

A stream node carries `StreamLength` and `Filter`, and never the bytes. Three reasons, all of which a diagnostic tool wants:

- **The read stays a description.** Handing back megabytes of compressed data would make an inspection of a large document a memory event; a length and a filter name is enough to decide whether to look further.
- **The decoder is elsewhere.** `IPdfContentReader` is the port that decodes images, attachments and metadata payloads, under the inline budget of Chapter 8. The graph says *that* a stream exists and how large it is; the content reader says *what* it decodes to.
- **`Filter` is stated, not applied.** `/FlateDecode`, `/DCTDecode` and a chain of them are reported as the document declares them, including filters this library does not implement. Nothing here guesses.

The operators of a page

`ReadOperationsAsync` reads one page's decoded content stream and reports it as the operators it states, in the order the page paints them:

```C#
var operations = await objectInspector.ReadOperationsAsync(
    source,
    pageNumber: 1,
    cancellationToken: cancellationToken);

foreach (var operation in operations.Items)
{
    Console.WriteLine($"{operation.Offset,6}  {operation.Operator} {string.Join(' ', operation.Operands)}");
}
```

`PdfContentOperationView` has four members: `Operator` (the two-or-three-letter name), `Operands` (the `PdfObjectView` nodes that came before it), `Offset` (where in the decoded stream it starts) and `PageNumber`.

The stream is **tokenized, not interpreted**. An operator this library implements nothing for is reported with its operands exactly like one it does — an audit has to see what the document says, not what the reader understood. Two consequences of that restraint:

- **Inline images are one operation.** `BI` is reported with the entries of its header; its samples are skipped, because samples are not operators, and the image read is where an inline image is described.
- **Nesting is bounded.** An operand that nests deeper than `PdfEngineOptions.MaxNestingDepth` stops being descended, and the read says so once with `ContentNestingTooDeep`. A hostile document cannot turn an inspection into an unbounded walk.

Two edges of the call are deliberate:

- **A page that does not exist is an empty result, not an exception.** "There is no page 500 in a three-page file" is an answer to a question about a document.
- **A page number below one throws `ArgumentOutOfRangeException`.** Page numbers are one-based everywhere in this library (Chapter 20's convention, stated in the preface); zero is a caller's mistake, not a document's.

What the graph is for

Question	Where the answer is
Does this file reference objects it does not contain?	`DanglingReference` warnings, and nodes with `IsResolved` cleared
Which objects did the file list, and how do they hang together?	`ReadObjectsAsync`, walked through `Children`
What does page 3 actually paint, in order?	`ReadOperationsAsync(source, 3)`
Why does a viewer show a page this library says is blank?	The page's `/Contents` stream length and filters, then the operation list
Is the producer writing something this library refuses to execute?	The operations: a `Do`, `BI`, or an action entry the merger copies or drops with a warning

What it is not:

- **It is not a mutation API.** There is no setter, no `Save`, and no way to edit an object and write the document back. Editing is `IPdfDocumentEditor`, and it works on the page assembly, not on objects (Chapter 14).
- **It is not a round-trip parser.** The graph does not preserve the byte layout of what it read, so it cannot be written back as the same file.
- **It is not an interpreter.** The operators are reported; nothing is executed — no JavaScript, no action, no external file (ADR-001).
- **It is not unlimited.** Reading the graph of a large file builds one node per object plus one per entry, so an inspection of a very large document allocates in proportion to its object count. Read it when something is wrong, not on every request; the recorded cost is in `docs/measurements.md`.

The `inspect-objects` example prints the first objects of the sample document, counts the dangling references, and lists the first operators page 1 paints.

23. Digital Signatures

Signing a document adds a signature to it. It does not rebuild the document. The signer appends one revision to the bytes it was given, covering everything that was there before it and its own signature dictionary, and leaves every earlier byte exactly where it was. That is the only shape in which a second signature survives: the first one still covers the bytes it signed, and a reader can tell that it does.

Two questions are answered about a signature, and they are answered separately:

- **Integrity** — do the bytes the signature covers still hash and verify to what the signer signed?
- **Trust** — does the signer's certificate lead to an anchor the caller named?

The two are independent. A document can be intact and signed by a certificate nobody trusts, and it can be signed by a trusted certificate over bytes that were changed afterwards. `IsValid` speaks only about integrity, because that is the half this library can decide on its own.

Signing

```C#
using System.Security.Cryptography.X509Certificates;
using SharpCorePDF.Domain.ValueObjects;
using SharpCorePDF.Infrastructure.Sources;

using var certificate = X509CertificateLoader.LoadPkcs12FromFile(
    "signing.pfx", password: null, X509KeyStorageFlags.EphemeralKeySet);

var options = new PdfSignatureOptions
{
    Certificate = certificate,
    FieldName = "Signature1",
    Reason = "Approved for release",
    Location = "Vienna",
    HashAlgorithm = PdfHashAlgorithm.Sha256,
    Appearance = new PdfSignatureAppearance
    {
        PageNumber = 1,
        Rectangle = new PdfRectangle(340d, 40d, 540d, 100d),
    },
};

await using var output = File.Create("statement-signed.pdf");
var info = await signer.SignAsync(new PdfFileSource("statement.pdf"), output, options);
```

`signer` is an `IPdfSigner`, registered by `AddSharpCorePDF()` and constructible by hand as `new PdfSignerService(new PdfDocumentSigner())` (Chapter 10). The source is opened, read to its end and disposed; the destination belongs to the caller and is flushed, not closed.

The certificate's key decides the signature algorithm — an RSA key signs with PKCS#1 v1.5, an ECDSA key with ECDSA — and `HashAlgorithm` decides the digest. `Chain` embeds the intermediate certificates, so a reader that trusts the root can build the path from the document alone.

`Appearance` is optional. Without it the widget is invisible and zero-sized, which is what a server-side signature usually wants. With it, the text and the border are drawn into a form XObject that travels in the signed revision, so the drawing is covered by the signature rather than added to it later. `Text` defaults to the signer's common name and the signing date. The rectangle is given as its four edges — left, bottom, right and top — in PDF points measured from the bottom left of the page.

The parts of the revision

Part	What it holds
Signature dictionary	`/Filter /Adobe.PPKLite`, `/SubFilter` — `adbe.pkcs7.detached`, or `ETSI.CAdES.detached` when `UseCades` is set — `/ByteRange`, `/Contents`, and the entries a reader shows: `/Reason`, `/Location`, `/ContactInfo`, `/M`.
`/ByteRange`	Four numbers: where the covered bytes start, how many there are, where they resume after the placeholder, and how many follow. The covered bytes are the whole revision except the `/Contents` value.
`/Contents`	The signature value, as a hexadecimal string in a placeholder wide enough for the certificate chain and the token.
The CMS	A detached CMS `SignedData`: the signer's certificate, the digest algorithm, the signed attributes — content type, signing time, message digest, and the signing-certificate attribute of ETSI EN 319 122 when CAdES is asked for — and the signature over them.
The widget	An annotation of `/Subtype /Widget` with `/FT /Sig`, appended to the page's `/Annots`, and the field that names it in the `/AcroForm` of the document.

The order matters, and it is why the value cannot be computed in one pass: the writer reserves the placeholder, hashes the revision with the gap removed, patches `/ByteRange`, and only then writes the signature into the gap. A count that changed after the hash would invalidate the hash.

Signing an archived document

A conversion rewrites a document; a signature appends a revision to the bytes it is handed. That makes the order the whole design: archive first, sign after.

C#

```
// 1. the archival document — Chapter 24
await using (var archival = File.Create("archive-3b.pdf"))
{
    var conversion = await archiveWriter.ConvertAsync(
        new PdfFileSource("master.pdf"), archival, PdfAProfile.PdfA3B,
        new PdfAConversionOptions
        {
            ModificationDate = new DateTimeOffset(2026, 1, 1, 0, 0, 0, TimeSpan.Zero),
            DocumentId = "00112233445566778899AABBCCDDEEFF",
        });

    if (!conversion.IsWritten)
    {
        return;   // conversion.UnsatisfiedRequirements names every requirement the source cannot meet
    }
}

// 2. the signature over the bytes the conversion wrote
await using (var output = File.Create("archive-3b-signed.pdf"))
{
    await signer.SignAsync(
        new PdfFileSource("archive-3b.pdf"),
        output,
        new PdfSignatureOptions
        {
            Certificate = certificate,
            FieldName = "ArchivalSignature",
            Reason = "Archival delivery",
            SigningTime = new DateTimeOffset(2026, 1, 1, 0, 0, 0, TimeSpan.Zero),
        });
}
```

The signed file is the archival file plus one revision, so the structures the profile asks for are still the ones the conversion wrote, and the signature covers all of them — pyHanko reports coverage `ENTIRE_FILE` for exactly this shape. The reverse order is not offered: converting a signed file reads it and writes a new one, which replaces the bytes the signature covers, so the signature would be reported invalid by any reader.

An archival *edit* is the one case where a conversion and a signature do combine in the other order: a `PdfEditPlan` with a profile appends its revision (Chapter 24), so a source that is already signed keeps that signature and gains the archival structure, and the outcome reports `PriorSignaturePreserved`.

The `sign-and-archive` scenario of `examples/SharpCorePDF.Examples` writes this pair — one authored page archived as PDF/A-3b and then signed — with a certificate the run makes and never writes to disk, so the repository carries no key. It prints the integrity and the trust of the result as two verdicts, anchored on that same certificate, and takes a timestamp token from `SCPDF_TIMESTAMP_TOKEN` only when the caller supplies one. On 2026-10-24 pyHanko read its output as field `ArchivalSignature`, `integrity intact` with coverage `ENTIRE_FILE`, by the signer the run made; `docs/parity.md` records the run.

Timestamps

A timestamp is the caller's, never this library's, because obtaining one opens a socket:

```C#
public sealed class MyAuthority : IPdfTimestampProvider
{
    public async Task<ReadOnlyMemory<byte>> GetTimestampTokenAsync(
        ReadOnlyMemory<byte> digest,
        PdfHashAlgorithm hashAlgorithm,
        CancellationToken cancellationToken = default)
    {
        // Ask your TSA for an RFC 3161 token over `digest`, computed with `hashAlgorithm`.
        return await myTsa.StampAsync(digest, hashAlgorithm, cancellationToken);
    }
}
```

Setting `TimestampProvider` hands the digest of the signature value to the provider and embeds the token it answers with as the `signatureTimeStampToken` unsigned attribute. A provider that throws stops the signature: the exception surfaces with the field it was for and no revision is written, so a document that only looks signed cannot be produced.

The token is embedded, not checked. Verifying a TSA's own signature over it needs trust anchors this release does not accept, so `PdfTrustOptions.CheckTimestampTrust` has no effect yet and `PdfSignatureInfo.HasTimestamp` reports only that a token is there — one that cannot be read as a CMS structure becomes a `TimestampInvalid` problem, which is a statement about the token's shape and not about its time.

Validating

```C#
var validations = await validator.ValidateAsync(
    new PdfFileSource("statement-signed.pdf"),
    new PdfTrustOptions { ExtraRoots = [myRoot] });

foreach (var validation in validations)
{
    Console.WriteLine(validation);

    foreach (var problem in validation.Problems)
    {
        Console.WriteLine($"  {problem.Kind}: {problem.Message}");
    }
}
```

`validator` is an `IPdfSignatureValidator`, registered by `AddSharpCorePDF()`. Every signature the document carries comes back, in the order the revisions were appended, which is the order a reader lists them in. A document with no signature yields an empty list; a document that is not a PDF raises `PdfSourceFormatException`, exactly like every other read.

`PdfSignatureValidation` carries the report — `Signer`, a `PdfSignatureInfo` — and the two verdicts:

Integrity	Meaning
`Valid`	The covered bytes hash and verify to what the signature states.
`Invalid`	The covered bytes do not match, or the signature does not verify against the certificate.
`Unknown`	The signature could not be checked at all: unreadable structure, unsupported algorithm, or covered bytes that are not there. Never a success.

`Trust` is `NotEvaluated` unless `PdfTrustOptions` is passed, `Trusted` when the chain reaches one of `ExtraRoots`, and `Untrusted` otherwise — including when `ExtraRoots` is empty, which is how a caller proves a document is signed by a certificate it already knows.

`Problems` explains every verdict. `Kind` is the contract — `DigestMismatch`, `CertificateExpired`, `ChainUntrusted`, `RevisionNotCovered`, `TimestampInvalid`, `FormFieldTreeTooDeep` and the rest — so a caller matches on the kind and logs the `Message`. A `switch` over it keeps a `default` arm, because a later release may add one.

What validation does not do

No machine trust store is consulted, no certificate is fetched, and no authority is asked whether a certificate was revoked. `AllowOnlineRevocationCheck` is a stated boundary rather than a feature: setting it yields a `PdfSignatureProblemKind.OnlineRevocationUnsupported` problem and makes no network call, because a library that quietly contacts a responder during validation is a privacy and a latency problem. Revocation data embedded in the CMS is still reported.

Signing an encrypted document works when the password is passed, and the result stays encrypted. Validation takes the same password for the same reason. Neither rewrites the document, so neither can repair one: a signature that does not verify stays that way.

The form the signatures live in is followed to `PdfEngineOptions.MaxNestingDepth` levels, the limit every other walk uses, so a document whose form nests deeper is answered from the part that was read. That is said rather than left to be assumed: the reader records `PdfWarningKind.FormFieldDepthExceeded`, and every validation carries a `FormFieldTreeTooDeep` problem stating that the search was cut off. Its verdict still describes the signature it names, because the cut is a fact about the document and not about that signature. A signature a viewer shows carries its widget in the page annotations, and those are scanned independently of the tree, so what the cut costs is the fields between the root and the widget — not the signature.

Reading the result

`SignAsync` returns a `PdfSignatureInfo` read back from the file it wrote, so the report describes the document the caller now holds rather than the arguments that produced it: `FieldName`, `SubFilter`, `HashAlgorithm`, `SignerName`, `SignedAt`, `ByteRange`, `SignatureLength`, `CoversWholeDocument` and `CoveredRevision`. Its `ToString()` is the field and the signer, and `PdfSignatureValidation.ToString()` is the same field with the two verdicts:

```text
Signature1: Valid, trust Trusted
```

Once a second signature is appended, the first one no longer covers the whole document: its `CoversWholeDocument` is `false` and its `CoveredRevision` names the revision it signed, while the second reports `true`. That is the expected outcome and not a defect — a signature protects the bytes that existed when it was made.

Evidence

A signature that only its own writer believes is not a signature, so this release is checked by two implementations that share no code with it:

```powershell
uv run --with pyhanko python docs/tools/validate-signatures.py `
  --trust-signer anchor.pem --require-trusted signed.pdf signed-twice.pdf
uv run --with pikepdf python docs/tools/validate-pdf.py signed.pdf signed-twice.pdf
```

pyHanko reports `integrity: intact` and `trust: trusted`, with coverage `ENTIRE_FILE` for a single signature and `ENTIRE_REVISION` followed by `ENTIRE_FILE` for two; qpdf reports no problems on either file. Both scripts live in `docs/tools/`, and `docs/parity.md` records what these runs measured.

24. Archival PDF (PDF/A)

PDF/A is a set of structural rules for documents that must still open in decades. Writing for a profile means writing the structures those rules name — a stated version, an XMP identification of the profile, an output intent, the program of every font, no encryption — and removing or refusing what the profile forbids.

It does not mean the library can tell you the file conforms. That answer comes from a validator, and this release keeps the two apart on purpose:

- the writer writes the structures and refuses what it cannot write correctly;
- veraPDF, an independent implementation, decides whether the result conforms;
- `IsWritten = true` says "the structures are there", never "this is a PDF/A document".

Writing an archival document

```C#
using SharpCorePDF.Domain.ValueObjects;
using SharpCorePDF.Domain.Ports;

using var destination = File.Create("archive-2b.pdf");

var result = await archiveWriter.ConvertAsync(
    new PdfFileSource("statement.pdf"),
    destination,
    PdfAProfile.PdfA2B,
    new PdfAConversionOptions
    {
        ModificationDate = new DateTimeOffset(2026, 1, 1, 0, 0, 0, TimeSpan.Zero),
        DocumentId = "00112233445566778899AABBCCDDEEFF",
    });

if (!result.IsWritten)
{
    foreach (var requirement in result.UnsatisfiedRequirements)
    {
        Console.WriteLine(requirement);
    }
}
```

`archiveWriter` is an `IPdfAWriter`, registered by `AddSharpCorePDF()` and constructible by hand as `new PdfAWriterService(new PdfAWriter())` (Chapter 10). The source is opened, read to its end and disposed; the destination belongs to the caller and is flushed, not closed.

`PdfAProfile` names three profiles, each level B:

Profile	Part of ISO 19005	PDF version written	Transparency	JPEG 2000	Embedded files
`PdfA1B`	19005-1	1.4	refused	refused	refused
`PdfA2B`	19005-2	1.7	allowed	allowed	refused
`PdfA3B`	19005-3	1.7	allowed	allowed	allowed

The profile decides the version of the file, whatever `PdfVersion` a caller might otherwise have asked for; a PDF/A-1b document that claims 1.7 is not one.

`PdfAConversionOptions` carries the document information and the identity of the result, nothing else. A fixed `ModificationDate` and `DocumentId` make two conversions of the same source byte-identical, which is what a test that compares files needs.

What the writer writes

- **The header version** the profile names, 1.4 or 1.7.
- **An XMP packet** in the catalog, uncompressed and UTF-8, carrying `pdfaid:part` and `pdfaid:conformance`, plus the document information mirrored into `dc:title`, `dc:creator`, `dc:description`, `pdf:Keywords`, `pdf:Producer`, `xmp:CreatorTool` and the two dates. A profile requires the dictionary and the packet to agree, so the same values are written to both.
- **An output intent** in the catalog: `/S /GTS_PDFA1`, an output condition identifier of `sRGB IEC61966-2.1`, and a destination profile of `/N 3`. The profile is generated by the library — a complete ICC v2 sRGB program, the same bytes on every platform — so nothing has to be shipped as an asset or licensed. A caller-supplied ICC profile is not part of this release.
- `/NeedAppearances false` on the form, when the source has one: a conformance reader must be able to paint the form without building appearances itself.
- **No encryption.** See below.

Everything else is copied: page content, images, fonts and their programs are written byte for byte, and the page tree is flattened the same way an edit flattens it.

What is removed, and what is refused

Two different things can stand between a source and a profile, and they are treated differently.

Removed, with a warning of kind `PdfAFeatureRemoved`: things whose absence changes no page.

Removed	Why it is safe
The JavaScript name tree of the catalog	A viewer may not run it in an archival document anyway.
A catalog open action that runs JavaScript	The document opens without running anything.
The `/NeedAppearances` flag	The appearances must be in the file, not built by the reader.
Custom document information entries	They have no XMP property without an extension schema.
The XMP packet and output intent of the source	Replaced by the ones the profile requires.

Refused, with `IsWritten = false` and a named requirement: things that cannot be repaired without changing what a page looks like, or that are data.

Refused	Why
A font that is not embedded	There is no program to embed. Standard fonts have none; supply a TrueType face (Chapter 12).
A CID font subset without `/CIDSet`, for 1b	ISO 19005-1 requires the set, and the writer cannot know which CIDs a program it did not subset describes.
Transparency for 1b — a soft mask, a blend mode, a transparency group	PDF 1.4 has no transparency, and making it opaque changes the page.
`JPXDecode` or `LZWDecode` for 1b	The first part of ISO 19005 forbids both filters.
Embedded files for 1b and 2b	Only PDF/A-3 allows attachments.
Optional content for 1b	PDF 1.4 has no layers.
An annotation without an appearance stream (except `Link` and `Popup`)	A viewer would have to invent the content.
A JavaScript, `Launch`, `Sound`, `Movie`, `Rendition`, `ImportData` or `SubmitForm` action	The profile forbids the action itself, not just its startup use.
An external file stream (`/F`)	The document must contain what it displays.

A refusal writes nothing to the destination, so a caller that ignores `IsWritten` does not get a file it might ship by mistake:

```text
PDF/A-2b: nothing written, 1 unsatisfied requirement(s)
the font 'Helvetica' is not embedded; a PDF/A document must carry the program of every font it uses.
```

Every conversion that does write adds a `PdfAProfileWritten` warning naming the profile and stating that conformance is established by an external validator.

Archival edits

An edit can target a profile, which is what an archive wants when it has to fix one page of a master:

```csharp
using SharpCorePDF.Domain.Aggregates;
using SharpCorePDF.Domain.ValueObjects;

var plan = PdfEditPlan.Create(PdfEditOutputOptions.Default with
{
    PdfAProfile = PdfAProfile.PdfA2B,
    TargetVersion = PdfVersion.V1_7,
    ModificationDate = new DateTimeOffset(2026, 1, 1, 0, 0, 0, TimeSpan.Zero),
});

await editor.ApplyAsync(new PdfFileSource("statement.pdf"), plan, destination);
```

The same analysis runs, and an edit that cannot be made conformant is refused with a `PdfEditPlanValidationException` whose errors name every requirement — before the writer opens the destination. `PdfEditResult.OutputVersion` reports the version the profile wrote.

A profile and encryption cannot both be written: `PdfEditOutputOptions.Validate()` reports the conflict and executing such a plan throws before the source is read, because ISO 19005 requires an archival document to be readable without a password. An edit that is not archival may be encrypted; `PdfEditOutputOptions. Encryption` writes AES by default and the writer reads its own output back with that password when `VerifyOutput` is on.

Evidence

Conformance is not a claim this library makes about itself, so the release is checked with veraPDF:

```powershell
python docs/tools/validate-pdfa.py --profile 2b archive-2b.pdf
python docs/tools/validate-pdfa.py archive-1b.pdf archive-3b.pdf
```

The tool finds veraPDF on `PATH`, through `VERAPDF_HOME`, or with `--verapdf`, and needs a Java runtime; `--profile` states the flavour, and without it veraPDF detects the flavour the document declares. It exits 0 when every file passes, 1 when one fails, and 2 when it cannot run at all.

The `archive-pdfa` scenario of `examples/SharpCorePDF.Examples` writes `archive-1b.pdf`, `archive-2b.pdf` and `archive-3b.pdf` from one authored page, and refuses a page set in Helvetica. On 2026-10-22, with veraPDF 1.30.3, all three files and the same three profiles of a real six-page document written by "Microsoft: Print To PDF" reported `PASS`; `docs/parity.md` records the runs. A document whose `Text` annotation carries no appearance stream was refused rather than written, which is the intended behaviour.

The `sign-and-archive` scenario signs what it archived: one authored page written as PDF/A-3b, and then a signature appended to that file (Chapter 23). The order is what makes both promises hold at once — the conversion writes the archival structures, and the signer adds a revision after them rather than rewriting anything — so a validator reads the same structures the conversion wrote, and the signature covers the whole file. Its certificate is made for the run and never written down, which is why the artifact can be produced without a key in the repository.

What this is not

- It is not a validator, and it does not report "PDF/A compliant".
- It is not PDF/UA or PDF/X.
- It does not fetch or embed an ICC profile chosen by the caller, and it writes sRGB only.
- It does not generate missing appearance streams, so a document whose widgets or notes have none is refused rather than silently filled in.
- It does not claim PDF/A-3 associated-file semantics: attachments are carried, but `/AF` relationships are not written.

25. How This Manual Is Built

This manual is produced by the library it documents. The chapters are Markdown files in `docs/manual/src`; the PDF beside them is generated by a console tool in `tools/SharpCorePDF.Manual`, which you can build and run yourself, and no page is typeset by hand. The tool is not packed with the library: it is a maintainer utility, and it is described here because its output is the book you are reading, and because almost everything it does is an application of the surface the earlier chapters describe.

Two facts shape the design. The only third-party part of the build is a browser, and the browser does one thing only: it turns HTML into laid-out pages. Everything that has to know what a PDF *is* — the running footer, the outline, the metadata, and reading the printed file back to measure it — is SharpCorePDF. The builder this replaced needed Python with `markdown`, `pygments` and `pypdf` for the same book; this one needs neither Python nor a PDF package other than this one.

The four stages

Stage	Reads	Writes	Done by
Markdown to HTML	`docs/manual/src/*.md`, `manual.css`	one HTML document	`MarkdownConverter`, `SyntaxHighlighter`, `ManualHtml`
HTML to pages	the HTML document	a printed PDF	Chrome or Edge, headless
Measure	the printed PDF	the page of every heading	`PageIndex` over `PdfContentReaderService`
Finish	the printed PDF and the measurements	the final PDF	`ManualFinisher` over `PdfEditPlan`

Each stage is one file, and each can be stopped so that you can look at what it produced: `--keep-html` leaves the generated HTML on disk, and `--no-stamp` writes the printed file with neither footer nor outline nor metadata.

Markdown to HTML

The converter understands the subset the chapters use and refuses the rest rather than guessing: ATX headings, fenced code with a language, pipe tables, single-line blockquotes, flat lists, inline code, bold, italic, links and horizontal rules. A chapter holds no raw HTML, no nested list and no image, so there is no code for them, and a table is taken as written — this book has no column alignment to interpret. That is a deliberate limit: a converter written for one book is easier to keep correct than a general one, and the chapters are held to it.

Code fences are highlighted into class names, not inline colours — `k` for a keyword, `s` for a string, `c` for a comment, `m` for a number, `n` for a name, `p` for punctuation. Those names are the ones the stylesheet has always styled, which makes the highlighting a contract between the highlighter and the CSS rather than a picture baked into the text: the same book can be restyled without passing the tokens through anything again. C#, shell, PowerShell and JSON have rule sets; any other fence is escaped and printed without colour, with its language as the label above the block.

Every heading gets an anchor that is unique in the book, not only in its chapter. Two chapters that both contain a section called "Overview" get distinct ids, because the contents page and the outline address a heading by its anchor, and a duplicate would send both links to the first one.

The browser lays out, and nothing else

Chrome is started headless with a throwaway profile, and the flag that matters most is `--no-pdf-header-footer`:

```powershell
chrome.exe --headless=new --disable-gpu --disable-extensions --no-pdf-header-footer `
  --run-all-compositor-stages-before-draw --virtual-time-budget=20000 `
  --user-data-dir=<temporary profile> --print-to-pdf=docs\manual\build\pass1.pdf `
  file:///<root>/docs/manual/build/manual.html
```

The browser's own header and footer would print the file URL on one side and the date on the other, which is not what a book carries, and its counter cannot write "Page 7 of 118" in the typeface the stylesheet chose. They are off, and the footer is written afterwards by SharpCorePDF. `--run-all-compositor-stages-before-draw` makes sure the paint that is captured is the final one, and `--virtual-time-budget` gives the layout time to settle before the capture, which matters at this size.

The document is loaded from a `file://` URI, not from a web server, so the stylesheet and the cover logo are read from the repository and the build needs no network. Nothing else about the machine matters: the browser's profile is created inside the build directory and removed after the print.

Measuring the contents

A page number is not known until the page exists, so the first pass prints a middle dot where each number will go. The printed file is then read back, and every heading is located in the text of the pages:

```C#
var probe = await PageIndex.ReadAsync(new PdfByteArraySource(printed, "pass1.pdf"), cancellationToken);

var (pages, _) = PageIndex.Locate(probe.Pages, headings, firstBodyPage);
```

`probe.Pages` holds the whitespace-collapsed text of each page, one string per page. A heading is located by searching forward from the page of the previous heading, and the search starts on the first page after the contents. The two rules do different jobs: searching forward stops a repeated section title from pulling the cursor back into an earlier chapter, and starting after the contents stops a contents row from matching the page it is itself printed on.

The second pass prints the measured numbers. The finished file is then read once more, because a wider row could push a heading to the next page and make the numbers that were just printed stale. The page column is right-aligned with tabular figures and a minimum width, so in practice nothing moves, but the check is cheap: if the measurements changed, the pass is repeated with them, at most twice.

The footer, the outline and the metadata

The last stage is the one that needs a PDF engine, and it is the reason the build is written in C#. For every page from the second onwards a one-page overlay is drawn with `PdfDocumentBuilder` — a rule, a title on the left, "Page N of M" on the right, in Helvetica 8 point — and composited over the page with `PdfEditPlan.Overlay`, the operation Chapter 14 describes:

```C#
var plan = PdfEditPlan.Create();

for (var page = 2; page <= info.PageCount; page++)
{
    var overlay = FooterOverlay(info.Pages[page - 1].Size, page, info.PageCount);
    plan = plan.Overlay(new PdfByteArraySource(overlay, $"footer-{page}.pdf"), page);
}

plan = plan.SetOutline(outline).SetMetadata(metadata);

var editor = new PdfDocumentEditorService(
    new PdfDocumentReader(PdfEngineOptions.Default),
    new PdfEditEngine(PdfEngineOptions.Default));

var finished = await editor.ApplyToArrayAsync(source, plan, cancellationToken: cancellationToken);
```

The cover carries no footer, exactly as page 1 of a book. The footer lives inside the 18mm bottom margin the stylesheet declares; `PdfTypeface.Measure` gives the width of "Page 7 of 118" so the right-hand run ends at the margin instead of drifting, and the two runs are drawn in the muted purple of the book's structure over a hairline of the same hue. One read, one write: the overlay, the outline and the metadata are one plan, so the file is opened once and closed once.

A detail is worth stating because it is easy to get wrong, and because it is the kind of thing a screenshot hides. The canvas an authored page is drawn on counts `y` downwards from the top-left corner, while the page it becomes counts upwards from the bottom-left one; `PdfDocumentBuilder` converts between them for you, which is what lets a page be written in layout coordinates. The footer, however, is defined against the bottom edge, so its baseline is `page.HeightPoints - 30`. Anchor it at `30` and the running footer is printed below the top edge of every page, where it reads as a header. The first build of this tool did exactly that, and it was found by decompressing a page's content stream and reading the text matrix — not by looking at the page, where a footer at the top is easy to mistake for a header the stylesheet meant.

The second half of that fault was not in this tool at all. Chromium writes the page it prints with a transform stated outside a `q` — `.24 0 0 -.24 0 841.92 cm`, which maps its own layout onto the page — and the streams of a page reach a reader as one stream, so the transform was still in force where the overlay was drawn. The footer came out at a quarter of its size, pressed into the top-left corner, where it reads as a short dash of purple above the first line. `PdfEditPlan.Overlay` now frames the original content in `q` / `Q` whenever it draws a layer, so a statement a page leaves open cannot reach what the plan draws after it, and the bytes of the page are still the bytes that were read. Neither fault was visible in a screenshot until the page was measured: a squashed footer looks like a rule, and a footer at the top looks like a header.

The outline is built from the same heading list the contents uses, so a bookmark and a contents row cannot disagree about either the text or the page: both are the heading, and the number measured for it. An `##` becomes a child of the `#` above it, which is why the bookmarks open as a tree in a reader's sidebar. A second list to keep in step was never an option; there is one list, and the contents and the outline are two renderings of it. The metadata — title, author, subject, keywords, creator and producer — is set in the same plan. The creator names the stages ("markdown + Chromium + SharpCorePDF"); the producer names the library, because the bytes were last written by it.

Checking what was written

The last thing the build does is read its own output. The finished file is opened again with this library's reader — not with a second PDF engine — and the properties the build promises are checked before the run is allowed to succeed:

- **It opens, and it kept its pages.** The count the page tree states must equal the number of pages the read returned, so a composition that lost or duplicated a page cannot pass as a finished book.
- **The cover states the edition it was built for.** A build given the wrong `--version`, or one whose cover fell off the front, is a failure rather than a file with a plausible name.
- **The metadata names the book.** Title, subject, author and producer are read back and compared with what the finishing plan set.
- **Every heading is where the contents says.** Each heading's text is searched for on the page that was measured for it, so a contents row cannot point at a page that does not carry its title.
- **The outline and the footer span the document.** One bookmark per heading, "Page 2 of N" on the second page — the cover carries none — and "Page N of N" on the last one.

A problem is printed with the check it failed and the run exits non-zero, so an edition that is mislabelled, truncated or off by a page cannot be committed by accident. The checks read the bytes that were written, because the finishing pass is an edit of a file Chromium produced: checking the intermediate file would prove the browser's work and say nothing about the overlay, the outline or the metadata composed after it.

What the build does not do

- It does not convert the result to PDF/A. The manual is an ordinary PDF 1.7 file, and Chapter 24 explains what converting it would add and what it would refuse.

- It does not check the prose. The self-check proves where a heading landed, not whether the sentence under it is true; a chapter that states a wrong page count builds happily and prints the wrong number.
- It does not run in CI. The tool is compiled with the rest of the solution, but the book is generated on a machine with a browser, before a release, and the release records the page count it produced.
- It does not publish the HTML. GitHub renders the Markdown chapters directly and the PDF is the artifact; the generated HTML is an intermediate file, kept only when `--keep-html` asks for it.
- It is not a general Markdown library. Nothing in it is shaped for another document: a feature this book does not need is a feature the tool does not have, which is why it stays readable.

Running it

POWERSHELL

```
dotnet run --project .\tools\SharpCorePDF.Manual                  # writes docs/SharpCorePDF-Manual-
<version>.pdf
dotnet run --project .\tools\SharpCorePDF.Manual -- --keep-html    # also keeps the generated HTML
dotnet run --project .\tools\SharpCorePDF.Manual -- --no-stamp     # stops after the browser printed
```

Chrome or Edge is the only prerequisite beyond the .NET SDK. `SHARPCOREPDF_CHROME` points the printer at a browser installed somewhere else, and the build fails with a message that says so rather than with a stack trace. The run prints what it did — chapters, headings, pages, bookmarks, the footer, and a `check` line that is `ok` only when every property above held when the written file was read back — and it exits with a non-zero code otherwise, so a build that silently lost its finishing pass cannot be mistaken for a good one.

Argument	What it changes
`--root <path>`	the repository root; found from the tool's location when it is not given
`--out <path>`	the file to write; `docs/SharpCorePDF-Manual-<version>.pdf` by default
`--version <x.y.z>`	the edition the cover, the footer and the metadata state; defaults to the tree's `<Version>` in `src/Directory.Build.props`
`--keep-html`	keep the generated HTML in `docs/manual/build`
`--no-stamp`	skip the footer, the outline and the metadata

That is the whole build: a converter that knows the subset of Markdown this book uses, a browser that lays the pages out, and this library for everything that turns printed pages into a book — a footer on every page but the first, a contents page whose numbers were measured from the file you are holding, bookmarks that agree with it, and the metadata a reader shows in its title bar. None of it is a separate engine, which is the point: the manual is a document this library produced, and the code that produced it is in the repository next to the code it describes.